

一、深度学习简介

1 深度学习概念

- 深度学习是机器学习的一类算法, 以人工神经网络为结构, 可以实现自动提取特征
- 深度学习核心思想是人工神经网络为结构, 自动提取特征

2 深度学习特点

- **多层非线性变换**: 深度学习模型由多个层次组成, 每一层都应用非线性激活函数对输入数据进行变换。较低的层级通常捕捉到简单的特征 (如边缘、颜色等), 而更高的层级则可以识别更复杂的模式 (如物体或面部识别)。
- **自动特征提取**: 与传统机器学习算法不同, 深度学习能够自动从原始数据中学习到有用的特征, 而不需要人工特征工程。这使得深度学习在许多领域中表现出色。
- **大数据和计算能力**: 深度学习模型通常需要大量的标注数据和强大的计算资源 (如GPU) 来进行训练。大数据和高性能计算使得深度学习在图像识别、自然语言处理等领域取得了显著突破。
- **可解释性差**: 深度学习模型内部的运作机制相对不透明, 被称为“黑箱”, 这意味着理解模型为什么做出特定决策可能会比较困难。这对某些应用场景来说是一个挑战。

3 深度学习模型

- **卷积神经网络 (Convolutional Neural Networks, CNN)**:
 - 主要用于图像处理任务, 如图像分类、目标检测、图像分割等。
 - 特点是使用卷积层来自动提取图像中的局部特征, 并通过池化层减少参数数量, 提高计算效率。
- **循环神经网络 (Recurrent Neural Networks, RNN)**:
 - 适用于处理序列数据, 例如自然语言处理 (NLP)、语音识别等。
 - RNN具有记忆功能, 可以处理输入数据的时间依赖性, 但标准RNN难以捕捉长期依赖关系。
- **自编码器 (Autoencoders)**:
 - 一种无监督学习模型, 通常用于降维、特征学习或者异常检测。
 - 自编码器由编码器和解码器两部分组成, 前者将输入压缩成一个较低维度的表示, 后者尝试从这个低维表示重建原始输入。
- **生成对抗网络 (Generative Adversarial Networks, GAN)**:

- 包含两个子网络：生成器和判别器。生成器负责创建看起来真实的假样本，而判别器则试图区分真假样本。
- GAN广泛应用于图像生成、视频合成等领域。
- **Transformer:**
 - 主要用于自然语言处理（NLP）任务，尤其是机器翻译、文本生成等。
 - Transformer摒弃了传统的递归结构，采用自注意力机制（self-attention），使得它能够并行处理整个句子的信息，在机器翻译、文本摘要等任务中表现出色。

4 深度学习应用场景

- ==计算机视觉（Computer Vision）==
 - **图像分类**：将图像分为不同的类别。常用于人脸识别、物体检测等。
 - 自动标注社交媒体照片、医疗影像中的病变检测。
 - **目标检测（Object Detection）**：在图像或视频中定位并分类多个对象。
 - 自动驾驶中的行人检测、监控视频中的入侵检测。
 - **面部识别**：通过分析面部特征进行身份验证或分类。
 - 手机解锁、安防监控系统。
 - **图像生成**：基于输入生成新的图像，如风格转换、图像超分辨率等。
 - 艺术风格迁移、老旧照片修复。
- ==自然语言处理（Natural Language Processing, NLP）==
 - **机器翻译**：使用深度学习模型将一种语言的文本自动翻译成另一种语言。
 - Google翻译、实时语音翻译。
 - **情感分析**：分析文本中的情感倾向，如正面、负面或中性。
 - 社交媒体监控、产品评论分析。
 - **文本生成**：生成符合语法和语义的自然语言文本。
 - 自动写作助手、新闻生成。
 - **语音识别**：将语音转化为文字。
 - 智能助手（如Siri、Alexa）、自动字幕生成。
 - **聊天机器人（Chatbot）**：通过深度学习理解用户输入并生成合理的回应。
 - 客服机器人、虚拟助手（如GPT类模型）。
- ==推荐系统（Recommendation Systems）==
 - **电影、音乐推荐**：根据用户历史的评分和行为，推荐相关的电影、音乐或电视剧。
 - Netflix、Spotify的个性化推荐。
 - **电商推荐**：根据用户的购买历史和浏览习惯推荐商品。

- 亚马逊、淘宝的商品推荐系统。
- **社交媒体推荐**：分析用户的社交行为，推荐相关内容或朋友。
- Facebook、Instagram的内容推荐。

...

二、PyTorch基本使用

1 PyTorch简介

PyTorch是一个基于Python语言的深度学习框架，它将数据封装成张量（Tensor）来进行处理。

PyTorch提供了灵活且高效的工具，用于构建、训练和部署机器学习和深度学习模型。

PyTorch广泛应用于学术研究和工业界，特别是在计算机视觉、自然语言处理、强化学习等领域。



PyTorch的安装：

```
pip install torch -i https://pypi.tuna.tsinghua.edu.cn/simple
```

2 张量简介

PyTorch中的张量就是元素为同一种数据类型的**多维矩阵**。在PyTorch中，张量以"类"的形式封装起来，对张量的一些运算、处理的方法被封装在类中。

PyTorch张量与NumPy数组类似，但PyTorch的张量具有GPU加速的能力（通过CUDA），这使得深度学习模型能够高效地在GPU上运行。

PyTorch提供了对张量的强大支持，可以进行高效的**数值计算**、**矩阵操作**、**自动求导**等。

张量是 PyTorch 中的**核心数据抽象**，PyTorch 支持各种张量子类型。通常地，一维张量称为向量/矢量（vector），二维张量称为矩阵（matrix）。



2.1 基本创建方法

`torch.tensor` 根据指定数据创建张量

`torch.Tensor` 根据形状创建张量，其也可用来创建指定数据的张量

`torch.IntTensor`、`torch.FloatTensor`、`torch.DoubleTensor` 创建指定类型的张量

代码：

"""

案例：

演示张量的基本创建方式。

张量(Tensor)：

PyTorch框架属于最常用的深度学习框架，无论是后续要学的ANN(人工神经网络)，还是CNN(卷积神经网络)，RNN(循环神经网络)

底层在处理数据时，都是使用张量来处理的。

张量 -> 存储同一类型元素的容器，且元素值必须是数值才可以。

张量的基本创建方式：

`torch.tensor` 根据指定数据创建张量

`torch.Tensor` 根据形状创建张量，其也可用来创建指定数据的张量

`torch.IntTensor`、`torch.FloatTensor`、`torch.DoubleTensor` 创建指定类型的张量

细节：

Tensor方式较之于 tensor方式，可以基于形状来直接创建张量。

需要你掌握的方式：

tensor(值，类型)，例如：

`data = np.random.randint(0, 10, size=(2, 3))`

`t3 = torch.tensor(data, dtype=torch.float)`

"""

导包

`import torch`

`import numpy as np`

1. 定义函数，演示：`torch.tensor` 根据指定数据创建张量

`def dm01():`

`# 场景1：标量 张量`

`t1 = torch.tensor(10)`

`print(f't1: {t1}, type: {type(t1)}')`

`print('-' * 30)`

```

# 场景2: 二维列表 -> 张量.
data = [[1, 2, 3], [4, 5, 6]]
t2 = torch.tensor(data)
print(f't2: {t2}, type: {type(t2)}')
print('-' * 30)

# 场景3: numpy nd数组 -> 张量.
data = np.random.randint(0, 10, size=(2, 3))
t3 = torch.tensor(data, dtype=torch.float)
print(f't3: {t3}, type: {type(t3)}')
print('-' * 30)

# 场景4: 尝试直接创建 指定维度(例如: 2行3列的)张量
# t4 = torch.tensor(2, 3) # 报错.
# print(f't4: {t4}, type: {type(t4)}')

```

2. 定义函数, 演示: `torch.Tensor` 根据形状创建张量, 其也可用来创建指定数据的张量

```

def dm02():
    # 场景1: 标量 张量
    t1 = torch.Tensor(10)
    print(f't1: {t1}, type: {type(t1)}')
    print('-' * 30)

    # 场景2: 二维列表 -> 张量.
    data = [[1, 2, 3], [4, 5, 6]]
    t2 = torch.Tensor(data)
    print(f't2: {t2}, type: {type(t2)}')
    print('-' * 30)

    # 场景3: numpy nd数组 -> 张量.
    data = np.random.randint(0, 10, size=(2, 3))
    t3 = torch.Tensor(data)
    print(f't3: {t3}, type: {type(t3)}')
    print('-' * 30)

    # 场景4: 尝试直接创建 指定维度(例如: 2行3列的)张量
    t4 = torch.Tensor(2, 3)
    print(f't4: {t4}, type: {type(t4)}')

```

3. 定义函数, 演示: `torch.IntTensor`、`torch.FloatTensor`、`torch.DoubleTensor` 创建指定类型的张量

```

def dm03():
    # 场景1: 标量 张量

```

```

t1 = torch.IntTensor(10)
print(f't1: {t1}, type: {type(t1)}')
print('-' * 30)

# 场景2: 二维列表 -> 张量.
data = [[1, 2, 3], [4, 5, 6]]
t2 = torch.IntTensor(data)
print(f't2: {t2}, type: {type(t2)}')
print('-' * 30)

# 场景3: numpy nd数组 -> 张量.
data = np.random.randint(0, 10, size=(2, 3))
t3 = torch.IntTensor(data)
print(f't3: {t3}, type: {type(t3)}')
print('-' * 30)

# 场景4: 如果类型不匹配, 会尝试自动转换类型.
data = np.random.randint(0, 10, size=(2, 3))
t4 = torch.FloatTensor(data) # 默认: float32
print(f't4: {t4}, type: {type(t4)}')

# 4. 定义测试函数.
if __name__ == '__main__':
    dm01() # 掌握
    # dm02()
    # dm03()

```

2.2 线性和随机张量

torch.arange() 和 torch.linspace() 创建线性张量
 torch.random.initial_seed() 和 torch.random.manual_seed() 随机种子设置
 torch.rand/randn() 创建随机浮点类型张量
 torch.randint(low, high, size=()) 创建随机整数类型张量

代码:

"""

案例:

演示PyTorch中如何创建 线性 和 随机张量.

涉及到的函数:

torch.arange() 和 torch.linspace() 创建线性张量

`torch.random.initial_seed()` 和 `torch.random.manual_seed()` 随机种子设置

`torch.rand/randn()` 创建随机浮点类型张量

`torch.randint(low, high, size=())` 创建随机整数类型张量

要掌握的函数：

`arange()`, `linspace()`, `manual_seed()`, `randint()`

"""

导包

`import torch`

1. 定义函数，演示：创建线性张量。

`def dm01():`

场景1：创建指定范围的 线性张量。

参1：起始值， 参2：结束值， 参3：步长。

`t1 = torch.arange(0, 10, 2) # 0, 2, 4, 6, 8`

`print(f't1: {t1}, type: {type(t1)}')`

`print('-' * 30)`

场景2：创建指定范围的 线性张量 -> 等差数列。

参1：起始值， 参2：结束值， 参3：元素的个数

`t2 = torch.linspace(1, 10, 4)`

`print(f't2: {t2}, type: {type(t2)}')`

2. 定义函数，演示：创建随机张量。

`def dm02():`

step1：设置随机种子。

`torch.initial_seed()` # 默认采用当前系统的时间戳作为随机种子。

`torch.manual_seed(3)` # 设置随机种子。

step2：创建随机张量。

场景1：均匀分布的(0, 1) 随机张量

`t1 = torch.rand(size=(2, 3))`

`print(f't1: {t1}, type: {type(t1)}')`

`print('-' * 30)`

场景2：符合正态分布的随机张量。

`t2 = torch.randn(size=(2, 3))`

`print(f't2: {t2}, type: {type(t2)}')`

`print('-' * 30)`

场景3：创建随机整数张量。

`t3 = torch.randint(low=1, high=10, size=(3, 5))`

```

        print(f't3: {t3}, type: {type(t3)}')

# 3. 测试函数.
if __name__ == '__main__':
    # dm01()
    dm02()

```

2.3 0/1/指定值张量

torch.ones 和 torch.ones_like 创建全1张量
 torch.zeros 和 torch.zeros_like 创建全0张量
 torch.full 和 torch.full_like 创建全为指定值张量

代码：

```

"""
案例：
    演示如何创建全0，全1，指定值的张量。

涉及到的函数如下：
    torch.ones 和 torch.ones_like 创建全1张量
    torch.zeros 和 torch.zeros_like 创建全0张量
    torch.full 和 torch.full_like 创建全为指定值张量

需要你掌握的函数：
    zeros(), full()
"""

# 导包
import torch

# 场景1: torch.ones 和 torch.ones_like 创建全1张量
t1 = torch.ones(2, 3)  # 创建2行3列全1张量
print(f't1: {t1}, type: {type(t1)}')
print('-' * 30)

# t2: 3行2列
t2 = torch.tensor([[1, 2], [3, 4], [5, 6]])
print(f't2: {t2}, type: {type(t2)}')
print('-' * 30)

# t3 -> 基于t2的形状，创建全1张量。
t3 = torch.ones_like(t2)

```

```

print(f't3: {t3}, type: {type(t3)}')    # 3行2列 -> 全1矩阵
print('*' * 30)

# 场景2: torch.zeros 和 torch.zeros_like 创建全0张量
t1 = torch.zeros(2, 3)    # 创建2行3列全0张量
print(f't1: {t1}, type: {type(t1)}')
print('-' * 30)

# t2: 3行2列
t2 = torch.tensor([[1, 2], [3, 4], [5, 6]])
print(f't2: {t2}, type: {type(t2)}')
print('-' * 30)

# t3 -> 基于t2的形状, 创建全0张量.
t3 = torch.zeros_like(t2)
print(f't3: {t3}, type: {type(t3)}')    # 3行2列 -> 全0矩阵
print('*' * 30)

# 场景3: torch.full 和 torch.full_like 创建全为指定值张量
t1 = torch.full(size=(2, 3), fill_value=255)    # 创建2行3列全255张量
print(f't1: {t1}, type: {type(t1)}')
print('-' * 30)

# t2: 3行2列
t2 = torch.tensor([[1, 2], [3, 4], [5, 6]])
print(f't2: {t2}, type: {type(t2)}')
print('-' * 30)

# t3 -> 基于t2的形状, 创建全255张量.
t3 = torch.full_like(t2, 255)
print(f't3: {t3}, type: {type(t3)}')    # 3行2列 -> 全0矩阵

```

2.4 指定元素类型张量

```

data.type(torch.DoubleTensor)
data.half/double/float/short/int/long()

```

代码:

```

"""
案例:
    创建指定类型的张量.

```

涉及到的函数：

```
type(torch支持的数据类型)
half()/double()/float()/short()/int()/long()
```

你要掌握的函数：

```
type()
"""

# 导包
import torch

# 场景1：直接创建指定类型的张量。
t1 = torch.tensor([1, 2, 3, 4, 5], dtype=torch.float) # 默认是：
float32
print(f't1: {t1}, (元素)类型: {t1.dtype}, (张量)类型: {type(t1)}')
# float32
print('-' * 30)

# 场景2：创建好张量后 -> 做类型转换。
# 思路1：type()函数，推荐掌握。
t2 = t1.type(torch.int16)
print(f't2: {t2}, (元素)类型: {t2.dtype}, (张量)类型: {type(t2)}')
# int16
print('-' * 30)

# 思路2：half()/double()/float()/short()/int()/int()
print(t2.half())      # float16
print(t2.float())     # float32, 默认
print(t2.double())    # float64
print(t2.short())     # int16
print(t2.int())       # int32
print(t2.long())      # int64, 默认
```

2.5 张量的类型转换

2.5.1 张量转换为NumPy数组

使用 `Tensor.numpy` 函数可以将张量转换为 `ndarray` 数组，但是共享内存，可以使用 `copy` 函数避免共享。

2.5.2 NumPy数组转换为张量

使用 `from_numpy` 可以将 `ndarray` 数组转换为 `Tensor`，默认共享内存，使用 `copy` 函数避免共享。

使用 `torch.tensor` 可以将 `ndarray` 数组转换为 `Tensor`，默认不共享内存。

2.5.3 标量张量和数字转换

对于只有一个元素的张量，使用`item()`函数将该值从张量中提取出来

代码：

"""

案例：

演示张量 和 `numpy`之间如何相互转换，以及 如何从标量张量中 提取其内容。

涉及到的API：

场景1：张量 -> `numpy` `nd`数组对象

张量对象.`numpy()` 共享内存

张量对象.`numpy().copy()` 不共享内存，链式编程写法。

场景2：`numpy` `nd`数组 -> 张量。

`from_numpy()` 共享内存

`torch.tensor(nd数组)` 不共享内存

场景3：从标量张量中 提取其内容。

标量张量.`item()`

掌握：

张量 -> `numpy`： 张量对象.`numpy()`

`numpy` -> 张量： `torch.tensor(nd数组)`

从标量张量中 提取其内容： 标量张量.`item()`

"""

`import torch`

`import numpy as np`

1. 定义函数，演示：张量 -> `numpy`

`def dm01():`

1. 创建张量。

`t1 = torch.tensor([1, 2, 3, 4, 5])`

`print(f't1: {t1}, type: {type(t1)}')`

2. 张量 -> `numpy`。

`n1 = t1.numpy()` # 共享内存。

`n1 = t1.numpy().copy()` # 不共享内存。

```
print(f'n1: {n1}, type: {type(n1)}')
```

3. 演示上述方式 是否共享内存.

```
n1[0] = 100
```

```
print(f'n1: {n1}') # [100, 2, 3, 4, 5]
```

```
print(f't1: {t1}') # [?, 2, 3, 4, 5]
```

2. 定义函数, 演示: numpy -> 张量

```
def dm02():
```

1. 创建numpy数组.

```
n1 = np.array([11, 22, 33])
```

```
print(f'n1: {n1}, type: {type(n1)}')
```

2. 把上述的numpy数组, 转换成张量.

t1 = torch.from_numpy(n1).type(torch.float32) # 转换 + 转类型.

```
t1 = torch.from_numpy(n1) # 共享内存.
```

```
print(f't1: {t1}, type: {type(t1)}')
```

```
t2 = torch.tensor(n1) # 不共享内存
```

```
print(f't2: {t2}, type: {type(t2)}')
```

3. 演示上述方式 是否共享内存.

```
n1[0] = 100
```

```
print(f'n1: {n1}') # 100, 22, 33
```

```
print(f't1: {t1}') # 100, 22, 33
```

```
print(f't2: {t2}') # 11, 22, 33
```

3. 定义函数, 演示: 从标量张量(只有1个值的张量) 中 提取其内容.

```
def dm03():
```

1. 创建张量.

```
t1 = torch.tensor(100) # 可以
```

```
# t1 = torch.tensor([100, ]) # 可以
```

```
# t1 = torch.tensor([100, 200]) # 不可以.
```

```
print(f't1: {t1}, type: {type(t1)}')
```

2. 从张量中提取内容.

```
a = t1.item()
```

```
print(f'value: {a}, type: {type(a)}')
```

4. 测试

```
if __name__ == '__main__':
```

```
# dm01()
```

```
# dm02()
```

2.6 张量数值计算

2.6.1 基本运算

加减乘除取负号：

add、sub、mul、div、neg

add_、sub_、mul_、div_、neg_（其中带下划线的版本会修改原数据）

代码：

"""

案例：

演示张量的基本运算。

涉及到的API：

add(), sub(), mul(), div(), neg() -> 加减乘除，取反，
subtract, multiply, divide

add_(), sub_(), mul_(), div_(), neg_() -> 功能同上，只不过可以修改源数据，类似于 Pandas部分的 inplace = True

需要你记忆的：

1. 可以用 +, -, *, / 符号来替代 上述的 加减乘除函数。
2. 如果是张量 和 数值运算，则：该数值会和张量中的每个值依次进行 对应的运算。

"""

导包

import torch

1. 创建张量。

t1 = torch.tensor([1, 2, 3])

2. 演示 加法。

t2 = t1.add(10) # 不会修改源数据。

t2 = t1 + 10 # 效果同上。

t1.add_(10) # 会修改源数据。

t1 += 10 # 效果同上

其它函数效果同上，这里就快速演示了。

t2 = t1.sub(1) # 减法

```
# t2 = t1.mul(2)      # 乘法
# t2 = t1.div(2)      # 除法, [0.5000, 1.0000, 1.5000]
t2 = t1.neg()         # 取反

# 3. 打印结果.
print(f't1: {t1}')
print(f't2: {t2}')
```

2.6.2 点乘运算

点乘指 (Hadamard) 的是相同形状的张量对应位置的元素相乘, 使用 `mul` 和运算符 `*` 实现。

例如:

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, B = \begin{bmatrix} 5 & 6 \\ 7 & 8 \end{bmatrix}$$

则 A, B 的 Hadamard 积:

$$A \circ B = \begin{bmatrix} 1 \times 5 & 2 \times 6 \\ 3 \times 7 & 4 \times 8 \end{bmatrix} = \begin{bmatrix} 5 & 12 \\ 21 & 32 \end{bmatrix}$$

2.6.3 矩阵乘法运算

矩阵乘法运算要求第一个矩阵 `shape: (n, m)`, 第二个矩阵 `shape: (m, p)`, 两个矩阵点积运算 `shape` 为: `(n, p)`。

运算符 `@` 用于进行两个矩阵的乘积运算

`torch.matmul` 对进行乘积运算的两矩阵形状没有限定。对于输入的 `shape` 不同的张量, 对应的最后几个维度必须符合矩阵运算规则

代码:

"""

案例:

演示张量的点乘 和 矩阵乘法操作。

点乘:

要求: 两个张量的维度保持一致, 对应元素直接做 乘法操作。

API:

```
t1 * t2
t1.mul(t2)          # multiply: 乘法
```

矩阵乘法:

要求: 两个张量, 第一个张量 的列数, 等于 第二个张量 的行数(A列 = B行)

结果：A行B列

API:

`t1 @ t2`

`t1.matmul(t2)`

`t1.dot(t2)`

扩展：只针对于一维张量有效。

.....

导包

`import torch`

1. 定义函数，演示张量：点乘

`def dm01():`

1. 定义张量，2行3列。

`t1 = torch.tensor([[1, 2, 3], [4, 5, 6]])`

`print(f't1: {t1}')`

2. 定义张量，2行3列。

`t2 = torch.tensor([[1, 2, 3], [4, 5, 6]])`

`print(f't2: {t2}')`

3. 演示 张量 点乘操作。

`t3 = t1 * t2`

`t3 = t1.mul(t2)` # 效果同上。

4. 打印结果。

`print(f't3: {t3}')`

2. 定义函数，演示张量：矩阵乘法

`def dm02():`

条件：A列=B行，结果：A行B列

1. 定义张量，2行3列。

`t1 = torch.tensor([[1, 2, 3], [4, 5, 6]])`

`print(f't1: {t1}')`

2. 定义张量，3行2列。

`t2 = torch.tensor([[1, 2], [3, 4], [5, 6]])`

`print(f't2: {t2}')`

3. 演示 张量 矩阵乘法操作。

`t3 = t1 @ t2`

`t3 = t1.matmul(t2)` # 效果同上，`matrix multiply`(矩阵乘法)

4. 打印结果。

`print(f't3: {t3}')`

```

# 5. 演示 dot()函数.
# t3 = t1.dot(t2)          # 报错, dot() 只针对于一维张量有效.
t4 = torch.tensor([1, 2, 3])
t5 = torch.tensor([4, 5, 6])
t6 = t4.dot(t5)
print(f't6: {t6}')

# 3. 测试.
if __name__ == '__main__':
    # dm01()
    dm02()

```

2.7 张量运算函数

PyTorch 为每个张量封装很多实用的计算函数:

- 均值
- 平方根
- 求和
- 指数计算
- 对数计算等等

代码:

```

"""
案例:
    演示张量常用的运算函数.
涉及到的 API(函数) 如下:
    sum(), max(), min(), mean()          -> 都有 dim 参
数, 0表示列, 1表示行
    pow(), sqrt(), exp(), log(), log2(), log10()  -> 没有dim参数
掌握的函数:
    sum(), max(), min(), mean()
"""

# 导包
import torch

# 1. 定义张量, 记录初值.
t1 = torch.tensor([
    [1, 2, 3],
    [4, 5, 6]
], dtype=torch.float)
print(f't1: {t1}')

```

2. 演示 有dim参数的 函数.

sum() 求和

```
print(t1.sum(dim=0))      # 按 列 求和
print(t1.sum(dim=1))      # 按 行 求和
print(t1.sum())           # 整 体 求和
print('-' * 30)
```

max()求最大值, min()同理, 这里就不演示了.

```
print(t1.max(dim=0))      # 按 列 求最大值
print(t1.max(dim=1))      # 按 行 求最大值
print(t1.max())           # 整 体 求最大值
print('-' * 30)
```

mean(), 计算平均值

```
print(t1.mean(dim=0))     # 按 列 求平均值
print(t1.mean(dim=1))     # 按 行 求平均值
print(t1.mean())          # 整 体 求平均值
print('*' * 30)
```

3. 演示 没有dim参数的 函数.

pow() n次幂

```
print(t1.pow(2))          # 每个数的平方
print(t1.pow(3))          # 每个数的立方
print(t1 ** 3)            # 效果同上.
print('-' * 30)
```

sqrt() 平方根

```
print(t1.sqrt())          # 每个数的平方根
print('-' * 30)
```

exp() e的n次幂, n就是矩阵中的每个元素, 这里是: e^1, e^2, e^3, e^4, e^5, e^6

```
print(t1.exp())
print('-' * 30)
```

log(), log2(), log10() 对数

```
print(t1.log())
print(t1.log2())
print(t1.log10())
```

2.8 张量索引操作

代码：

```
"""
```

案例：

演示 张量的 索引操作。

分类：

简单行列索引

列表索引

范围索引

布尔索引

多维索引

掌握：

简单行列索引， 范围索引， 多维索引。

```
"""
```

```
# 导包
```

```
import torch
```

```
# 1. 设置随机种子。
```

```
torch.manual_seed(24)
```

```
# 2. 创建随机张量。
```

```
t1 = torch.randint(1, 10, (5, 5))
```

```
print(f't1: {t1}')
```

```
print('-' * 30)
```

```
# 3. 演示张量的索引操作。
```

```
# 场景1：简单行列索引， 格式： 张量对象[行， 列]
```

```
# 需求1：获取第2行的数据。
```

```
print(t1[1])
```

```
print(t1[1, :]) # 效果同上， 这里的:表示 所有列
```

```
# 需求2：所有行的第3列数据
```

```
print(t1[:, 2])
```

```
print('-' * 30)
```

```
# 场景2：列表索引， 前边的表示行， 后边的表示列。
```

```
# 需求1：返回(0, 1)， (1, 2)两个位置的元素。
```

```
print(t1[[0, 1], [1, 2]])
```

```
# 需求2：返回(1, 2)， (3, 4)两个位置的元素。
```

```
print(t1[[1, 3], [2, 4]])
```

```
# 需求3: 获取第0, 1行的 1, 2列共4个元素.
print(t1[[0], [1]], [1, 2])
print('-' * 30)

# 场景3: 范围索引
# 需求1: 前3行, 前2列.
print(t1[:3, :2])

# 需求2: 第2行到最后一行, 前2列的数据.
print(t1[1:, :2])

# 需求3: 所有奇数行, 偶数列.
print(t1[1::2, ::2])
print('-' * 30)

# 场景4: 布尔索引
# print(t1[torch.tensor([True, False, False, True, True]), :])
# 演示布尔写法, 看看就好.

# 需求1: 第3列 大于5的行数据.
print(t1[t1[:, 2] > 5])

# 需求2: 第2行大于5的 列数据.
# 理解1: 在第2行基础上 找到该行中大于5的列索引, 然后找所有行中对应列的元素.
print(t1[:, t1[1, :] > 5])
print(t1[:, t1[1] > 5])      # 效果同上.

# 理解2: 在第2行的基础上, 找该行所有列中大于5的元素
print(t1[1, t1[1, :] > 5])
print('-' * 30)

# 场景5: 多维索引
# 创建3维张量, 即: 2个3行4列的矩阵.
t2 = torch.randint(1, 10, (2, 3, 4))
print(f't2: {t2}')

# 需求1: 获取0轴上的第1个数据.
print(t2[0, :, :])

# 需求2: 获取1轴上的第1个数据.
print(t2[:, 0, :])

# 需求3: 获取2轴上的第1个数据.
```

```
print(t2[:, :, 0])
```

2.9 张量形状操作

reshape()函数

reshape 函数可以在保证张量数据不变的前提下改变数据的维度，将其转换成指定的形状。

squeeze()和**unsqueeze()**函数

squeeze 函数删除形状为 1 的维度（降维），**unsqueeze** 函数添加形状为1的维度（升维）。

transpose()和**permute()**函数

transpose 函数可以实现交换张量形状的指定维度，例如：一个张量的形状为（2，3，4）可以通过 **transpose** 函数把 3 和 4 进行交换，将张量的形状变为（2，4，3）。**permute** 函数可以一次交换更多的维度。

view()和**contiguous()**函数

view 函数也可以用于修改张量的形状，只能用于修改连续的张量。在 **PyTorch** 中，有些张量的底层数据在内存中的存储顺序与其在张量中的逻辑顺序不一致，**view** 函数无法对这样的张量进行变形处理，例如：一个张量经过了 **transpose** 或者 **permute** 函数的处理之后，就无法使用 **view** 函数进行形状操作。

代码：

```
"""
```

案例：

演示张量的形状操作。

涉及到的API：

reshape()：在不改变张量内容的前提下，对其形状做改变。

unsqueeze() 在指定的轴上增加一个(1)维度，等价于：升维。

squeeze() 删除所有为1的维度，等价于：降维。

transpose() 一次只能交换2个维度。

permute() 一次可以同时交换多个维度。

view() 只能修改连续的张量的形状，连续张量 = 内存中存储顺序 和在张量中显示的顺序相同。

contiguous() 把不连续的张量 -> 连续的张量，即：基于张量中显示的顺序，修改内存中的存储顺序。

is_contiguous() 判断张量是否是连续的。

需要你掌握的函数：

reshape()，**unsqueeze()**，**permute()**，**view()**

```
"""
```

```
# 导包
```

```
import torch
```

指定随机种子

```
torch.manual_seed(24)
```

1. 定义函数，演示：reshape()函数。

```
def dm01():
```

1. 定义2行3列的张量。

```
t1 = torch.randint(1, 10, size=(2, 3))
```

```
print(f't1: {t1}, shape: {t1.shape}, row: {t1.shape[0]},  
columns: {t1.shape[1]}, {t1.shape[-1]}')
```

2. 通过reshape()函数，把t1 -> 3行2列，1行6列，6行1列。

```
t2 = t1.reshape(3, 2)
```

t2 = t1.reshape(1, 6)

t2 = t1.reshape(6, 1)

```
print(f't2: {t2}, shape: {t2.shape}, row: {t2.shape[0]},  
columns: {t2.shape[1]}, {t2.shape[-1]}')
```

3. 尝试通过reshape()函数，把t1 -> 2行5列的结果。

t3 = t1.reshape(2, 5) # 报错，转之前共计 2*3=6个元素，转之后共计 2*5=10个元素，不一致。

```
# print(f't3: {t3}')
```

2. 定义函数，演示：unsqueeze()函数，squeeze()函数。

```
def dm02():
```

1. 定义2行3列的张量。

```
t1 = torch.randint(1, 10, size=(2, 3))
```

```
print(f't1: {t1}, shape: {t1.shape}') # (2, 3)
```

2. 在0维上，添加一个维度。

```
t2 = t1.unsqueeze(0)
```

```
print(f't2: {t2}, shape: {t2.shape}') # (1, 2, 3)
```

3. 在1维上，添加一个维度。

```
t3 = t1.unsqueeze(1)
```

```
print(f't3: {t3}, shape: {t3.shape}') # (2, 1, 3)
```

4. 在2维上，添加一个维度。

```
t4 = t1.unsqueeze(2)
```

```
print(f't4: {t4}, shape: {t4.shape}') # (2, 3, 1)
```

5. 在3维上(不存在)，添加一个维度。

t5 = t1.unsqueeze(3) # 报错，越界。

```
# print(f't5: {t5}, shape: {t5.shape}')          # (2, 3, ???,
1)
```

6. 删除所有为1的维度.

```
t6 = torch.randint(1, 10, size=(2, 1, 3, 1, 1))
print(f't6: {t6}, shape: {t6.shape}')          # (2, 1, 3, 1,
1)
```

```
t7 = t6.squeeze()
```

```
print(f't7: {t7}, shape: {t7.shape}')          # (2, 3)
```

3. 定义函数，演示：transpose()函数，permute()函数.

```
def dm03():
```

1. 定义张量.

```
t1 = torch.randint(1, 10, size=(2, 3, 4))
```

```
print(f't1: {t1}, shape: {t1.shape}')
```

```
print('-' * 30)
```

2. 改变维度从 (2, 3, 4) -> (4, 3, 2)

```
# t2 = t1.transpose(0, 2)
```

```
t2 = t1.transpose(0, -1)          # 效果同上.
```

```
print(f't2: {t2}, shape: {t2.shape}')
```

3. 改变维度从 (2, 3, 4) -> (4, 2, 3)

```
t3 = t1.permute(2, 0, 1)
```

```
print(f't3: {t3}, shape: {t3.shape}')
```

4. 定义函数，演示：view()函数，contiguous()函数，is_contiguous()函数.

```
def dm04():
```

思路：演示 view()无法改变 不连续的张量的形状，可以通过
is_contiguous()判断张量是否连续，也可以通过 contiguous()把不连续的张量 ->
连续张量.

1. 定义张量.

```
t1 = torch.randint(1, 10, size=(2, 3))
```

```
print(f't1: {t1}, shape: {t1.shape}')
```

2. 判断张量是否连续. 即：张量中的顺序 和 内存中存储顺序是否一致.

```
# print(t1.is_contiguous())          # True
```

3. 通过 view()函数，修改上述张量的形状. 从(2, 3) -> (3, 2)

```
t2 = t1.view(3, 2)
```

```
print(f't2: {t2}, shape: {t2.shape}')
```

```

print(t2.is_contiguous())          # True

# 4. 通过 transpose() 交换维度 -> 交换之后，不连续了。
t3 = t1.transpose(0, 1)
print(f't3: {t3}, shape: {t3.shape}')
print(t3.is_contiguous())          # False

# 5. 尝试把 t3张量 从(3, 2)，通过 view() 转成 (2, 3)。
# t4 = t3.view(2, 3)              # t3不连续，所以view()无法改变形状。报错。
# print(f't4: {t4}, shape: {t4.shape}')

# 6. 可以通过 contiguous()函数，把 t3张量 -> 连续张量 -> 然后就能通过view修改形状了。
t5 = t3.contiguous().view(2, 3)
print(f't5: {t5}, shape: {t5.shape}')

# 5. 测试
if __name__ == '__main__':
    # dm01()
    # dm02()
    # dm03()
    dm04()

```

2.10 张量拼接操作

`torch.cat()`

`torch.cat()` 函数可以将多个张量根据指定的维度拼接起来，不改变维度数。

`torch.stack()`

`torch.stack()` 函数会在一个新的维度上连接一系列张量，这会增加一个新维度，并且所有输入张量的形状必须完全相同。

代码：

"""

案例：

演示张量的拼接操作。

涉及到的API：

`cat()` 不改变维度数，拼接张量。除了拼接的那个维度外，其它维度数必须保持一致。

`stack()` 会改变维度数，拼接张量。所有的维度都必须保持一致。

"""

```
# 导包
import torch

# 设置随机种子.
torch.manual_seed(24)

# 1. 创建两个张量.
t1 = torch.randint(1, 10, (2, 3))
print(f't1: {t1}, shape: {t1.shape}')

t2 = torch.randint(1, 10, (2, 3))
print(f't2: {t2}, shape: {t2.shape}')

# 2. 演示张量的拼接
# 思路1: cat() 拼接张量.
# t3 = torch.cat([t1, t2], dim=0)      # (2, 3) + (2, 3) = (4, 3)
# print(f't3: {t3}, shape: {t3.shape}')

# t4 = torch.cat([t1, t2], dim=1)      # (2, 3) + (2, 3) = (2, 6)
# print(f't4: {t4}, shape: {t4.shape}')

# t5 = torch.cat([t1, t2], dim=-1)     # 效果同上
# print(f't5: {t5}, shape: {t5.shape}')

# t6 = torch.cat([t1, t2], dim=2) # 报错.
# print(f't6: {t6}, shape: {t6.shape}')
print('-' * 30)

# 思路2: stack() 拼接张量, 可以是新维度, 但是无论新旧维度, 所有维度都必须保持一致.
# t7 = torch.stack([t1, t2], dim=0)    # (2, 3) + (2, 3) = (2, 2, 3)
# print(f't7: {t7}, shape: {t7.shape}')

# t8 = torch.stack([t1, t2], dim=1)    # (2, 3) + (2, 3) = (2, 2, 3)
# print(f't8: {t8}, shape: {t8.shape}')

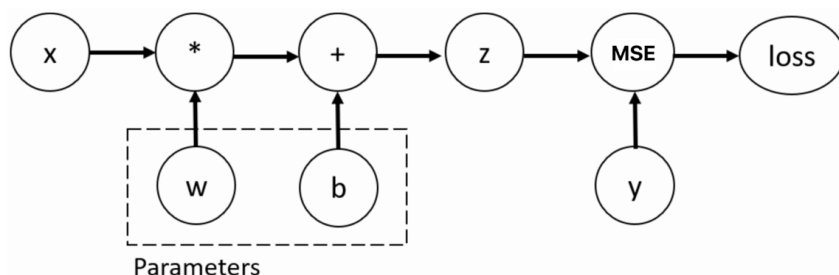
t9 = torch.stack([t1, t2], dim=2)     # (2, 3) + (2, 3) = (2, 3, 2)
print(f't9: {t9}, shape: {t9.shape}')

# 报错.
# t10 = torch.stack([t1, t2], dim=3)   # (2, 3) + (2, 3) = (2, 3, ?, 2)
# print(f't10: {t10}, shape: {t10.shape}')
```

3 自动微分

自动微分模块

训练神经网络时，最常用的算法就是反向传播。在该算法中，参数（模型权重）会根据损失函数关于对应参数的梯度进行调整。为了计算这些梯度，PyTorch内置了名为 `torch.autograd` 的微分模块。它支持任意计算图的自动梯度计算：



PyTorch不支持向量张量对向量张量的求导，只支持标量张量对向量张量的求导

x如果是张量,y必须是标量(一个值)才可以进行求导

计算梯度：`y.backward()`，y是一个标量

获取x点的梯度值：`x.grad`，会累加上一次的梯度值

代码：

.....

案例：

演示自动微分模块，具体如何求导。

回顾：

权重更新公式：

$$w_{\text{新}} = w_{\text{旧}} - \text{学习率} * \text{梯度}$$

$$\text{梯度} = \text{损失函数的导数}$$

关于损失函数的导数，无需我们手动计算，因为非常常用，所以 `PyTorch` 模块内置有自动微分模块，专门实现针对于不同的损失函数求导，

从而实现结合反向传播，更新权重参数 `w` 和偏置参数 `b`

细节：

只有标量张量才能求导，且大多数底层操作的都是浮点型，记得转型。

.....

导包

```
import torch
```

1. 定义变量，记录：初始的权重 `w(旧)`

参1：初始值，参2：是否自动微分(求导)，参3：数据类型

```

w = torch.tensor(10, requires_grad=True, dtype=torch.float)

# 2. 定义loss变量, 表示损失函数.
loss = 2 * w ** 2          # loss = 2w2 -> 求导: 4w

# 3. 打印梯度函数类型(了解)
# print(f'梯度函数类型: {type(loss.grad_fn)}')      # <class
# 'MulBackward0'>
# print(loss.sum())

# 4. 计算梯度, 梯度 = 损失函数的导数, 计算完毕后, 会记录到 w.grad属性中.
loss.sum().backward()      # 保证loss是1个标量.
# loss.backward()          # 这里因为y本身就是标量, 可以不写sum()

# 5. 代入 权重更新公式: w新 = w旧 - 学习率 * 梯度
w.data = w.data - 0.01 * w.grad

# 6. 打印最终结果.
print(f'更新后的权重: {w}')      # 9.6

```

梯度下降法求最优解

- 梯度下降法公式: $w = w - r * grad$ (r是学习率, grad是梯度值)
- 清空上一次的梯度值: `x.grad.zero_()`

代码:

"""

案例:

演示自动微分模块, 循环实现 计算梯度, 更新参数.

需求:

求 $y = x^2 + 20$ 的极小值点 并打印y是最小值时 w的值(梯度)

解题步骤:

1. 定义点 `w=10 requires_grad=True dtype=torch.float32`
2. 定义函数 `loss = w**2 + 20`
3. 利用梯度下降法 循环迭代1000 求最优解
 - 3.1 正向计算(前向传播)
 - 3.2 梯度清零 `w.grad.zero_()`
 - 3.3 反向传播
 - 3.4 梯度更新 `w.data = w.data - 0.01 * w.grad`

回顾:

$w_{\text{新}} = w_{\text{旧}} - \text{学习率} * \text{梯度}$

```

"""
# 导包
import torch

# 1. 定义点 w=10 requires_grad=True dtlosspe=torch.float32
# 参1: 初始值, 参2: 自动微分(求导), 参3: 数据类型, 浮点型.
w = torch.tensor(10, requires_grad=True, dtype=torch.float32)

# 2. 定义函数 loss = w**2 + 20
loss = w ** 2 + 20          # 求导: loss' = 2w

# 3. 利用梯度下降法 循环迭代100 求最优解
print(f'开始 权重初始值: {w}, (0.01 * w.grad): 无, loss: {loss}')
# 10, 无, 120

# 迭代100次, 求最优解.
for i in range(1, 101):
    # 3.1 正向计算(前向传播)
    loss = w ** 2 + 20

    # 3.2 梯度清零 w.grad.zero_() 默认梯度会累加.
    # 至此(第一次的时候), 还没有计算梯度, 所以w.grad = None, 要做非空判断.
    if w.grad is not None:
        w.grad.zero_()

    # 3.3 反向传播
    loss.sum().backward()

    # 3.4 梯度更新 w.data = w.data - 0.01 * w.grad
    # print(f'梯度值为: {w.grad}')
    w.data = w.data - 0.01 * w.grad

    # 3.5 打印本次 梯度更新后的 权重参数结果.
    print(f'第 {i} 次, 权重初始值: {w}, (0.01 * w.grad): {0.01 * w.grad:.5f}, loss: {loss:.5f}')

# 4. 打印最终结果.
print(f'最终结果 权重: {w}, 梯度: {w.grad:.5f}, loss: {loss:.5f}')

```

自动微分模块应用

代码:

```

"""
案例:

```

演示自动微分的真实应用场景。

结论：

1. 先前向传播(正向传播) 计算出 预测值(z)
2. 基于损失函数，结合 预测值(z) 和 真实值(y)，来计算 梯度。
3. 结合权重更新公式 $w_{\text{新}} = w_{\text{旧}} - \text{学习率} * \text{梯度}$ ，来更新 权重。

"""

导包

```
import torch
```

1. 定义x，表示：特征(输入数据)，假设：2行5列，全1矩阵。

```
x = torch.ones(2, 5)
```

```
print(f'x: {x}')
```

2. 定义y，表示：标签(真实值)，假设：2行3列，全0矩阵。

```
y = torch.zeros(2, 3)
```

```
print(f'y: {y}')
```

3. 初始化(可自动微分的)权重 和 偏置。

```
w = torch.randn(5, 3, requires_grad=True)
```

```
print(f'w: {w}')
```

```
b = torch.randn(3, requires_grad=True)
```

```
print(f'b: {b}')
```

4. 前向传播(正向传播)，计算出 预测值(z)。

```
z = torch.matmul(x, w) + b
```

```
# z = x @ w + b
```

```
print(f'z: {z}')
```

5. 定义损失函数。

```
criterion = torch.nn.MSELoss() # neural network: 神经网络
```

```
loss = criterion(z, y) # loss = 损失
```

```
print(f'loss: {loss}')
```

6. 进行自动微分，求导，结合反向传播，更新权重。

```
loss.backward()
```

7. 打印w, b 用来更新的梯度。

```
print(f'w的梯度: {w.grad}')
```

```
print(f'b的梯度: {b.grad}')
```

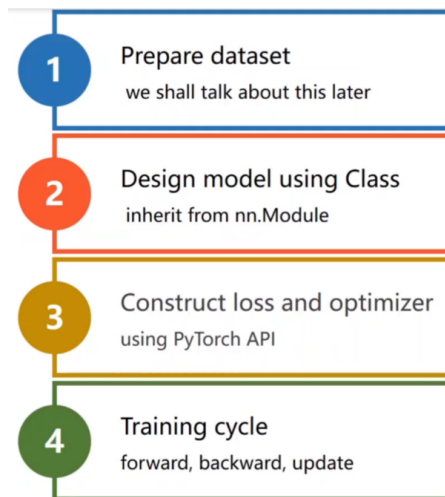
后续就是： $w_{\text{新}} = w_{\text{旧}} - \text{学习率} * \text{梯度}$ ，来更新 权重。

线性回归案例

模型构建流程

我们使用 PyTorch 的各个组件来构建线性回归的实现。在pytorch中进行模型构建的整个流程一般分为四个步骤：

- 准备训练集数据
- 构建要使用的模型
- 设置损失函数和优化器
- 模型训练



- 使用 PyTorch 的 nn.MSELoss() 代替平方损失函数
- 使用 PyTorch 的 data.DataLoader 代替数据加载器
- 使用 PyTorch 的 optim.SGD 代替优化器
- 使用 PyTorch 的 nn.Linear 代替假设函数

代码：

```
# 导入相关模块
import torch
from torch.utils.data import TensorDataset # 构造数据集对象
from torch.utils.data import DataLoader # 数据加载器
from torch import nn # nn模块中有平方损失函数和假设函数
from torch import optim # optim模块中有优化器函数
from sklearn.datasets import make_regression # 创建线性回归模型数据集
import matplotlib.pyplot as plt # 可视化

plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

# 1. 定义函数，创建线性回归样本数据。
def create_dataset():
    # 1. 创建数据集对象。
    x, y, coef = make_regression(
        n_samples=100, # 100条样本(100个样本点)
        n_features=1, # 1个特征(1个特征点)
        noise=10, # 噪声，噪声越大，样本点越散，噪声越小，样本点越集中
```

```

        coef=True, # 是否返回系数, 默认为False, 返回值为None
        bias=14.5, # 偏置
        random_state=3 # 随机种子, 随机种子相同, 输出数据相同
    )

```

2. 把上述的数据, 封装成 张量对象.

```

x = torch.tensor(x, dtype=torch.float32)
y = torch.tensor(y, dtype=torch.float32)

```

3. 返回结果.

```

return x, y, coef

```

2. 定义函数, 表示模型训练.

```

def train(x, y, coef):

```

1. 创建数据集对象. 把 tensor -> 数据集对象 -> 数据加载器.

```

dataset = TensorDataset(x, y)

```

2. 创建数据加载器对象.

参1: 数据集对象, 参2: 批次大小, 参3: 是否打乱数据(训练集打乱, 测试集不打乱)

```

dataloader = DataLoader(dataset, batch_size=16, shuffle=True)

```

3. 创建初始的 线性回归模型.

参1: 输入特征维度, 参2: 输出特征维度.

```

model = nn.Linear(1, 1)

```

4. 创建损失函数对象.

```

criterion = nn.MSELoss()

```

5. 创建优化器对象.

参1: 模型参数, 参2: 学习率.

```

optimizer = optim.SGD(model.parameters(), lr=0.01)

```

6. 具体的训练过程.=

6.1 定义变量, 分别表示: 训练轮数, 每轮的(平均)损失值, 训练总损失值, 训练的样本数.

```

epochs, loss_list, total_loss, total_sample = 100, [], 0.0, 0

```

6.2 开始训练, 按轮训练.

```

for epoch in range(epochs): # epoch的值: 0, 1, 2...99

```

6.3 每轮是分 批次 训练的, 所以从 数据加载器中 获取 批次数据.

```

    for train_x, train_y in dataloader: # 7批(16, 16, 16, 16,
16, 16, 4)

```

6.4 模型预测.

```

        y_pred = model(train_x)

```

6.5 计算(每批的平均)损失值.

```

        loss = criterion(y_pred, train_y.reshape(-1, 1)) #

```

-1 自动计算.

6.6 计算总损失 和 样本(批次)数

```

        total_loss += loss.item()

```

```

        total_sample += 1
        # 6.7 梯度清零 + 反向传播 + 梯度更新.
        optimizer.zero_grad() # 梯度清零
        loss.backward() # 反向传播, 计算梯度
        optimizer.step() # 梯度更新

    # 6.8 把本轮的(平均)损失值, 添加到列表中.
    loss_list.append(total_loss / total_sample)
    print(f'轮数: {epoch + 1}, 平均损失值: {total_loss /
total_sample}')

# 7. 打印(最终的)训练结果.
print(f'{epochs} 轮的平均损失分别为: {loss_list}')
print(f'模型参数, 权重: {model.weight}, 偏置: {model.bias}')

# 8. 绘制损失曲线.
#                               100轮           每轮的平均损失值
plt.plot(range(epochs), loss_list)
plt.title('损失值曲线变化图')
plt.grid() # 绘制网格线
plt.show()

# 9. 绘制预测值和真实值的关系.
# 9.1 绘制样本点分布情况.
plt.scatter(x, y)
# 9.2 绘制训练模型的预测值.
# x: 100个样本点的特征.
y_pred = torch.tensor(data = [v * model.weight + model.bias
for v in x])
# 9.3 计算真实值.
y_true = torch.tensor(data = [v * coef + 14.5 for v in x])
# 9.4 绘制预测值 和 真实值的 折线图.
plt.plot(x, y_pred, color='red', label='预测值')
plt.plot(x, y_true, color='green', label='真实值')
# 9.5 图例, 网格.
plt.legend()
plt.grid()
# 9.6 显示图像.
plt.show()

plt.show()

```

3. 测试.

```

if __name__ == '__main__':
    # 3.1 创建数据集.
    x, y, coef = create_dataset()
    # print(f'x: {x}, y: {y}, coef: {coef}')

    # 3.2 模型训练.
    train(x, y, coef)

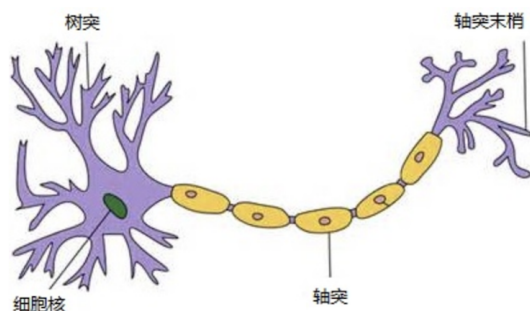
```

三、神经网络基础

1 神经网络

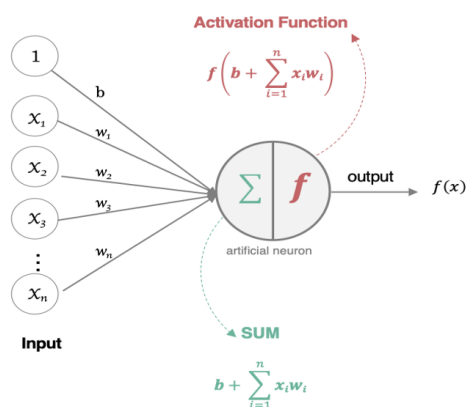
1.1 神经网络简介

人工神经网络（Artificial Neural Network，简称为**ANN**）也简称为神经网络（NN），是一种模仿生物神经网络结构和功能的**计算模型**。人脑可以看做是一个生物神经网络，由众多的**神经元**连接而成。各个神经元传递复杂的电信号，树突接收到**输入信号**，然后对信号进行处理，通过轴突**输出信号**。下图是生物神经元示意图：



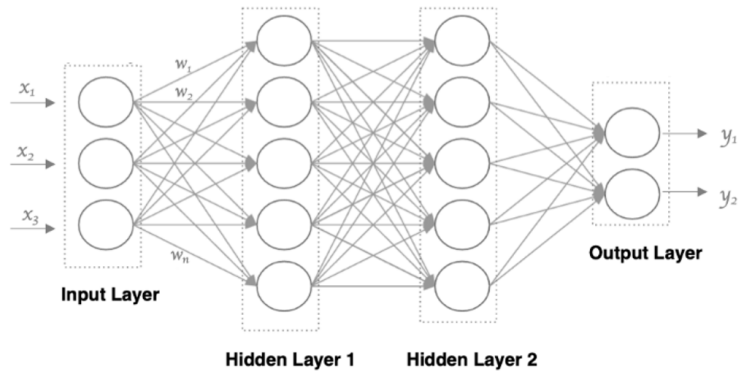
当电信号通过树突进入到细胞核时，会逐渐聚集电荷。达到一定的电位后，细胞就会被激活，通过轴突发出电信号。

神经网络是由多个神经元组成，构建神经网络就是在构建神经元。以下是神经网络中神经元的构建说明：



这个过程就像，来源不同树突(树突都会有不同的权重)的信息，进行的加权计算，输入到细胞中做加和，再通过激活函数输出细胞值。

接下来，我们使用多个神经元来构建神经网络，相邻层之间的神经元相互连接，并给每一个连接分配一个强度，如下图所示：



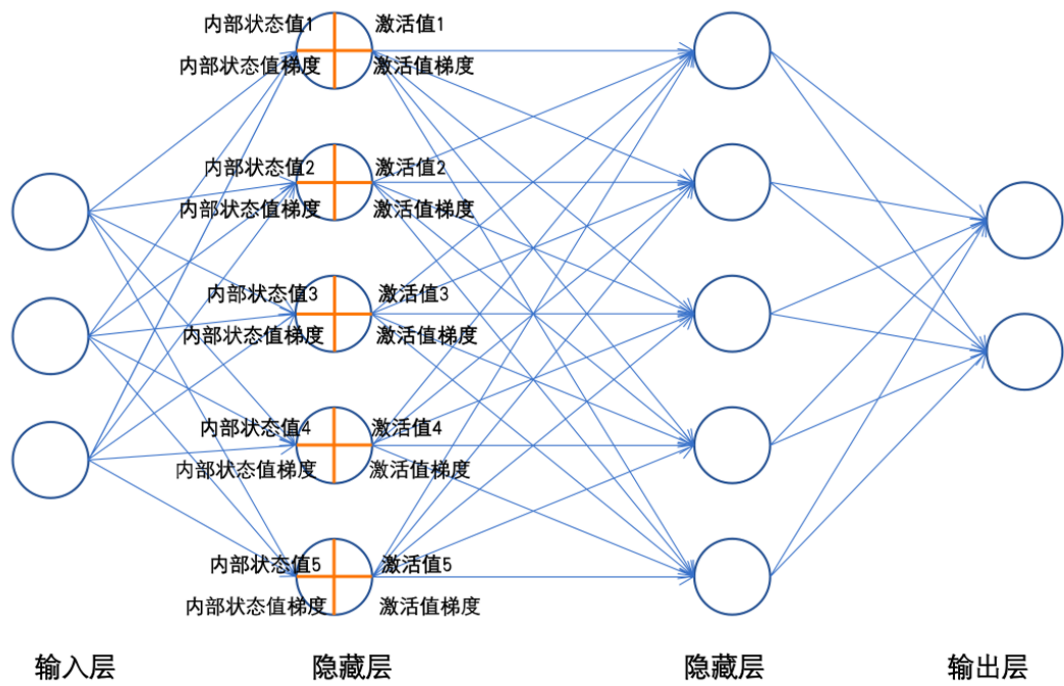
神经网络中信息只向一个方向移动，即从输入节点向前移动，通过隐藏节点，再向输出节点移动。其中的基本部分是：

- 1. **输入层 (Input Layer)**：即输入x的那一层（如图像、文本、声音等）。每个输入特征对应一个神经元。输入层将数据传递给下一层的神经元。
- 2. **输出层 (Output Layer)**：即输出y的那一层。输出层的神经元根据网络的任务（回归、分类等）生成最终的预测结果。
- 3. **隐藏层 (Hidden Layers)**：输入层和输出层之间都是隐藏层，神经网络的“深度”通常由隐藏层的数量决定。隐藏层的神经元通过加权和激活函数处理输入，并将结果传递到下一层。

特点是：

- 同一层的神经元之间没有连接
- 第 N 层的每个神经元和第 N-1层 的所有神经元相连（这就是full connected的含义），这就是全连接神经网络
- 全连接神经网络接收的样本数据是二维的，数据在每一层之间需要以二维的形式传递
- 第N-1层神经元的输出就是第N层神经元的输入
- 每个连接都有一个权重值（w系数和b系数）

神经网络内部状态值和激活值



神经网络内部状态值和激活值

每一个神经元工作时，前向传播会产生两个值，**内部状态值（加权求和值）**和**激活值**；反向传播时会产生激活值梯度和内部状态值梯度。

内部状态值

神经元或隐藏单元的内部存储值，它反映了当前神经元接收到的输入、历史信息以及网络内部的权重计算结果。

$z=W \cdot x+b$

W：权重矩阵

x：输入值

b：偏置

激活值

通过激活函数（如 ReLU、Sigmoid、Tanh）对内部状态值进行非线性变换后得到的结果。激活值决定了当前神经元的输出。

$a=f(z)$

f：激活函数

z：内部状态值

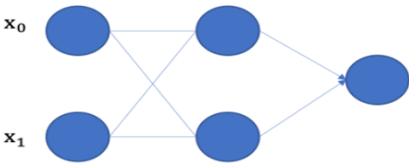
高级数字化人才培养专家

1.2 激活函数

激活函数用于对每层的**输出数据进行变换**,进而为整个网络注入了**非线性因素**。此时,神经网络就可以拟合各种曲线。

- 1.没有引入非线性因素的网络等价于使用一个线性模型来拟合
- 2.通过给网络输出增加激活函数,实现引入非线性因素,使得网络模型可以逼近任意函数,提升网络对复杂问题的拟合能力.

如果不使用激活函数，整个网络虽然看起来复杂，其本质还相当于一种**线性模型**，如下公式所示:



$f_{c1}=w_0^{c1}x_0+w_1^{c1}x_1+b^{c1}$

$f_{c2}=w_0^{c2}x_0+w_1^{c2}x_1+b^{c2}$

$f_{out}=w_0^{out}f_{c1}+w_1^{out}f_{c2}+b^{out}$
 $=w_0^{out}(w_0^{c1}x_0+w_1^{c1}x_1+b^{c1})+w_1^{out}(w_0^{c2}x_0+w_1^{c2}x_1+b^{c2})+b^{out}$
 $=(w_0^{out}w_0^{c1}+w_1^{out}w_0^{c2})x_0+(w_0^{out}w_1^{c1}+w_1^{out}w_1^{c2})x_1+(w_0^{out}b^{c1}+w_1^{out}b^{c2}+b^{out})$
 $k_0=w_0^{out}w_0^{c1}+w_1^{out}w_0^{c2}$
 $k_1=w_0^{out}w_1^{c1}+w_1^{out}w_1^{c2}$
 $c=w_0^{out}b^{c1}+w_1^{out}b^{c2}+b^{out}$
 $=k_0x_0+k_1x_1+c$

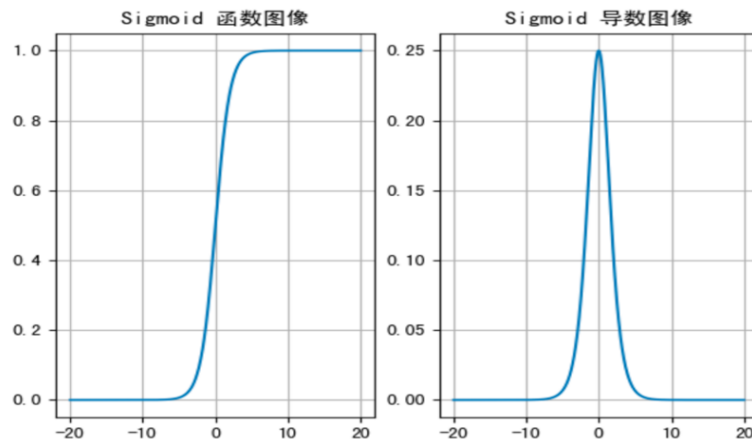
1.2.1 sigmoid函数

激活函数公式：
$$f(x) = \frac{1}{1 + e^{-x}}$$

激活函数求导公式：

$$f'(x) = \left(\frac{1}{1+e^{-x}}\right)' = \frac{1}{1+e^{-x}} \left(1 - \frac{1}{1+e^{-x}}\right) = f(x)(1 - f(x))$$

sigmoid 激活函数的函数图像如下：



- sigmoid 函数可以将任意的输入映射到 (0, 1) 之间，当输入的值大致在 <-6 或者 >6 时，意味着输入任何值得到的激活值都是差不多的，这样会丢失部分的信息。比如：输入 100 和输出 10000 经过 sigmoid 的激活值几乎都是等于 1 的，但是输入的数据之间相差 100 倍的信息就丢失了。
- 对于 sigmoid 函数而言，输入值在 [-6, 6] 之间输出值才会有明显差异，输入值在 [-3, 3] 之间才会有比较好的效果。
- 通过上述导数图像，我们发现导数数值范围是 (0, 0.25)，当输入 <-6 或者 >6 时，sigmoid 激活函数图像的导数接近为 0，此时网络参数将更新极其缓慢，或者无法更新。
- 一般来说，sigmoid 网络在 5 层之内就会产生梯度消失现象。而且，该激活函数并不是以 0 为中心的，所以在实践中这种激活函数使用的很少。sigmoid 函数一般只用于二分类的输出层。

代码：

```
"""
```

案例：

绘制激活函数Sigmoid的 函数图像 和 导数图像。

Sigmoid激活函数介绍：

激活函数的目的：

给模型增加非线性功能，让模型(神经元)既可以做分类，还可以做回归问题。

激活函数的分类：

Sigmoid：

ReLU：

Tanh：

Softmax：

Sigmoid激活函数：

主要应用于 二分类的输出层，且适用于 浅层神经网络(不超过5层)。
数据在 $[-6, 6]$ 之间有效果，在 $[-3, 3]$ 之间效果明显。
求导后范围在 $[0, 0.25]$

细节：

绘制激活函数图像时出现以下提示，需要将 `anaconda3/Lib/site-packages/torch/lib`目录下的`libomp5md.dll`文件删除

OMP: Error #15: Initializing libomp5md.dll, but found libomp5md.dll already initialized.

"""

导包

```
import torch
```

```
import matplotlib.pyplot as plt
```

```
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
```

```
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
```

```
import os
```

```
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
```

1. 创建画布和坐标轴，1行2列。

```
fig, axes = plt.subplots(1, 2)
```

2. 生成 $-20 \sim 20$ 之间的 1000个数据点。

```
x = torch.linspace(-20, 20, 1000)
```

```
# print(f'x: {x}')
```

3. 计算上述1000个点，Sigmoid激活函数处理后的值。

```
y = torch.sigmoid(x)
```

```
# print(f'y: {y}')
```

4. 在第1个子图中绘制Sigmoid激活函数的图像。

```
axes[0].plot(x, y)
```

```
axes[0].set_title('Sigmoid激活函数图像')
```

```
axes[0].grid()
```

5. 在第2个图上，绘制Sigmoid激活函数的导数图像。

5.1 重新生成 $-20 \sim 20$ 之间的 1000个数据点。

参1：起始值，参2：结束值，参3：元素的个数，参4：是否要求导。

```
x = torch.linspace(-20, 20, 1000, requires_grad=True)
```

5.2 具体的计算上述1000个点，Sigmoid激活函数导数后的值。

```
torch.sigmoid(x).sum().backward()
```

5.3 绘制图像.

```
axes[1].plot(x.detach(), x.grad)
axes[1].set_title('Sigmoid激活函数导数图像')
axes[1].grid()
plt.show()
```

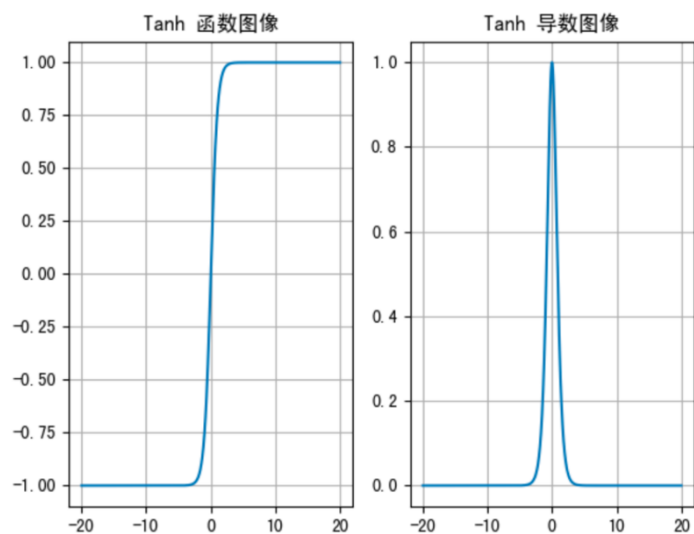
1.2.2 tanh函数

Tanh 的公式如下:

$$f(x) = \frac{1 - e^{-2x}}{1 + e^{-2x}}$$

激活函数求导公式: $f'(x) = \left(\frac{1 - e^{-2x}}{1 + e^{-2x}}\right)' = 1 - f^2(x)$

Tanh 的函数图像、导数图像如下:



- Tanh 函数将输入映射到 **(-1, 1)** 之间，图像以 0 为中心，在 0 点对称，当输入 大概 <-3 或者 >3 时将被映射为 -1 或者 1。其**导数值范围 (0, 1)**，当输入的值大概 <-3 或者 > 3 时，其导数近似 0。
- 与 Sigmoid 相比，它是**以 0 为中心的**，且梯度相对于sigmoid大，使得其收敛速度要比 Sigmoid 快，减少迭代次数。然而，从图中可以看出，Tanh 两侧的导数也为 0，同样会造成梯度消失。
- 若使用时可在**隐藏层使用tanh函数**，在**输出层使用sigmoid函数**。

代码:

.....

案例:

绘制激活函数**Tanh**的 函数图像 和 导数图像。

Sigmoid激活函数介绍:

激活函数的目的:

给模型增加非线性功能，让模型(神经元)既可以做分类，还可以做回归问题。
激活函数的分类：

Sigmoid:

ReLU:

Tanh:

Softmax:

Sigmoid激活函数:

主要应用于 二分类的输出层，且适用于 浅层神经网络(不超过5层)。

数据在 $[-6, 6]$ 之间有效果，在 $[-3, 3]$ 之间效果明显，会将数据值映射到：
 $[0, 1]$

求导后范围在 $[0, 0.25]$

Tanh:

主要应用于 隐藏层，且适用于 浅层神经网络(不超过5层)。

数据在 $[-3, 3]$ 之间有效果，在 $[-1, 1]$ 之间效果明显，会将数据值映射到：
 $[-1, 1]$

求导后范围在 $[0, 1]$ ，较之于Sigmoid，收敛速度快。

细节:

绘制激活函数图像时出现以下提示，需要将 `anaconda3/Lib/site-packages/torch/lib`目录下的`libiomp5md.dll`文件删除

OMP: Error #15: Initializing libiomp5md.dll, but found libiomp5md.dll already initialized.

"""

导包

import torch

import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签

plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

import os

os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"

1. 创建画布和坐标轴，1行2列。

fig, axes = plt.subplots(1, 2)

2. 生成 $-20 \sim 20$ 之间的 1000个数据点。

x = torch.linspace(-20, 20, 1000)

print(f'x: {x}')

3. 计算上述1000个点，Tanh激活函数处理后的值。

y = torch.tanh(x)

```
# print(f'y: {y}')
```

4. 在第1个子图中绘制Tanh激活函数的图像.

```
axes[0].plot(x, y)
axes[0].set_title('Tanh激活函数图像')
axes[0].grid()
```

5. 在第2个图上, 绘制Tanh激活函数的导数图像.

5.1 重新生成 -20 ~ 20之间的 1000个数据点.

参1: 起始值, 参2: 结束值, 参3: 元素的个数, 参4: 是否要求导.

```
x = torch.linspace(-20, 20, 1000, requires_grad=True)
```

5.2 具体的计算上述1000个点, Tanh激活函数导数后的值.

```
torch.tanh(x).sum().backward()
```

5.3 绘制图像.

```
axes[1].plot(x.detach(), x.grad)
axes[1].set_title('Tanh激活函数导数图像')
axes[1].grid()
plt.show()
```

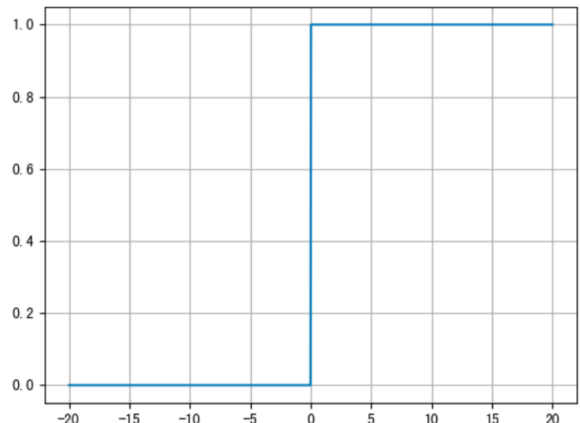
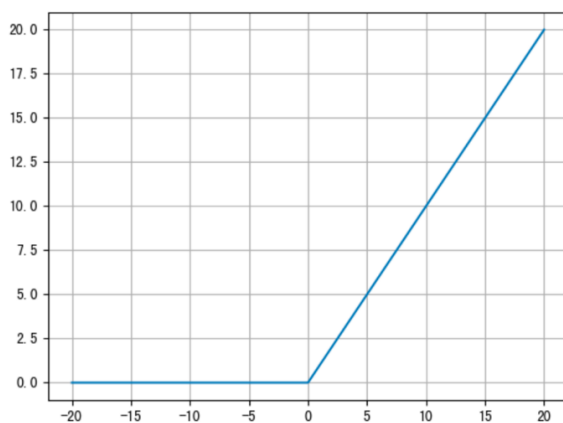
1.2.3 ReLU函数

ReLU 公式如下: $f(x) = \max(0, x)$

激活函数求导公式:

$$f'(x) = 0 \text{ 或 } 1$$

ReLU 的函数图像如下:



- ReLU 激活函数将小于 0 的值映射为 0，而大于 0 的值则保持不变，它更加重视正信号，而忽略负信号，这种激活函数运算更为简单，能够提高模型的训练效率。
- 当 $x < 0$ 时，ReLU导数为0，而当 $x > 0$ 时，则不存在饱和问题。所以，ReLU 能够在 $x > 0$ 时保持梯度不衰减，从而缓解梯度消失问题。然而，随着训练的推进，部分输入会落入小于0区域，导致对应权重无法更新。这种现象被称为“神经元死亡”。
- ReLU是目前最常用的激活函数。与sigmoid相比，ReLU的优势是：

采用sigmoid函数，计算量大（指数运算），反向传播求误差梯度时，计算量相对大，而采用Relu激活函数，整个过程的计算量节省很多。sigmoid函数反向传播时，很容易就会出现梯度消失的情况，从而无法完成深层网络的训练。Relu会使一部分神经元的输出为0，这样就造成了网络的稀疏性，并且减少了参数的相互依存关系，缓解了过拟合问题的发生。

代码：

"""

案例：

绘制激活函数ReLU的 函数图像 和 导数图像。

Sigmoid激活函数介绍：

激活函数的目的：

给模型增加非线性功能，让模型(神经元)既可以做分类，还可以做回归问题。

激活函数的分类：

Sigmoid:

ReLU:

Tanh:

Softmax:

Sigmoid激活函数：

主要应用于 二分类的输出层，且适用于 浅层神经网络(不超过5层)。

数据在 $[-6, 6]$ 之间有效果，在 $[-3, 3]$ 之间效果明显，会将数据值映射到：

$[0, 1]$

求导后范围在 $[0, 0.25]$

Tanh:

主要应用于 隐藏层，且适用于 浅层神经网络(不超过5层)。

数据在 $[-3, 3]$ 之间有效果，在 $[-1, 1]$ 之间效果明显，会将数据值映射到：

$[-1, 1]$

求导后范围在 $[0, 1]$ ，较之于Sigmoid，收敛速度快。

ReLU:

计算公式为： $\max(0, x)$ ，计算量相对较小，训练成本低。多应用于 隐藏层，且适合 深层神经网络。

求导后，值要么是0，要么是1，较之于Tanh，收敛速度更快。

默认情况下ReLU只考虑 正样本，可以使用LeakyReLU，PReLU 来考虑 正负样本。

细节：

绘制激活函数图像时出现以下提示，需要将 `anaconda3/Lib/site-packages/torch/lib`目录下的`libiomp5md.dll`文件删除

OMP: Error #15: Initializing libiomp5md.dll, but found libiomp5md.dll already initialized.

"""

导包

```
import torch
```

```
import matplotlib.pyplot as plt
```

```
plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签
```

```
plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号
```

```
import os
```

```
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
```

1. 创建画布和坐标轴，1行2列。

```
fig, axes = plt.subplots(1, 2)
```

2. 生成 -20 ~ 20之间的 1000个数据点。

```
x = torch.linspace(-20, 20, 1000)
```

```
# print(f'x: {x}')
```

3. 计算上述1000个点，ReLU激活函数处理后的值。

```
y = torch.relu(x)
```

```
# print(f'y: {y}')
```

4. 在第1个子图中绘制ReLU激活函数的图像。

```
axes[0].plot(x, y)
```

```
axes[0].set_title('ReLU激活函数图像')
```

```
axes[0].grid()
```

5. 在第2个图上，绘制ReLU激活函数的导数图像。

5.1 重新生成 -20 ~ 20之间的 1000个数据点。

参1：起始值，参2：结束值，参3：元素的个数，参4：是否需要求导。

```
x = torch.linspace(-20, 20, 1000, requires_grad=True)
```

5.2 具体的计算上述1000个点，ReLU激活函数导数后的值。

```
torch.relu(x).sum().backward()
```

5.3 绘制图像。

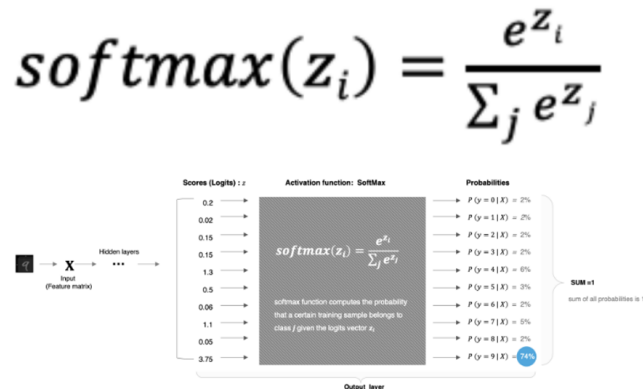
```
axes[1].plot(x.detach(), x.grad)
```

```
axes[1].set_title('ReLU激活函数导数图像')
```

```
axes[1].grid()
plt.show()
```

1.2.4 softmax函数

softmax用于**多分类**过程中，它是二分类函数sigmoid在多分类上的推广，目的是**将多分类的结果以概率的形式展现出来**。计算方法如下图所示：



Softmax 就是将网络输出的 logits 通过 softmax 函数，就映射成为(0,1)的值，而**这些值的累和为1**（满足概率的性质），那么我们将它理解成概率，**选取概率最大（也就是值对应最大的）节点**，作为我们的**预测目标类别**。

代码：

.....

案例：

绘制激活函数**Softmax**的 函数图像 和 导数图像。

Sigmoid激活函数介绍：

激活函数的目的：

给模型增加非线性功能，让模型(神经元)既可以做分类，还可以做回归问题。

激活函数的分类：

Sigmoid：

ReLU：

Tanh：

Softmax：

Sigmoid激活函数：

主要应用于 二分类的输出层，且适用于 浅层神经网络(不超过5层)。

数据在 $[-6, 6]$ 之间有效果，在 $[-3, 3]$ 之间效果明显，会将数据值映射到：

$[0, 1]$

求导后范围在 $[0, 0.25]$

Tanh：

主要应用于 隐藏层，且适用于 浅层神经网络(不超过5层)。

数据在 $[-3, 3]$ 之间有效果, 在 $[-1, 1]$ 之间效果明显, 会将数据值映射到: $[-1, 1]$

求导后范围在 $[0, 1]$, 较之于Sigmoid, 收敛速度快.

ReLU:

计算公式为: $\max(0, x)$, 计算量相对较小, 训练成本低. 多应用于 隐藏层, 且适合 深层神经网络.

求导后, 值要么是0, 要么是1, 较之于Tanh, 收敛速度更快.

默认情况下ReLU只考虑 正样本, 可以使用LeakyReLU, PReLU 来考虑 正负样本.

Softmax:

将多分类的结果以概率的形式展示, 且概率和相加为1, 最终选取概率值最大的分类 作为最终结果.

记忆: 如何选择激活函数

隐藏层:

ReLU > Leaky ReLU > PReLU > Tanh > Sigmoid

输出层:

二分类: Sigmoid

多分类: Softmax

回归问题: identity

细节:

绘制激活函数图像时出现以下提示, 需要将 `anaconda3/Lib/site-packages/torch/lib/libiomp5md.dll` 文件删除

OMP: Error #15: Initializing libiomp5md.dll, but found libiomp5md.dll already initialized.

"""

导包

import torch

import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei'] # 用来正常显示中文标签

plt.rcParams['axes.unicode_minus'] = False # 用来正常显示负号

1. 定义张量, 记录: 分类数据.

scores = torch.tensor([0.2, 0.02, 0.15, 0.15, 1.3, 0.5, 0.06, 1.1, 0.05, 3.75])

scores = torch.tensor([[0.2, 0.35, 0.1, 0.46], [0.1, 0.13, 0.05, 2.79]])

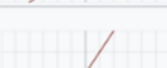
```
# 2. dim = 0, 按行计算
```

```
probabilities = torch.softmax(scores, dim=1)
```

```
print(probabilities)
```

总结

常用激活函数

Name	Plot	Equation	Derivative (with respect to x)	Range
Identity function		$f(x) = x$	$f'(x) = 1$	$(-\infty, \infty)$
Binary step		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x \neq 0 \\ ? & \text{for } x = 0 \end{cases}$	$\{0, 1\}$
Logistic		$f(x) = \frac{1}{1 + e^{-x}}$	$f'(x) = f(x)(1 - f(x))$	$(0, 1)$
TanH		$f(x) = \tanh(x) = \frac{2}{1 + e^{-2x}} - 1$	$f'(x) = 1 - f(x)^2$	$(-1, 1)$
Rectified linear unit (ReLU)		$f(x) = \begin{cases} 0 & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$[0, \infty)$
Leaky ReLU		$f(x) = \begin{cases} 0.01x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(x) = \begin{cases} 0.01 & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$
Parameteric rectified linear unit (PReLU)		$f(\alpha, x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$
Randomized leaky rectified linear unit (RReLU)		$f(\alpha, x) = \begin{cases} \alpha x & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\infty, \infty)$
Exponential linear unit (ELU)		$f(\alpha, x) = \begin{cases} \alpha(e^x - 1) & \text{for } x < 0 \\ x & \text{for } x \geq 0 \end{cases}$	$f'(\alpha, x) = \begin{cases} f(\alpha, x) + \alpha & \text{for } x < 0 \\ 1 & \text{for } x \geq 0 \end{cases}$	$(-\alpha, \infty)$

激活函数选择方法

激活函数的选择方法

对于隐藏层:

1. 优先选择ReLU激活函数
2. 如果ReLU效果不好, 那么尝试其他激活, 如Leaky ReLU等。
3. 如果你使用了ReLU, 需要注意一下Dead ReLU问题, 避免出现0梯度从而导致过多的神经元死亡。
4. 少用使用sigmoid激活函数, 可以尝试使用tanh激活函数

对于输出层:

1. 二分类问题选择sigmoid激活函数
2. 多分类问题选择softmax激活函数
3. 回归问题选择identity激活函数

2 参数初始化

2.1 参数初始化的作用

- 参数 $\rightarrow$ w和b, 创建初版模型时指定w和b的值
- 选择合适的参数, 计算得到的加权求和的值会落到激活函数合理的区间, 加快模型收敛速度以及增加不同学习特征值

2.2 常见的参数初始化的方法

- **随机初始化**

- 均匀分布初始化: 权重参数初始化从区间均匀随机取值, 默认区间为 $(0, 1)$ 。可以设置为在 $(-1/\sqrt{d}, 1/\sqrt{d})$ 均匀分布中生成当前神经元的权重, 其中d为神经元的输入数量。

即在 $(-\frac{1}{\sqrt{d}}, \frac{1}{\sqrt{d}})$ 均匀分布中生成当前神经元的权重,

- 正态分布初始化: 随机初始化从均值为0, 标准差是1的高斯分布中取样, 使用一些很小的值对参数W进行初始化
- **优点:** 能有效打破对称性
- **缺点:** 随机选择范围不当可能导致梯度问题
- **适用场景:** 浅层网络或低复杂度模型。隐藏层1-3层, 总层数不超过5层。
- **全0初始化:** 将神经网络中的所有权重参数初始化为0
 - **优点:** 实现简单
 - **缺点:** 无法打破对称性, 所有神经元更新方向相同, 无法有效训练
 - **适用场景:** 几乎不使用, 仅用于偏置项的初始化
- **全1初始化:** 将神经网络中的所有权重参数初始化为1
 - **优点:** 实现简单
 - **缺点**
 - 无法打破对称性, 所有神经元更新方向相同, 无法有效训练
 - 会导致激活值在网络中呈指数增长, 容易出现梯度爆炸
 - **适用场景**
 - 测试或调试: 比如验证神经网络是否能正常前向传播和反向传播
 - 特殊模型结构: 某些稀疏网络或特定的自定义网络中可能需要手动设置部分参数为1
 - 偏置初始化: 偶尔可以将偏置初始化为小的正值 (如 0.1), 但很少用1作为偏置的初始值
- **固定值初始化:** 将神经网络中的所有权重参数初始化为某个固定值

- **优点**：实现简单
- **缺点**
 - 无法打破对称性，所有神经元更新方向相同，无法有效训练
 - 初始权重过大或过小可能导致梯度爆炸或梯度消失
- **适用场景**
 - 测试或调试
- **kaiming初始化**，也叫做**HE初始化**：专为ReLU和其变体设计，考虑到ReLU激活函数的特性，对输入维度进行缩放
 - HE初始化分为正态分布的HE初始化、均匀分布的HE初始化
 - 正态分布的he初始化
 - w权重值从均值为0, 标准差为std中随机采样, $\text{std} = \sqrt{2 / \text{fan_in}}$
 - std值越大, w权重值离均值0分布相对较广, 计算得到的内部状态值有较大的正值或负值
 - 均匀分布的he初始化
 - 它从[-limit, limit] 中的均匀分布中抽取样本, limit 是 $\sqrt{6 / \text{fan_in}}$
 - fan_in 输入神经元的个数, **当前层接受的来自上一层的神经元的数量**。简单来说, 就是当前层接收多少个输入
 - **优点**：适合 ReLU, 能保持梯度稳定
 - **缺点**：对非 ReLU 激活函数效果一般
 - **适用场景**：深度网络(10层及以上), 使用 ReLU、Leaky ReLU 激活函数
- **xavier初始化**，也叫做**Glorot初始化**：根据网络输入和输出的维度自动选择权重范围，使输入和输出的方差相同
 - xavier初始化分为正态分布的xavier初始化、均匀分布的xavier初始化
 - 正态化的Xavier初始化
 - w权重值从均值为0, 标准差为std中随机采样, $\text{std} = \sqrt{2 / (\text{fan_in} + \text{fan_out})}$
 - std值越小, w权重值离均值0分布相对集中, 计算得到的内部状态值有较小的正值或负值
 - 均匀分布的Xavier初始化
 - [-limit, limit] 中的均匀分布中抽取样本, limit 是 $\sqrt{6 / (\text{fan_in} + \text{fan_out})}$
 - fan_in 是输入神经元个数, **当前层接受的来自上一层的神经元的数量**。简单来说, 就是当前层接收多少个输入

- fan_out 是输出神经元个数，**当前层**输出的神经元的数量，也就是当前层会传递给**下一层**的神经元的数量。简单来说，就是当前层会产生多少个输出。
- **优点**：适用于Sigmoid、Tanh 等激活函数，解决梯度消失问题
- **缺点**：对 ReLU 等激活函数表现欠佳
- **适用场景**：深度网络(10层及以上)，使用 Sigmoid 或 Tanh 激活函数

代码:

"""

案例：

演示参数初始化的 7 种方式。

参数初始化的目的：

1. 防止梯度消失 或者 梯度爆炸。
2. 提高收敛速度。
3. 打破对称性。

参数初始化的方式：

无法打破对称性的：

全0，全1，固定值

可以打破对称性的：

随机初始化，正态分布初始化，kaiming初始化，xavier初始化

总结：

1. 记忆 kaiming初始化，xavier初始化，全0初始化。
2. 关于初始化的选择上：
 - 激活函数ReLU及其系列：优先用 kaiming
 - 激活函数非ReLU：优先用 xavier
 - 如果是浅层网络：可以考虑使用 随机初始化

"""

导包

import torch.nn as nn # neural network: 神经网络

import torch.nn as nn

1. 均匀分布随机初始化

def dm01():

1. 创建1个线性层，输入维度5，输出维度3

linear = nn.Linear(5, 3)

2. 对权重(w)进行随机初始化，从0-1均匀分布产生参数

```
nn.init.uniform_(linear.weight)
# 3. 对偏置(b)进行随机初始化, 从0-1均匀分布产生参数
nn.init.uniform_(linear.bias)
# 4. 打印生成结果.
print(linear.weight.data)
print(linear.bias.data)
```

2. 固定初始化

```
def dm02():
    # 1. 创建1个线性层, 输入维度5, 输出维度3
    linear = nn.Linear(5, 3)
    # 2. 对权重(w)进行初始化, 设置固定值为: 3
    nn.init.constant_(linear.weight, 3)
    # 3. 对偏置(b)进行初始化, 设置固定值为: 3
    nn.init.constant_(linear.bias, 3)
    # 4. 打印生成结果.
    print(linear.weight.data)
    print(linear.bias.data)
```

3. 全0初始化

```
def dm03():
    # 1. 创建1个线性层, 输入维度5, 输出维度3
    linear = nn.Linear(5, 3)
    # 2. 对权重(w)进行初始化, 全0初始化
    nn.init.zeros_(linear.weight)
    # 3. 对偏置(b)进行初始化, 全0初始化
    nn.init.zeros_(linear.bias)
    # 4. 打印生成结果.
    print(linear.weight.data)
    print(linear.bias.data)
```

4. 全1初始化

```
def dm04():
    # 1. 创建1个线性层, 输入维度5, 输出维度3
    linear = nn.Linear(5, 3)
    # 2. 对权重(w)进行初始化, 全1初始化
    nn.init.ones_(linear.weight)
    # 3. 打印生成结果.
    print(linear.weight.data)
```

5. 正态分布随机初始化

```

def dm05():
    # 1. 创建1个线性层，输入维度5，输出维度3
    linear = nn.Linear(5, 3)
    # 2. 对权重(w)进行初始化，正态分布初始化(均值为0，标准差为1)
    nn.init.normal_(linear.weight)
    # 3. 打印生成结果.
    print(linear.weight.data)

# 6. kaiming 初始化
def dm06():
    # 1. 创建1个线性层，输入维度5，输出维度3
    linear = nn.Linear(5, 3)
    # 2. 对权重(w)进行初始化，正态分布初始化(均值为0，标准差为1)
    # kaiming 正态分布初始化
    nn.init.kaiming_normal_(linear.weight)

    # kaiming 均匀分布初始化
    nn.init.kaiming_uniform_(linear.weight)

    # 3. 打印生成结果.
    print(linear.weight.data)

# 7. xavier 初始化
def dm07():
    # 1. 创建1个线性层，输入维度5，输出维度3
    linear = nn.Linear(5, 3)
    # 2. 对权重(w)进行初始化，正态分布初始化(均值为0，标准差为1)
    # xavier 正态分布初始化
    nn.init.xavier_normal_(linear.weight)

    # xavier 均匀分布初始化
    nn.init.xavier_uniform_(linear.weight)

    # 3. 打印生成结果.
    print(linear.weight.data)

# 测试
if __name__ == '__main__':
    # dm01()          # 均匀分布随机初始化

```

# dm02()	# 固定初始化
# dm03()	# 全0初始化
# dm04()	# 全1初始化
# dm05()	# 正态分布
# dm06()	# kaiming初始化
dm07()	# xavier初始化

2.3 参数初始化的选择

激活函数的选择：根据激活函数的类型选择对应的初始化方法

Sigmoid/Tanh: xavier 初始化

ReLU/Leaky ReLU: kaiming 初始化

网络的深度

浅层网络：随机初始化即可

深层网络：需要考虑方差平衡，如 xavier 或 kaiming 初始化

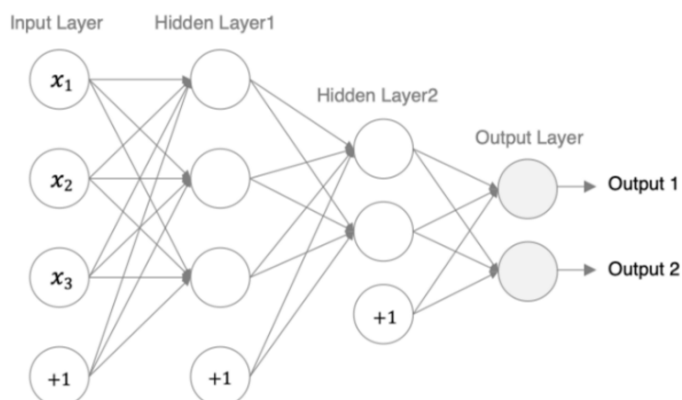
3 神经网络搭建和参数计算

在pytorch中定义深度神经网络其实就是层堆叠的过程，继承自nn.Module，实现两个方法：

`__init__`方法中定义网络中的层结构，主要是全连接层，并进行初始化

`forward`方法，在实例化模型的时候，底层会自动调用该函数。该函数中为初始化定义的layer传入数据，进行前向传播等。

我们来构建如下图所示的神经网络模型：



编码设计如下：

1. 第1个隐藏层：权重初始化采用标准化的xavier初始化 激活函数使用sigmoid
2. 第2个隐藏层：权重初始化采用标准化的He初始化 激活函数采用relu
3. out输出层线性层 假若多分类，采用softmax做数据归一化

代码：

```
"""
```

案例：

演示神经网络搭建流程。

深度学习案例的4个步骤：

1. 准备数据.
2. 搭建神经网络
3. 模型训练
4. 模型测试

神经网络搭建流程：

1. 定义一个类，继承：`nn.Module`
2. 在`__init__()`方法中，搭建神经网络.
3. 在 `forward()`方法中,完成：前向传播.

```
"""
```

```
# 导包
```

```
import torch
```

```
import torch.nn as nn
```

```
from torchsummary import summary # 计算模型参数,查看模型结构, pip  
install torchsummary -i https://mirrors.aliyun.com/pypi/simple/
```

```
# todo: 1.搭建神经网络，即：自定义继承 nn.Module
```

```
class ModelDemo(nn.Module):
```

```
    # todo: 1.1 在init魔法方法中，完成初始化：父类成员，及 神经网络搭建.
```

```
    def __init__(self):
```

```
        # 1.1 初始化父类成员.
```

```
        super().__init__()
```

```
        # 1.2 搭建神经网络 -> 隐藏层 + 输出层
```

```
        # 隐藏层1: 输入特征数 3, 输出特征数 3
```

```
        self.linear1 = nn.Linear(3, 3)
```

```
        # 隐藏层2: 输入特征数 3, 输出特征数 2
```

```
        self.linear2 = nn.Linear(3, 2)
```

```
        # 输出层: 输入特征数 2, 输出特征数 2
```

```
        self.output = nn.Linear(2, 2)
```

```
        # 1.3 对隐藏层进行参数初始化.
```

```
        # 隐藏层1
```

```
        nn.init.xavier_normal_(self.linear1.weight)
```

```
        nn.init.zeros_(self.linear1.bias)
```

```
        # 隐藏层2
```

```

nn.init.kaiming_normal_(self.linear2.weight)
nn.init.zeros_(self.linear2.bias)

# todo: 1.2 前向传播: 输入层 -> 隐藏层 -> 输出层
def forward(self, x):
    # 1.1 第1层 隐藏层计算: 加权求和 + 激活函数(Sigmoid)
    # 分解版写法.
    # x = self.linear1(x)      # 加权求和
    # x = torch.sigmoid(x)    # 激活函数

    # 合并版写法.
    x = torch.sigmoid(self.linear1(x))

    # 1.2 第2层 隐藏层计算: 加权求和 + 激活函数(ReLU)
    x = torch.relu(self.linear2(x))

    # 1.3 第3层 输出层计算: 加权求和 + 激活函数(Softmax)
    # dim=-1, 表示按行计算, 一条样本一条样本的处理.
    x = torch.softmax(self.output(x), dim=-1)

    # 1.4 返回预测值.
    return x

# todo: 2.模型训练.
def train():
    # 1. 创建模型对象.
    my_model = ModelDemo()
    # print(f'my_model: {my_model}')

    # 2. 创建数据集样本, 随机生成.
    data = torch.randn(size=(5, 3))
    print(f'data: {data}')
    print(f'data.shape: {data.shape}') # (5行, 3
列)
    print(f'data.requires_grad: {data.requires_grad}') # False

    # 3. 调用神经网络模型 -> 进行模型训练.
    output = my_model(data) # 底层自动调用了 forward()方法, 进行
前向传播.
    print(f'output: {output}')
    print(f'output.shape: {output.shape}') # (5
行, 2列)
    print(f'output.requires_grad: {output.requires_grad}') #
True

```

```

print('-' * 30)

# 4. 计算 和 查看模型参数.
print('===== 计算模型参数 =====')
# 参1: (神经网络)模型对象, 参2: 输入数据维度(5行3列)
summary(my_model, input_size=(5, 3))

print('===== 查看模型参数 =====')
for name, param in my_model.named_parameters():
    print(f'name: {name}')
    print(f'param: {param} \n')

# todo: 3.测试
if __name__ == '__main__':
    train()

```

小结:

1.神经网络的搭建方法

- 定义继承自nn.Module的模型类
- 在init方法中定义网络中的层结构
- 在forward方法中定义数据传输方式

2.网络参数量的统计方法

- 统计每一层中的权重w和偏置b的数量

神经网络的优缺点

1.优点

- 精度高，性能优于其他的机器学习方法，甚至在某些领域超过了人类
- 可以近似任意的非线性函数
- 近年来在学界和业界受到了热捧，有大量的框架和库可供调。

2.缺点

- 黑箱，很难解释模型是怎么工作的
- 训练时间长，需要大量的计算力
- 网络结构复杂，需要调整超参数
- 小数据集上表现不佳，容易发生拟合

4 损失函数

4.1 分类任务损失函数

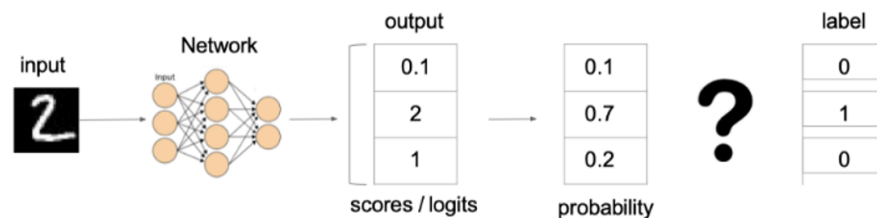
4.1.1 多分类任务损失函数

在多分类任务通常使用softmax将logits转换为概率的形式，所以多分类的交叉熵损失也叫做**softmax损失**，它的计算方法是：

$$\mathcal{L} = - \sum_{i=1}^n \overset{\text{labels (one-hot)}}{y_i} \log(\underset{\text{Softmax}}{S(f_{\theta}(\mathbf{x}_i)))})$$

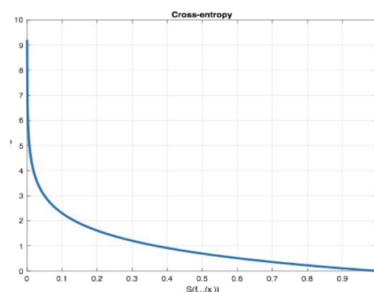
其中：

1. y 是样本 x 属于某一个类别的真实概率
2. 而 $f(x)$ 是样本属于某一类别的预测分数
3. S 是softmax激活函数,将属于某一类别的预测分数转换成概率
4. L 用来衡量真实值 y 和预测值 $f(x)$ 之间差异性的损失结果



上图中的交叉熵损失为: $-(0\log(0.10) + 1\log(0.7) + 0\log(0.2)) = -\log 0.7$

从概率角度理解, 我们的目的是最小化正确类别所对应的预测概率的对数的负值(损失值最小), 如下图所示:



高级数

代码:

.....

案例:

演示 多分类任务的交叉熵损失函数.

损失函数介绍:

概述:

损失函数也叫成本函数, 目标函数, 代价函数, 误差函数, 就是用来衡量 模型好坏(模型拟合情况)的.

分类:

分类问题:

多分类交叉熵损失: **CrossEntropyLoss**

二分类交叉熵损失: **BCELoss**

回归问题:

MAE: Mean Absolute Error, 平均绝对误差.

MSE: Mean Squared Error, 均方误差.

Smooth L1: 结合上述两个的特点做的升级, 优化.

多分类交叉熵损失: **CrossEntropyLoss**

设计思路:

$Loss = - \sum y \log(S(f(x)))$

简单记忆:

x: 样本

f(x): 加权求和

S(f(x)): 处理后的概率

y: 样本x属于某一个类别的 真实概率.

大白话解释:

损失函数结果 = 最小化 正确类别所对应的 预测概率的对数的 负值(损失值最小)...

细节：

`CrossEntropyLoss = Softmax()` + 损失计算，后续如果用这个损失函数，则：输出层就不用额外调用 `softmax()` 激活函数了。

"""

导包

import torch

import torch.nn as nn

1. 定义函数，演示：多分类交叉熵损失。

def dm01():

1. 手动创建样本的真实值 -> 就是上述公式中的 y

y_true = torch.tensor([[0, 1, 0], [1, 0, 0]],
dtype=torch.float)

y_true = torch.tensor([1, 2])

2. 手动创建样本的预测值 -> 就是上述公式中的 $f(x)$

y_pred = torch.tensor([[0.1, 0.8, 0.1], [0.7, 0.2, 0.1]],
requires_grad=True, dtype=torch.float)

3. 创建多分类交叉熵损失函数。

criterion = nn.CrossEntropyLoss() # 平均损失，来源于参数：
reduction: str = "mean",

4. 计算损失值。

loss = criterion(y_pred, y_true)

print(f'损失值: {loss}')

2. 测试

if __name__ == '__main__':

dm01()

4.1.2 二分类任务损失函数

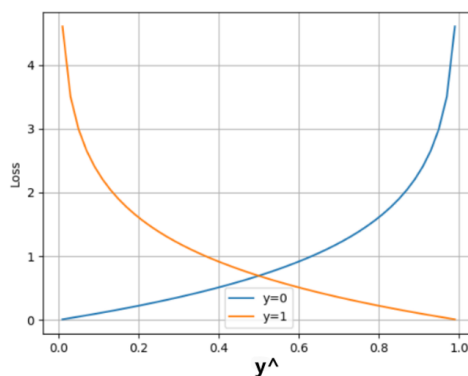
在处理二分类任务时，我们不再使用softmax激活函数，而是使用sigmoid激活函数，那损失函数也相应的进行调整，使用二分类的交叉熵损失函数：

$$L = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

其中：

1. y 是样本 x 属于某一个类别的真实概率
2. 而 $y^{\wedge}$ 是样本属于某一类别的预测概率
3. L 用来衡量真实值 y 与预测值 $y^{\wedge}$ 之间差异性的损失结果。

在pytorch中实现时使用nn.BCELoss()，如下所示：



代码：

```
"""
```

案例：

演示二分类任务的损失函数。

二分类任务的损失函数(BCELoss)：

公式：

$$\text{Loss} = -y \log(\text{预测值}) - (1 - y) \log(1 - \text{预测值})$$

细节：

因为公式中没有包含Sigmoid激活函数，所以使用BCELoss的时候，还需要手动指定 Sigmoid。

```
"""
```

```
# 导包
```

```
import torch
```

```
import torch.nn as nn
```

```
# 1. 定义函数，演示：二分类任务的损失函数。
```

```
def dm01():
```

```
    # 1. 设置真实值。
```

```
    y_true = torch.tensor([0, 1, 0], dtype=torch.float)
```

```
    # 2. 设置预测值(概率)
```

```
    y_pred = torch.tensor([0.6901, 0.5423, 0.2639])
```

```
    # 3. 创建二分类交叉熵损失函数。
```

```
    criterion = nn.BCELoss()    # reduction: str = "mean" -> 均值
```

```
    # 4. 计算损失值。
```

```
    loss = criterion(y_pred, y_true)
```

```
print(f'损失值: {loss}')
```

```
# 2. 测试
```

```
if __name__ == '__main__':  
    dm01()
```

4.2 回归任务损失函数

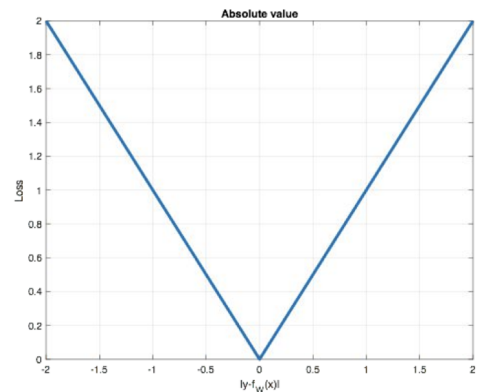
4.2.1 MAE损失函数

Mean absolute loss(MAE)也被称为L1 Loss, 是以绝对误差作为距离。损失函数公式:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n |y_i - f_{\theta}(x_i)|$$

特点是:

1. 由于L1 loss具有稀疏性, 为了惩罚较大的值, 因此常常将其作为正则项添加到其他loss中作为约束。
2. L1 loss的最大问题是梯度在零点不平滑, 导致会跳过极小值。



4.2.2 MSE损失函数

Mean Squared Loss/ Quadratic Loss(MSE loss)也被称为L2 loss, 或欧氏距离, 它以误差的平方和的均值作为距离

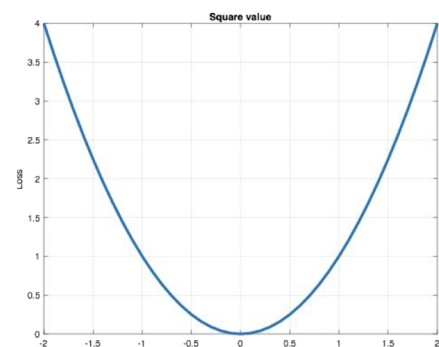
损失函数公式:

$$\mathcal{L} = \frac{1}{n} \sum_{i=1}^n (y_i - f_{\theta}(x_i))^2$$

曲线如下图所示:

特点是:

1. L2 loss也常常作为正则项。
2. 当预测值与目标值相差很大时, 梯度容易爆炸。



4.2.3 Smooth L1损失函数

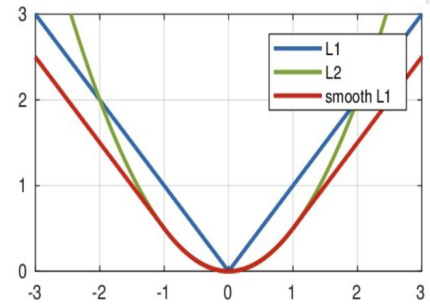
Smooth L1说的是光滑之后的L1。损失函数公式:

$$\text{smooth}_{L_1}(x) = \begin{cases} 0.5x^2 & \text{if } |x| < 1 \\ |x| - 0.5 & \text{otherwise} \end{cases}$$

其中: $x=f(x)-y$ 为真实值和预测值的差值。

从右图中可以看出, 该函数实际上就是一个分段函数

1. 在 $[-1,1]$ 之间实际上就是L2损失, 这样解决了L1的不光滑问题
2. 在 $[-1,1]$ 区间外, 实际上就是L1损失, 这样就解决了离群点梯度爆炸的问题



代码:

"""

案例:

演示 回归任务的损失函数介绍.

回归任务常用损失函数如下:

MAE: Mean Absolute Error, 平均绝对误差.

公式:

误差绝对值之和 / 样本总数

类似于**L1**正则化, 权重可以降低**0**, 数据会变得稀疏.

弊端:

在**0**点不平滑, 可能错过最小值.

MSE: Mean Squared Error, 均方误差.

公式:

误差平方之和 / 样本总数

弊端:

如果差值过大, 可能存在梯度爆炸的情况.

Smooth L1:

就是基于**MAE** 和 **MSE**做的综合, 在 $[-1, 1]$ 是 **L2(MSE)**, 其它段时**L1**.

这样即解决了**L1**不平滑的问题(**0**点不可导, 可能错过最小值)

又解决了**L2(MSE)**的 梯度爆炸的问题.

"""

导包

import torch

import torch.nn as nn

1. 定义函数, 演示: **MAE** 损失函数.

def dm01():

```
# 1. 定义变量, 记录: 真实值.
y_true = torch.tensor([2.0, 2.0, 2.0], dtype=torch.float)

# 2. 定义变量, 记录: 预测值.
y_pred = torch.tensor([1.0, 1.0, 1.9], requires_grad=True)

# 3. 创建MAE损失函数对象.
criterion = nn.L1Loss()

# 4. 计算损失.
loss = criterion(y_pred, y_true)

# 5. 输出损失.
print(f'MAE: {loss}')
```

2. 定义函数, 演示: MSE 损失函数.

```
def dm02():
    # 1. 定义变量, 记录: 真实值.
    y_true = torch.tensor([2.0, 2.0, 2.0], dtype=torch.float)

    # 2. 定义变量, 记录: 预测值.
    y_pred = torch.tensor([1.0, 1.0, 1.9], requires_grad=True)

    # 3. 创建MSE损失函数对象.
    criterion = nn.MSELoss()

    # 4. 计算损失.
    loss = criterion(y_pred, y_true)

    # 5. 输出损失.
    print(f'MSE: {loss}')
```

3. 定义函数, 演示: Smooth L1 损失函数.

```
def dm03():
    # 1. 定义变量, 记录: 真实值.
    y_true = torch.tensor([2.0, 2.0, 2.0], dtype=torch.float)

    # 2. 定义变量, 记录: 预测值.
    y_pred = torch.tensor([1.0, 1.0, 1.9], requires_grad=True)

    # 3. 创建Smooth L1损失函数对象.
    criterion = nn.SmoothL1Loss()
```

```
# 4. 计算损失.  
loss = criterion(y_pred, y_true)
```

```
# 5. 输出损失.  
print(f'Smooth L1: {loss}')
```

```
# 4. 测试  
if __name__ == '__main__':  
    # dm01()      # 0.6999999988079071  
    # dm02()      # 0.6700000166893005  
    dm03()        # 0.33500000834465027
```

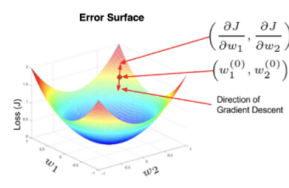
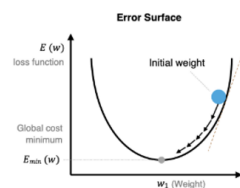
5 网络优化方法

5.1 梯度下降算法

梯度下降法是一种寻找使损失函数最小化的方法。从数学角度来看，梯度的方向是函数增长速度最快的方向，那么梯度的反方向就是函数减少最快的方向，所以有：

$$w_{ij}^{new} = w_{ij}^{old} - \eta \frac{\partial E}{\partial w_{ij}}$$

其中， η 是学习率，如果学习率太小，那么每次训练之后得到的效果都太小，增大训练的时间成本。如果，学习率太大，那就有可能直接跳过最优解，进入无限的训练中。解决的方法就是，学习率也需要随着训练的进行而变化。



在进行模型训练时，有三个基础的概念：

1. **Epoch**: 使用全部数据对模型进行一次完整训练，训练轮次
2. **Batch_size**: 使用训练集中的小部分样本对模型权重进行以此反向传播的参数更新，每次训练每批次样本数量
3. **Iteration**: 使用一个 Batch 数据对模型进行一次参数更新的过程

假设数据集有 50000 个训练样本，现在选择 Batch Size = 256 对模型进行训练。

每个 Epoch 要训练的图片数量：50000

训练集具有的 Batch 个数：50000/256+1=196

每个 Epoch 具有的 Iteration 个数：196

10个 Epoch 具有的 Iteration 个数：1960

在深度学习中，梯度下降的几种方式的根本区别就在于 Batch Size不同,如下表所示：

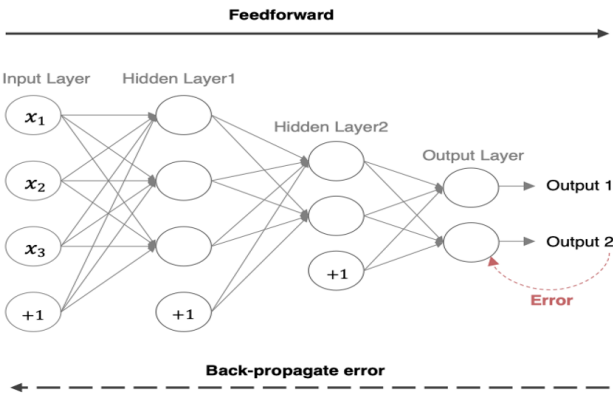
梯度下降方式	Training Set Size	Batch Size	Number of Batches
BGD	N	N	1
SGD	N	1	N
Mini-Batch	N	B	$N/B + 1$

注：上表中 Mini-Batch 的 Batch 个数为 $N / B + 1$ 是针对未整除的情况。整除则是 N / B 。

5.2 反向传播算法

前向传播：指的是数据输入到神经网络中，逐层向前传输，一直运算到输出层为止。

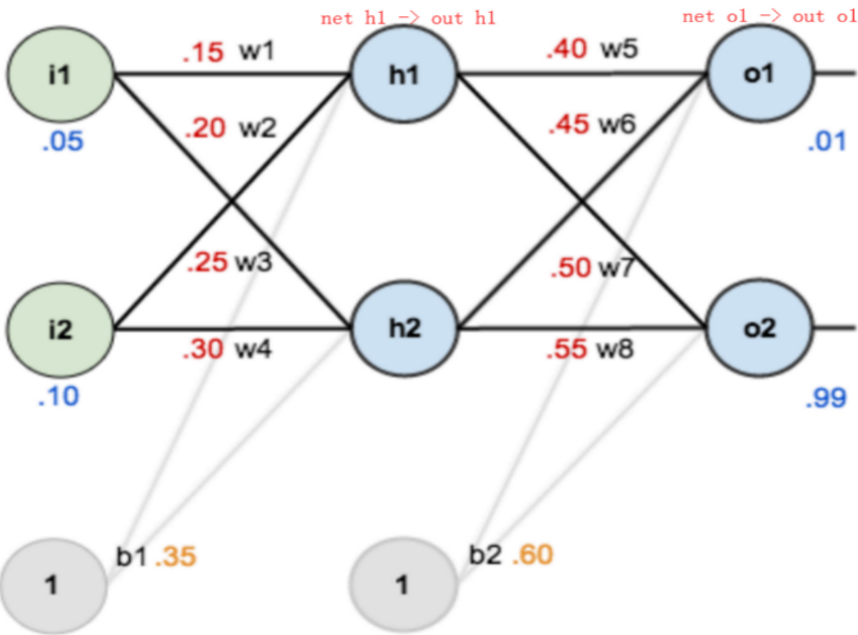
反向传播 (Back Propagation)：利用损失函数 ERROR值，从后往前，结合梯度下降算法，依次求各个参数的偏导，并进行参数更新



举例：

反向传播算法利用链式法则对神经网络中的各个节点的权重进行更新。

如下图是一个简单的神经网络用来举例：激活函数为sigmoid



首先进行前向传播

反向传播(BP算法)[了解]

前向传播运算过程：

$$net_{h1} = w_1 * i_1 + w_2 * i_2 + b_1 * 1$$

$$net_{h1} = 0.15 * 0.05 + 0.2 * 0.1 + 0.35 * 1 = 0.3775$$

$$out_{h1} = \frac{1}{1+e^{-net_{h1}}} = \frac{1}{1+e^{-0.3775}} = 0.593269992$$

$$out_{h2} = 0.596884378$$



$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$net_{o1} = 0.4 * 0.593269992 + 0.45 * 0.596884378 + 0.6 * 1 = 1.105905967$$

$$out_{o1} = \frac{1}{1+e^{-net_{o1}}} = \frac{1}{1+e^{-1.105905967}} = 0.75136507$$

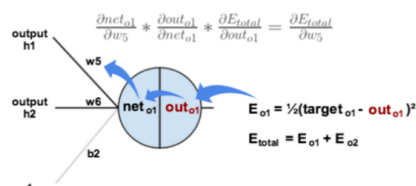
$$out_{o2} = 0.772928465$$



$$E_{total} = \sum \frac{1}{2} (target - output)^2$$

$$E_{total} = E_{o1} + E_{o2} = 0.274811083 + 0.023560026 = 0.298371109$$

接下来是**反向传播**，我们先来求最简单的，求误差E对w5的导数。要求误差E对w5的导数，需要先求误差E对out o1的导数，再求out o1对net o1的导数，最后再求net o1对w5的导数，经过这个处理，我们就可以求出误差E对w5的导数（偏导），如下图所示：



$$E_{total} = \frac{1}{2} (target_{o1} - out_{o1})^2 + \frac{1}{2} (target_{o2} - out_{o2})^2$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = 2 * \frac{1}{2} (target_{o1} - out_{o1})^{2-1} * -1 + 0$$

$$\frac{\partial E_{total}}{\partial out_{o1}} = -(target_{o1} - out_{o1}) = -(0.01 - 0.75136507) = 0.74136507$$

$$out_{o1} = \frac{1}{1+e^{-net_{o1}}}$$

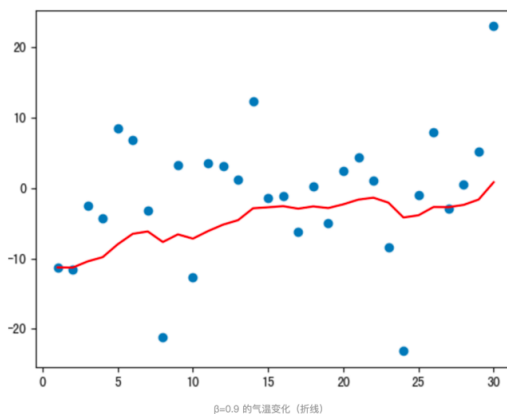
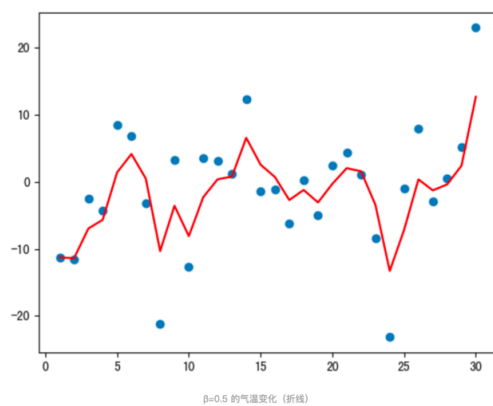
$$\frac{\partial out_{o1}}{\partial net_{o1}} = out_{o1} (1 - out_{o1}) = 0.75136507 (1 - 0.75136507) = 0.186815602$$

$$net_{o1} = w_5 * out_{h1} + w_6 * out_{h2} + b_2 * 1$$

$$\frac{\partial net_{o1}}{\partial w_5} = 1 * out_{h1} * w_5^{(1-1)} + 0 + 0 = out_{h1} = 0.593269992$$

$$\frac{\partial E_{total}}{\partial w_5} = 0.74136507 * 0.186815602 * 0.593269992 = 0.082167041$$

梯度下降的优化方法-指数加权平均



上图是 β 为0.5和0.9时的结果，从中可以看出：

- 指数加权平均绘制出的气温变化曲线更加平缓，
- β 的值越大，则绘制出的折线越加平缓，波动越小。（ $1-\beta$ 越小,t时刻的 S_t 越不依赖 Y_t 的值）
- β 值一般默认都是 0.9

代码：

"""

案例：

演示近30天，天气分布情况。

结论：

针对于 β (调节权重系数)来讲，其值越大说明：越依赖指数加权平均，越不依赖本地的梯度值，数据就越：平缓。

"""

```
import torch
```

```
import matplotlib.pyplot as plt
```

```
ELEMENT_NUMBER = 30
```

```
import os
```

```
os.environ["KMP_DUPLICATE_LIB_OK"] = "TRUE"
```

```
# 1. 实际平均温度
```

```
def dm01():
```

```
    # 固定随机数种子
```

```
    torch.manual_seed(0)
```

```
    # 产生30天的随机温度
```

```
    temperature = torch.randn(size=[ELEMENT_NUMBER, ]) * 10
```

```
    print(temperature)
```

```
    # 绘制平均温度
```

```
    days = torch.arange(1, ELEMENT_NUMBER + 1, 1)
```

```
    plt.plot(days, temperature, color='r')
```

```
    plt.scatter(days, temperature)
```

```

plt.show()

# 2. 指数加权平均温度
def dm02(beta=0.9):
    # 固定随机数种子
    torch.manual_seed(0)
    # 产生30天的随机温度
    temperature = torch.randn(size=[ELEMENT_NUMBER, ]) * 10
    print(temperature)

    exp_weight_avg = []
    # idx从1开始
    for idx, temp in enumerate(temperature, 1):
        # 第一个元素的 EWA 值等于自身
        if idx == 1:
            exp_weight_avg.append(temp)
            continue
        # 第二个元素的 EWA 值等于上一个 EWA 乘以  $\beta$  + 当前气温乘以  $(1-\beta)$ 
        # idx-2: 2-2=0, exp_weight_avg列表中第一个值的下标值
        new_temp = exp_weight_avg[idx - 2] * beta + (1 - beta) *
temp
            exp_weight_avg.append(new_temp)

    days = torch.arange(1, ELEMENT_NUMBER + 1, 1)
    plt.plot(days, exp_weight_avg, color='r')
    plt.scatter(days, temperature)
    plt.show()

if __name__ == '__main__':
    dm01()      # 不考虑 权重系数，每个值的权重都一致。
    dm02(0.5)   # 考虑 权重系数，值越小，数据越陡。
    dm02(0.9)   # 考虑 权重系数，值越大，数据越平缓。9

```

梯度下降的优化方法-动量算法Momentum

梯度计算公式:

$$s_t = \beta s_{t-1} + (1 - \beta) g_t$$

参数更新公式:

$$w_t = w_{t-1} - \eta s_t$$

s_t 是当前时刻指数加权平均梯度值

s_{t-1} 是历史指数加权平均梯度值

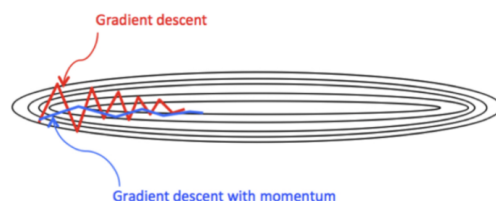
g_t 是当前时刻的梯度值

β 是调节权重系数, 通常取 0.9 或 0.99

η 是学习率

w_t 是当前时刻模型权重参数

Monmomentum 优化方法是如何一定程度上克服 “平缓”、“鞍点” 的问题呢?



- 当处于鞍点位置时, 由于当前的梯度为 0, 参数无法更新。但是 Momentum 动量梯度下降算法已经在先前积累了一些梯度值, 很有可能使得跨过鞍点。
- 由于 mini-batch 普通的梯度下降算法, 每次选取少数的样本梯度确定前进方向, 可能会出现震荡, 使得训练时间变长。Momentum 使用移动加权平均, 平滑了梯度的变化, 使得前进方向更加平缓, 有利于加快训练过程。

5.3.1 AdaGrad

AdaGrad 通过对不同的参数分量使用不同的学习率, AdaGrad 的学习率总体会逐渐减小。

其计算步骤如下:

1. 初始化学习率 η 、初始化参数 w 、小常数 $\sigma = 1e-10$
2. 初始化梯度累计变量 $s = 0$
3. 从训练集中采样 m 个样本的小批量, 计算梯度 g_t
4. 累积平方梯度: $s_t = s_{t-1} + g_t \odot g_t$, $\odot$ 表示各个分量相乘
5. 学习率 η 的计算公式如下:

$$\eta = \frac{\eta}{\sqrt{s_t} + \sigma}$$

6. 权重参数更新公式如下:

$$w_t = w_{t-1} - \frac{\eta}{\sqrt{s_t} + \sigma} * g_t$$

7. 重复 3-7 步骤

AdaGrad 缺点是可能会使得学习率过早、过量的降低, 导致模型训练后期学习率太小, 较难找到最优解。

5.3.1 RMSprop

RMSProp 优化算法是对 AdaGrad 的优化。最主要的不同是，其使用**指数加权平均梯度**替换历史梯度的平方和。其计算过程如下：

1. 初始化学习率 η 、初始化权重参数 w 、小常数 $\sigma = 1e-10$
2. 初始化梯度累计变量 $s = 0$
3. 从训练集中采样 m 个样本的小批量，计算梯度 g_t
4. 使用指数加权平均累计历史梯度， $\odot$ 表示各个分量相乘，公式如下：

$$s_t = \beta s_{t-1} + (1-\beta) g_t \odot g_t$$

5. 学习率 η 的计算公式如下：

$$\eta = \frac{\eta}{\sqrt{s_t} + \sigma}$$

6. 权重参数更新公式如下：

$$w_t = w_{t-1} - \frac{\eta}{\sqrt{s_t} + \sigma} * g_t$$

7. 重复 3-7 步骤

5.3.1 Adam

- Momentum 使用指数加权平均计算当前的梯度值
- AdaGrad、RMSProp 使用自适应的学习率
- Adam优化算法 (Adaptive Moment Estimation, 自适应矩估计) 将 Momentum 和 RMSProp 算法结合在一起
 - 修正梯度：使用梯度的指数加权平均
 - 修正学习率：使用梯度平方的指数加权平均
- 原理：Adam 是结合了 **Momentum** 和 **RMSProp** 优化算法的优点的自适应学习率算法。它计算了梯度的一阶矩（平均值）和二阶矩（梯度的方差）的自适应估计，从而动态调整学习率。

- 梯度计算公式： $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$

$$s_t = \beta_2 s_{t-1} + (1 - \beta_2) g_t^2$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{s}_t = \frac{s_t}{1 - \beta_2^t}$$

- 权重参数更新公式:

$$w_t = w_{t-1} - \frac{\eta}{\sqrt{\hat{s}_t} + \epsilon} \hat{m}_t$$

其中， m_t 是梯度的一阶矩估计， s_t 是梯度的二阶矩估计， $\hat{m}_t$ 、 $\hat{s}_t$ 偏差校正后的估计。

代码：

.....

案例：

演示 梯度下降优化方法。

梯度下降相关介绍：

概述：

梯度下降是结合 本次损失函数的导数（作为梯度） 基于学习率 来更新权重的。

公式：

$w_{\text{新}} = w_{\text{旧}} - \text{学习率} * (\text{本次的}) \text{梯度}$

存在的问题：

1. 遇到平缓区域，梯度下降(权重更新)可能会慢。
2. 可能会遇到 鞍点(梯度为0)
3. 可能会遇到 局部最小值。

解决思路：

从上述的 学习率 或者 梯度入手，进行优化，于是有了：动量法Momentum，自适应学习率AdaGrad，RMSProp，综合衡量：Adam

动量法Momentum：

动量法公式：

$$S_t = \beta * S_{t-1} + (1 - \beta) * G_t$$

解释：

S_t ： 本次的指数移动加权平均结果。

β ： 调节权重系数，越大，数据越平缓，历史指数移动加权平均 比重越大，本次梯度权重越小。

S_{t-1} ： 历史的指数移动加权平均结果。

G_t ： 本次计算出的梯度(不考虑历史梯度)。

加入动量法后的 梯度更新公式：

$$W_{\text{新}} = W_{\text{旧}} - \text{学习率} * S_t$$

自适应学习率：AdaGrad(Adaptive Gradient Estimation)

公式：

累计平方梯度：

$$S_t = S_{t-1} + G_t * G_t$$

解释：

S_t ： 累计平方梯度

S_{t-1} ： 历史累计平方梯度。

G_t ： 本次的梯度。

学习率：

$$\text{学习率} = \text{学习率} / (\text{sqrt}(S_t) + \text{小常数})$$

解释：

小常数：1e-10，目的：防止分母变为0

梯度下降公式：

$$W_{\text{新}} = W_{\text{旧}} - \text{调整后的学习率} * G_t$$

缺点：

可能会导致学习率过早，过量的降低，导致模型后期学习率太小，较难找到最优解。

自适应学习率：RMSProp(Root Mean Square Propagation) -> 可以看做是对AdaGrad做的优化，加入 调和权重系数。

公式：

指数加权平均 累计历史平方梯度：

$$S_t = \beta * S_{t-1} + (1 - \beta) * G_t * G_t$$

解释：

S_t ： 累计平方梯度

S_{t-1} ： 历史累计平方梯度。

G_t ： 本次的梯度。

β ： 调和权重系数。

学习率：

$$\text{学习率} = \text{学习率} / (\text{sqrt}(S_t) + \text{小常数})$$

解释：

小常数：1e-10，目的：防止分母变为0

梯度下降公式：

$$w_{\text{新}} = w_{\text{旧}} - \text{调整后的学习率} * G_t$$

优点：

RMSProp通过引入 衰减系数 β ，控制历史梯度 对 历史梯度信息获取的多少。

自适应矩估计：Adam(Adaptive Moment Estimation)

思路：

即优化学习率，又优化梯度。

公式：

一阶矩：算均值。

$$M_t = \beta_1 * M_{t-1} + (1 - \beta_1) * G_t \quad \text{充当：梯度}$$

$$S_t = \beta_2 * S_{t-1} + (1 - \beta_2) * G_t * G_t \quad \text{充当：学习}$$

率

二阶矩：梯度的方差。

$$M_t^{\wedge} = M_t / (1 - \beta_1^{\wedge} t)$$

$$S_t^{\wedge} = S_t / (1 - \beta_2^{\wedge} t)$$

权重更新公式：

$$w_{\text{新}} = w_{\text{旧}} - \text{学习率} / (\text{sqrt}(S_t^{\wedge}) + \text{小常数}) * M_t^{\wedge}$$

大白话翻译：

Adam = RMSProp + Momentum

总结：如何选择梯度下降优化方法

简单任务和较小的模型：

SGD，动量法

复杂任务或者有大量数据：

Adam

需要处理稀疏数据或者文本数据：

AdaGrad，RMSProp

"""

导包

import torch

import torch.nn as nn

import torch.optim as optim

1. 定义函数，演示：梯度下降优化方法 -> 动量法(Momentum)

def dm01_momentum():

1. 初始化权重参数。

```

w = torch.tensor([1.0], requires_grad=True,
dtype=torch.float32)
# 2. 定义损失函数
criterion = ((w ** 2) / 2.0)
# 3. 创建优化器(函数对象) -> 基于SGD(随机梯度下降), 加入参数
momentum, 就是 动量法.
# 参1: (待优化的)参数列表, 参2: 学习率, 参3: 动量参数.
optimizer = optim.SGD(params=[w], lr=0.01, momentum=0.9) # 细
节: momentum=0(默认), 只考虑: 本次梯度.
# 4. 计算梯度值: 梯度清零 + 反向传播 + 参数更新
optimizer.zero_grad()
criterion.sum().backward()
optimizer.step()
print(f'w: {w}, w.grad: {w.grad}')

# 5.重复上述的步骤, 第2次 更新权重参数.
# 5.1 定义损失函数.
criterion = ((w ** 2) / 2.0)
# 5.2 计算梯度值: 梯度清零 + 反向传播 + 参数更新
optimizer.zero_grad()
criterion.sum().backward()
optimizer.step()
# 5.3 打印结果.
print(f'w: {w}, w.grad: {w.grad}')

```

2. 定义函数, 演示: 梯度下降优化方法 -> 自适应学习率(AdaGrad)

```

def dm02_adagrad():
    # 1. 初始化权重参数.
    w = torch.tensor([1.0], requires_grad=True,
dtype=torch.float32)
    # 2. 定义损失函数
    criterion = ((w ** 2) / 2.0)
    # 3. 创建优化器(函数对象)
    # 思路1: 基于SGD(随机梯度下降), 加入参数 momentum, 就是 动量法.
    # 参1: (待优化的)参数列表, 参2: 学习率, 参3: 动量参数.
    # optimizer = optim.SGD(params=[w], lr=0.01, momentum=0.9) #
    细节: momentum=0(默认), 只考虑: 本次梯度.

    # 思路2: 基于AdaGrad(自适应学习率).
    optimizer = optim.Adagrad(params=[w], lr=0.01)

    # 4. 计算梯度值: 梯度清零 + 反向传播 + 参数更新
    optimizer.zero_grad()

```

```

criterion.sum().backward()
optimizer.step()
print(f'w: {w}, w.grad: {w.grad}')

# 5.重复上述的步骤, 第2次 更新权重参数.
# 5.1 定义损失函数.
criterion = ((w ** 2) / 2.0)
# 5.2 计算梯度值: 梯度清零 + 反向传播 + 参数更新
optimizer.zero_grad()
criterion.sum().backward()
optimizer.step()
# 5.3 打印结果.
print(f'w: {w}, w.grad: {w.grad}')

# 3. 定义函数, 演示: 梯度下降优化方法 -> 自适应学习率(RMSProp)
def dm03_rmsprop():
    # 1. 初始化权重参数.
    w = torch.tensor([1.0], requires_grad=True,
dtype=torch.float32)
    # 2. 定义损失函数
    criterion = ((w ** 2) / 2.0)
    # 3. 创建优化器(函数对象)
    # 思路1: 基于SGD(随机梯度下降), 加入参数 momentum, 就是 动量法.
    # 参1: (待优化的)参数列表, 参2: 学习率, 参3: 动量参数.
    # optimizer = optim.SGD(params=[w], lr=0.01, momentum=0.9) #
    细节: momentum=0(默认), 只考虑: 本次梯度.

    # 思路2: 基于AdaGrad(自适应学习率).
    # optimizer = optim.Adagrad(params=[w], lr=0.01)

    # 思路3: 基于RMSProp(自适应学习率).
    optimizer = optim.RMSprop(params=[w], lr=0.01, alpha=0.99)

    # 4. 计算梯度值: 梯度清零 + 反向传播 + 参数更新
    optimizer.zero_grad()
    criterion.sum().backward()
    optimizer.step()
    print(f'w: {w}, w.grad: {w.grad}')

    # 5.重复上述的步骤, 第2次 更新权重参数.
    # 5.1 定义损失函数.
    criterion = ((w ** 2) / 2.0)
    # 5.2 计算梯度值: 梯度清零 + 反向传播 + 参数更新
    optimizer.zero_grad()
    criterion.sum().backward()

```

```

optimizer.step()
# 5.3 打印结果.
print(f'w: {w}, w.grad: {w.grad}')

# 4. 定义函数, 演示: 梯度下降优化方法 -> 自适应矩估计(Adam)
def dm04_adam():
    # 1. 初始化权重参数.
    w = torch.tensor([1.0], requires_grad=True,
dtype=torch.float32)
    # 2. 定义损失函数
    criterion = ((w ** 2) / 2.0)
    # 3. 创建优化器(函数对象)
    # 思路1: 基于SGD(随机梯度下降), 加入参数 momentum, 就是 动量法.
    # 参1: (待优化的)参数列表, 参2: 学习率, 参3: 动量参数.
    # optimizer = optim.SGD(params=[w], lr=0.01, momentum=0.9) #
    细节: momentum=0(默认), 只考虑: 本次梯度.

    # 思路2: 基于AdaGrad(自适应学习率).
    # optimizer = optim.Adagrad(params=[w], lr=0.01)

    # 思路3: 基于RMSProp(自适应学习率).
    # optimizer = optim.RMSprop(params=[w], lr=0.01, alpha=0.99)

    # 思路4: 基于Adam(自适应矩估计).
    optimizer = optim.Adam(params=[w], lr=0.01, betas=(0.9,
0.999)) # betas=(梯度用的 衰减系数, 学习率用的 衰减系数)

    # 4. 计算梯度值: 梯度清零 + 反向传播 + 参数更新
    optimizer.zero_grad()
    criterion.sum().backward()
    optimizer.step()
    print(f'w: {w}, w.grad: {w.grad}')

    # 5. 重复上述的步骤, 第2次 更新权重参数.
    # 5.1 定义损失函数.
    criterion = ((w ** 2) / 2.0)
    # 5.2 计算梯度值: 梯度清零 + 反向传播 + 参数更新
    optimizer.zero_grad()
    criterion.sum().backward()
    optimizer.step()
    # 5.3 打印结果.
    print(f'w: {w}, w.grad: {w.grad}')

```

5. 测试

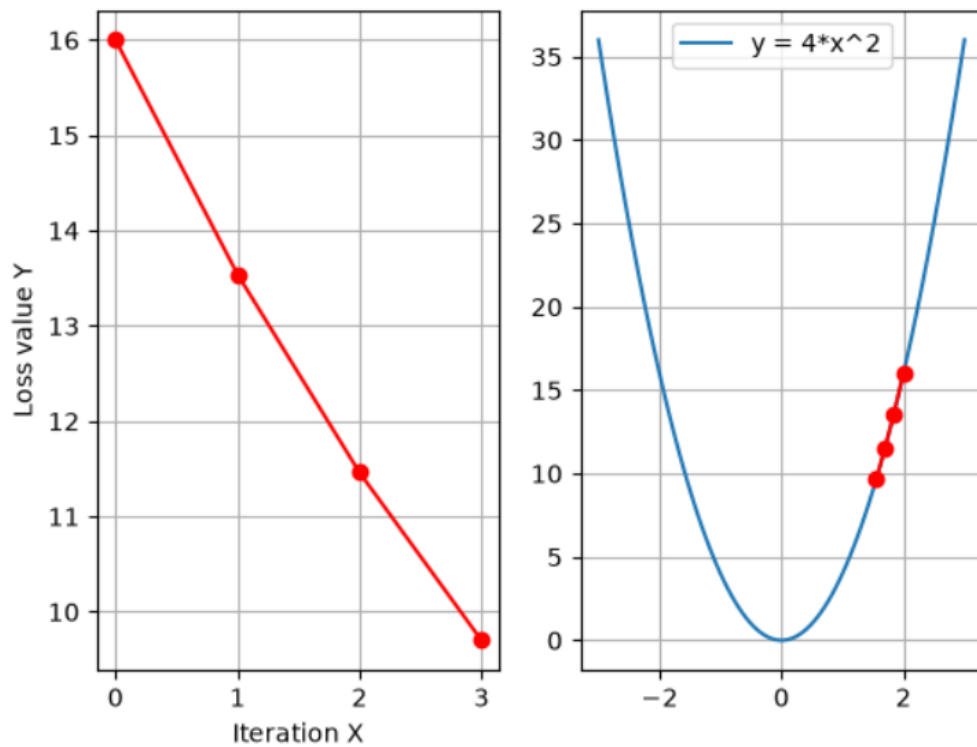
```
if __name__ == '__main__':  
    dm01_momentum()  
    # dm02_adagrad()  
    # dm03_rmsprop()  
    # dm04_adam()
```

5.4 学习率衰减方法

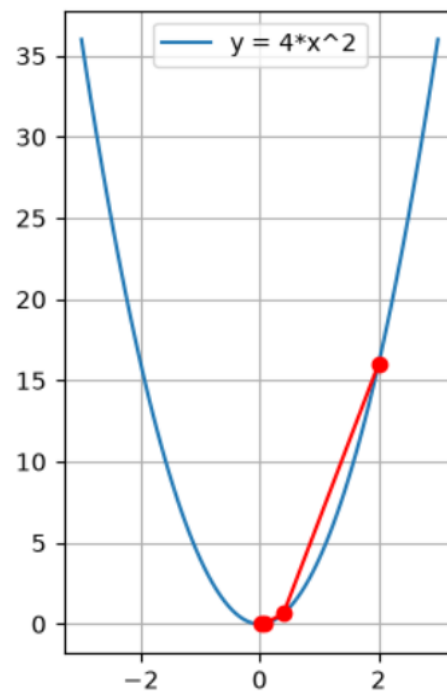
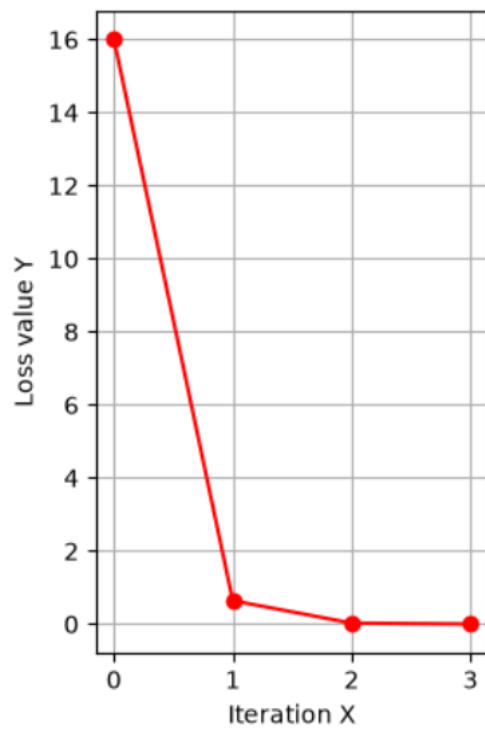
前言：

学习率对梯度的影响

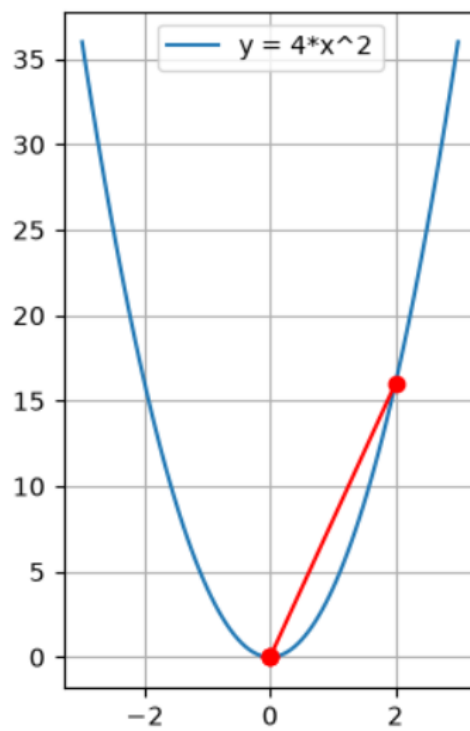
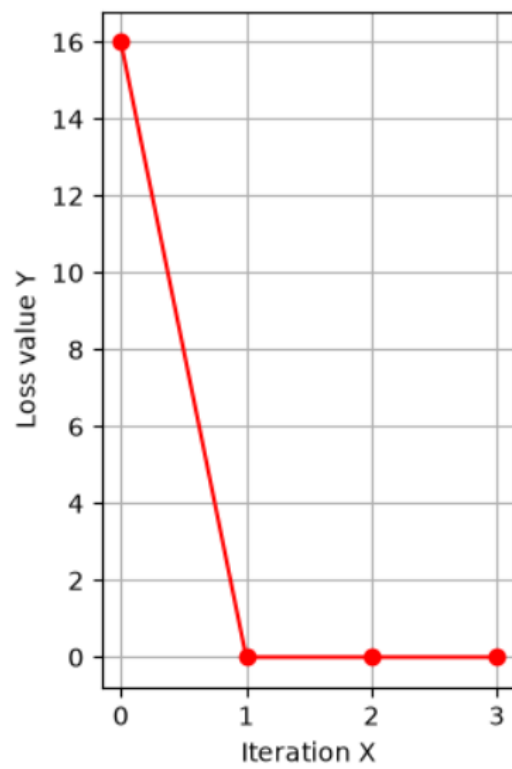
$\eta = 0.01$



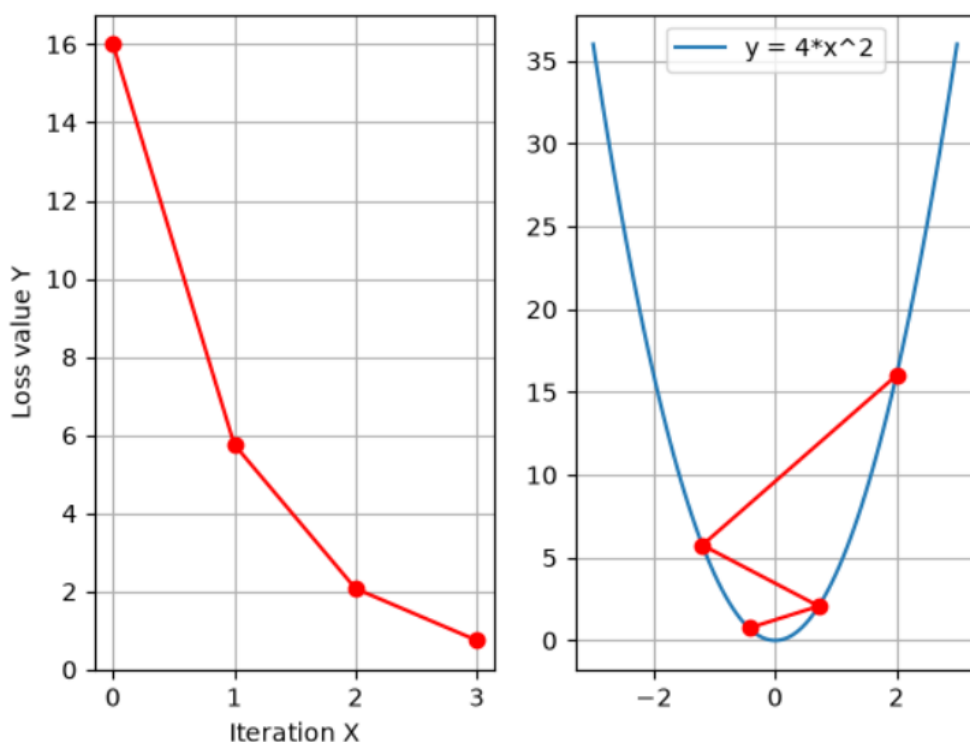
$\eta = 0.1$



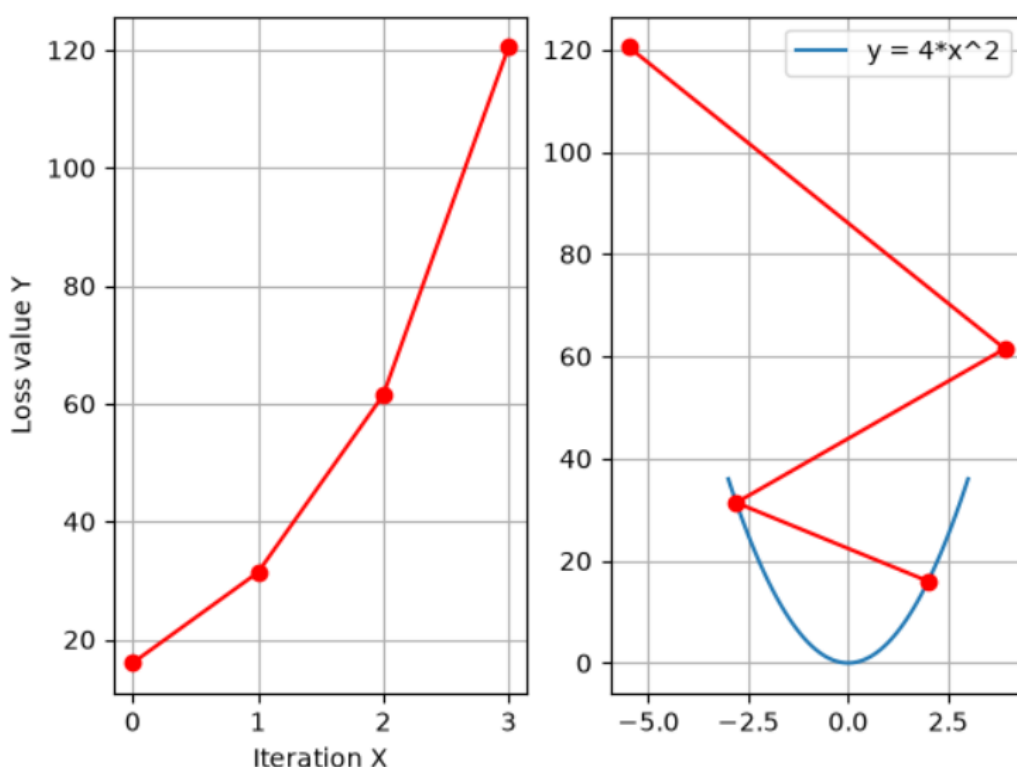
$lr = 0.125$



$lr = 0.2$



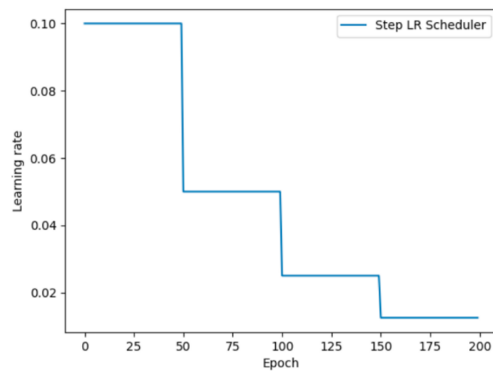
$lr = 0.3$



从这几组图可以看出，**梯度决定参数更新的方向，学习率决定沿梯度方向更新的步长**，即参数更新量约为“学习率 × 梯度”。学习率过大时，更新步长太大，容易在最优值两侧来回跳动甚至发散；学习率适中时，损失能够较快且稳定地下降并逐渐接近最小值；学习率过小时，虽然能够稳定下降，但收敛速度会很慢。因此，选择合适的学习率是梯度下降能否快速、稳定收敛的关键。

5.4.1 等间隔学习率衰减

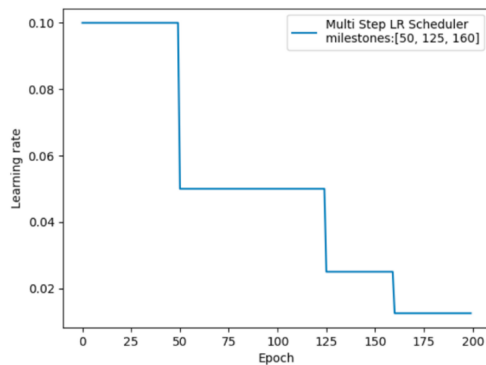
等间隔学习率衰减方式如下所示：



```
# step_size: 调整间隔数=50
# gamma: 调整系数=0.5
# 调整方式:  $lr = lr * gamma$ 
lr_scheduler.StepLR(optimizer, step_size, gamma=0.1)
```

5.4.2 指定间隔学习率衰减

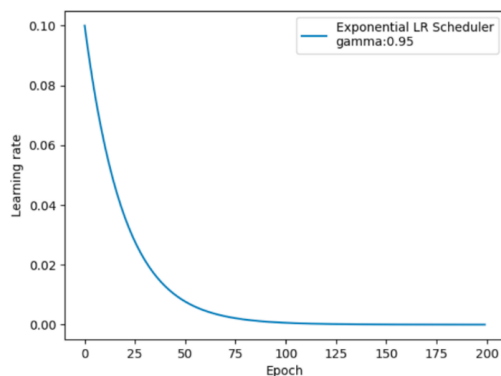
指定间隔学习率衰减的效果如下：



```
# milestones: 设定调整轮次:[50, 125, 160]
# gamma: 调整系数
# 调整方式:  $lr = lr * gamma$ 
optim.lr_scheduler.MultiStepLR(optimizer, milestones,
gamma=0.1)
```

5.4.3 按指数学习率衰减

按指数学习率衰减的效果如下：



```
# gamma: 指数的底
# 调整方式
#  $lr = lr * gamma^{epoch}$ 
optim.lr_scheduler.ExponentialLR(optimizer, gamma)
```

代码：

.....

案例：

演示学习率衰减策略。

学习率衰减策略介绍：

目的：

较之于AdaGrad，RMSProp，Adam方式，我们可以通过 等间隔，指定间隔，指数等方式，来手动控制学习率的调整。

分类：

等间隔学习率衰减

指定间隔学习率衰减

指数学习率衰减

等间隔学习率衰减：

step_size：间隔的轮数，即：多少轮调整一次学习率。

gamma：学习率衰减系数，即： $lr_{\text{新}} = lr_{\text{旧}} * \text{gamma}$

指定间隔学习率衰减：

milestones = [50, 125, 160] 里边定义的是要调整学习率的 轮数。

gamma：学习率衰减系数，即： $lr_{\text{新}} = lr_{\text{旧}} * \text{gamma}$

指数间隔学习率衰减：

前期学习率衰减快，中期慢，后期更慢，更符合梯度下降规律。

公式：

$$lr_{\text{新}} = lr_{\text{旧}} * \text{gamma} ** \text{epoch}$$

总结：

等间隔学习率衰减：

优点：

直观，易于调试，适用于 大批量数据。

缺点：

学习率变化较大，可能跳过最优解。

应用场景：

大型数据集，较为简单的任务。

指定间学习率衰减：

优点：

易于调试，稳定训练过程。

缺点：

在某些情况下可能衰减过快，导致优化提前停滞。

应用场景：

对训练平稳性要求较高的任务。

指数学习率衰减：

优点：

平滑，且考虑历史更新，收敛稳定性较强。

缺点：

超参调节较为复杂，可能需要更多的资源。

应用场景：

高精度训练，避免过快收敛。

```

"""

# 导包
import torch
from torch import optim
import matplotlib.pyplot as plt

# 1. 定义函数，演示：等间隔学习率衰减
def dm01():
    # 1. 定义变量，记录初始的 学习率，训练的轮数，每轮训练的批次数。
    lr, epochs, iteration = 0.1, 200, 10

    # 2. 创建数据集。 y_true, x, w
    # 真实值。
    y_true = torch.tensor([0])
    # 输入特征
    x = torch.tensor([1.0], dtype=torch.float32)
    # 权重参数w，需要自动微分(求导)
    w = torch.tensor([1.0], requires_grad=True,
dtype=torch.float32)

    # 3. 创建优化器对象，动量法 -> 加速模型的收敛，减少震荡。
    # 参1：待优化的参数，参2：学习率，参3：动量系数
    optimizer = optim.SGD([w], lr=lr, momentum=0.9)

    # 4. 创建学习率衰减对象。
    # 思路1：创建等间隔学习率衰减对象。
    # 参1：优化器对象，参2：间隔的轮数(多少轮调整一次学习率)，参3：学习率衰
    减系数。
    scheduler = optim.lr_scheduler.StepLR(optimizer,
step_size=50, gamma=0.5) # [0.1, 0.1, 0.1... 0.05...]

    # 5. 创建两个列表，分别表示：训练轮数，每轮训练用的学习率
    # epoch_list = [0, 1, 2, 3.... 50, 51, 52...100, 101, 101...
    150, 151...199]
    # lr_list = [0.1, 0.1, 0.1, 0.05.....,0.025.....,
    0.0125...]
    lr_list, epoch_list = [], []

    # 6. 循环遍历训练轮数，进行具体的训练。
    for epoch in range(epochs): # epoch: 0 ~ 199
        # 7. 获取当前轮数 和 学习率，并保存到列表中。
        epoch_list.append(epoch)

```

```

lr_list.append(scheduler.get_last_lr())    # 获取最后的
lr(learning rate, 学习率)

# 8. 循环遍历，每轮每批次进行训练.
for batch in range(iteration):
    # 9. 先计算预测值，然后基于损失函数计算损失.
    y_pred = w * x
    # 10. 计算损失，最小二乘法.
    loss = (y_pred - y_true) ** 2
    # 11. 梯度清零 + 反向传播 + 优化器更新参数.
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()
    # 12. 更新学习率.
    scheduler.step()
# 13. 打印结果:
print(f'lr_list: {lr_list}')              # [0.1, 0.1, 0.1...,
0.05.....,0.025....., 0.0125...]

# 14. 可视化.
# x轴: 训练的轮数, y轴: 每轮训练用的学习率
plt.plot(epoch_list, lr_list)
plt.xlabel('Epoch')
plt.ylabel('Learning Rate')
plt.show()

# 2. 定义函数，演示：指定间隔学习率衰减
def dm02():
    # 1. 定义变量，记录初始的 学习率，训练的轮数，每轮训练的批次数.
    lr, epochs, iteration = 0.1, 200, 10

    # 2. 创建数据集. y_true, x, w
    # 真实值.
    y_true = torch.tensor([0])
    # 输入特征
    x = torch.tensor([1.0], dtype=torch.float32)
    # 权重参数w，需要自动微分(求导)
    w = torch.tensor([1.0], requires_grad=True,
dtype=torch.float32)

    # 3. 创建优化器对象，动量法 -> 加速模型的收敛，减少震荡.
    # 参1: 待优化的参数，参2: 学习率，参3: 动量系数
    optimizer = optim.SGD([w], lr=lr, momentum=0.9)

```

```

# 4. 创建学习率衰减对象.
# 思路1: 创建等间隔学习率衰减对象.
# 参1: 优化器对象, 参2: 间隔的轮数(多少轮调整一次学习率), 参3: 学习率衰减系数.
scheduler = optim.lr_scheduler.StepLR(optimizer,
step_size=50, gamma=0.5) # [0.1, 0.1, 0.1... 0.05...]

# 思路2: 创建指定间隔学习率衰减对象.
# 定义变量, 记录要修改学习率的轮数.
milestones = [50, 125, 160]
scheduler = optim.lr_scheduler.MultiStepLR(optimizer,
milestones=milestones, gamma=0.5)

# 5. 创建两个列表, 分别表示: 训练轮数, 每轮训练用的学习率
# epoch_list = [0, 1, 2, 3.... 50, 51, 52...100, 101, 101...
150, 151...199]
# lr_list = [0.1, 0.1, 0.1, 0.05.....,0.025.....,
0.0125...]
lr_list, epoch_list = [], []

# 6. 循环遍历训练轮数, 进行具体的训练.
for epoch in range(epochs): # epoch: 0 ~ 199
    # 7. 获取当前轮数 和 学习率, 并保存到列表中.
    epoch_list.append(epoch)
    lr_list.append(scheduler.get_last_lr()) # 获取最后的
lr(learning rate, 学习率)

# 8. 循环遍历, 每轮每批次进行训练.
for batch in range(iteration):
    # 9. 先计算预测值, 然后基于损失函数计算损失.
    y_pred = w * x
    # 10. 计算损失, 最小二乘法.
    loss = (y_pred - y_true) ** 2
    # 11. 梯度清零 + 反向传播 + 优化器更新参数.
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

    # 12. 更新学习率.
    scheduler.step()

# 13. 打印结果:
print(f'lr_list: {lr_list}') # [0.1, 0.1, 0.1...,
0.05.....,0.025....., 0.0125...]

# 14. 可视化.
# x轴: 训练的轮数, y轴: 每轮训练用的学习率

```

```
plt.plot(epoch_list, lr_list)
plt.xlabel('Epoch')
plt.ylabel('Learning Rate')
plt.show()
```

3. 定义函数，演示：指数学习率衰减

```
def dm03():
```

1. 定义变量，记录初始的 学习率，训练的轮数，每轮训练的批次。

```
lr, epochs, iteration = 0.1, 200, 10
```

2. 创建数据集。 y_true, x, w

真实值。

```
y_true = torch.tensor([0])
```

输入特征

```
x = torch.tensor([1.0], dtype=torch.float32)
```

权重参数w，需要自动微分(求导)

```
w = torch.tensor([1.0], requires_grad=True,
dtype=torch.float32)
```

3. 创建优化器对象，动量法 -> 加速模型的收敛，减少震荡。

参1：待优化的参数，参2：学习率，参3：动量系数

```
optimizer = optim.SGD([w], lr=lr, momentum=0.9)
```

4. 创建学习率衰减对象。

思路1：创建等间隔学习率衰减对象。

参1：优化器对象，参2：间隔的轮数(多少轮调整一次学习率)，参3：学习率衰减系数。

```
# scheduler = optim.lr_scheduler.StepLR(optimizer,
step_size=50, gamma=0.5) # [0.1, 0.1, 0.1... 0.05...]
```

思路2：创建指定间隔学习率衰减对象。

定义变量，记录要修改学习率的轮数。

```
# milestones = [50, 125, 160]
```

```
# scheduler = optim.lr_scheduler.MultiStepLR(optimizer,
milestones=milestones, gamma=0.5)
```

思路3：创建指数学习率衰减对象。

```
scheduler = optim.lr_scheduler.ExponentialLR(optimizer,
gamma=0.95)
```

5. 创建两个列表，分别表示：训练轮数，每轮训练用的学习率

```
# epoch_list = [0, 1, 2, 3... 50, 51, 52...100, 101, 101...
150, 151...199]
```

```

    # lr_list = [0.1, 0.1, 0.1, 0.05.....,0.025.....,
0.0125...]
    lr_list, epoch_list = [], []

    # 6. 循环遍历训练轮数，进行具体的训练。
    for epoch in range(epochs):      # epoch: 0 ~ 199
        # 7. 获取当前轮数 和 学习率，并保存到列表中。
        epoch_list.append(epoch)
        lr_list.append(scheduler.get_last_lr())      # 获取最后的
lr(learning rate, 学习率)

        # 8. 循环遍历，每轮每批次进行训练。
        for batch in range(iteration):
            # 9. 先计算预测值，然后基于损失函数计算损失。
            y_pred = w * x
            # 10. 计算损失，最小二乘法。
            loss = (y_pred - y_true) ** 2
            # 11. 梯度清零 + 反向传播 + 优化器更新参数。
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

        # 12. 更新学习率。
        scheduler.step()

    # 13. 打印结果：
    print(f'lr_list: {lr_list}')      # [0.1, 0.1, 0.1...,
0.05.....,0.025....., 0.0125...]

    # 14. 可视化。
    # x轴：训练的轮数，y轴：每轮训练用的学习率
    plt.plot(epoch_list, lr_list)
    plt.xlabel('Epoch')
    plt.ylabel('Learning Rate')
    plt.show()

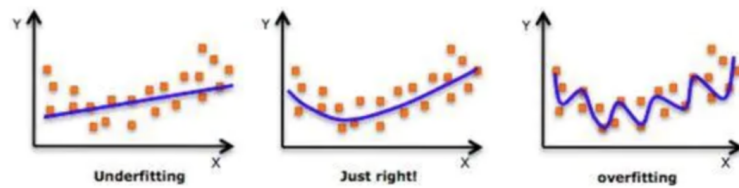
# 4. 测试
if __name__ == '__main__':
    # dm01()
    # dm02()
    dm03()

```

6 正则化方法

6.1 Dropout随机失活

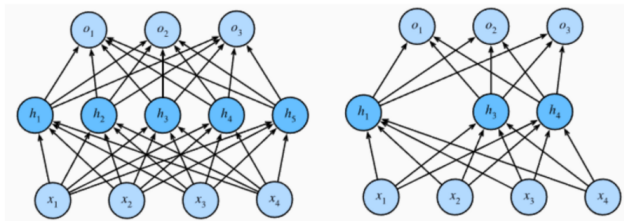
正则化



- 在设计机器学习算法时希望在新样本上的泛化能力强。许多机器学习算法都采用相关的策略来减小测试误差，这些策略被统称为正则化。
- 神经网络强大的表示能力经常遇到过拟合，所以需要使用不同形式的正则化策略。
- 目前在深度学习中使用较多的策略有范数惩罚，DropOut，特殊的网络层等，接下来我们对其进行详细的介绍。

Dropout正则化

在神经网络中模型参数较多，在数据量不足的情况下，很容易过拟合。Dropout（随机失活）是一个简单有效的正则化方法。



- 在训练过程中，Dropout的实现是让神经元以超参数 p 的概率停止工作或者激活被置为0,未被置为0的进行缩放，缩放比例为 $1/(1-p)$ 。训练过程可以认为是对完整的神经网络的一些子集进行训练，每次基于输入数据只更新子网络的参数。
- 在测试过程中，随机失活不起作用。

代码：

.....

案例：

代码演示 随机失活。

正则化的作用：

缓解模型的过拟合情况。

正则化的方式：

L1正则化：权重可以变为0，相当于：降维。

L2正则化：权重可以无限接近0

DropOut：随机失活，每批次样本训练时，随机让一部分神经元死亡，防止一些特征对结果的影响较大(防止过拟合)

BN(批量归一化)：...

.....

导包

import torch

```

import torch.nn as nn

# 1. 定义函数，演示：随机失活(DropOut)
def dm01():
    # 1. 创建隐藏层输出结果.
    t1 = torch.randint(0, 10, size=(1, 4)).float()
    print(f't1: {t1}')          # t1: tensor([[0., 5., 6., 3.]])

    # 2. 进行下一层 加权求和 和 激活函数计算.
    # 2.1 创建全连接层(充当线性层)
    # 参1: 输入特征维度, 参2: 输出特征维度.
    linear1 = nn.Linear(4, 5)

    # 2.2 加权求和.
    l1 = linear1(t1)
    print(f'l1: {l1}')

    # 2.3 激活函数.
    output = torch.relu(l1)
    print(f'output: {output}')

    # 3. 对激活值进行随机失活dropout处理 -> 只有训练阶段有, 测试阶段没有.
    dropout = nn.Dropout(p=0.5) # 每个神经元都有50%的概率被 kill.
    # 具体的 随机失活动作.
    d1 = dropout(output)
    print(f'd1(随机失活后的数据): {d1}')          # 未被失活的进行缩放, 缩
    放比例为:  $1 / (1 - p) = 2$ 

# 2. 测试
if __name__ == '__main__':
    dm01()

```

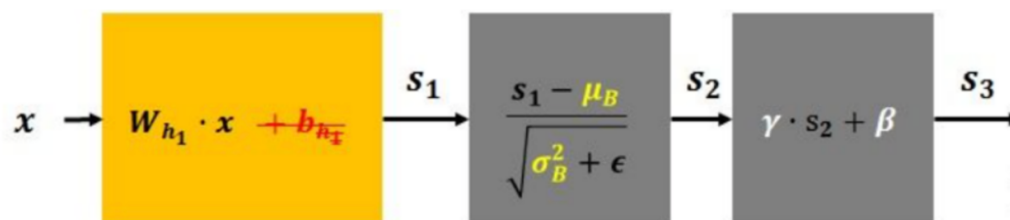
```

C:\Software\DevelopSoftware\Anaconda3\python.exe D:\workspace\ai_03_work_gz\DL_Project\day05\05_dr
t1: tensor([[6., 8., 1., 0.]]) 输入的特征
l1: tensor([[ 3.5535, -1.1570,  4.9350,  2.2249,  2.2844]], 加权求和处理结果
      grad_fn=<AddmmBackward0>)
output: tensor([[3.5535, 0.0000, 4.9350, 2.2249, 2.2844]], grad_fn=<ReluBackward0>) 激活函数处理结果
d1(随机失活后的数据): tensor([[7.1070, 0.0000, 0.0000, 4.4497, 0.0000]], grad_fn=<MulBackward0>)
      随机失活处理结果, p=0.5, 失活概率: 50%, 缩放比例:  $1 / (1 - 0.5) = 2$ 

```

6.2 BN批量归一化

批量归一化(Batch Normalization)



先对数据标准化，再对数据重构（缩放+平移），如下所示：

$$f(x) = \lambda \cdot \frac{x - E(x)}{\sqrt{\text{Var}(x) + \epsilon}} + \beta$$

1. λ 和 β 是可学习的参数，它相当于对标准化后的值做了一个线性变换， λ 为系数， β 为偏置；
2. ϵ 通常指为 $1e-5$ ，避免分母为 0；
3. $E(x)$ 表示变量的均值；
4. $\text{Var}(x)$ 表示变量的方差；

批量归一化层在计算机视觉领域使用较多。

高级

代码：

.....

案例：

代码演示批量归一化，它(批量归一化)也属于正则化的一种，也是用于缓解模型的过拟合情况的。

批量归一化：

思路：

先对数据做标准化(会丢失一些信息)，然后再对数据做缩放(λ ，理解为： w 权重)和平移(β ，理解为： b 偏置)，再找补回一些信息。

应用场景：

批量归一化在计算机视觉领域使用较多。

BatchNorm1d: 主要应用于全连接层或处理一维数据的网络，例如文本处理。它接收形状为 $(N, \text{num_features})$ 的张量作为输入。

BatchNorm2d: 主要应用于卷积神经网络，处理二维图像数据或特征图。它接收形状为 (N, C, H, W) 的张量作为输入。

BatchNorm3d: 主要用于三维卷积神经网络 (3D CNN)，处理三维数据，例如视频或医学图像。它接收形状为 (N, C, D, H, W) 的张量作为输入。

.....

导包

import torch

import torch.nn as nn

1. 定义函数，处理 二维数据.

```
def dm01():
```

1. 创建图像样本数据.

1张图片，2个通道，3行4列(像素点)

```
input_2d = torch.randn(size=(1, 2, 3, 4))
```

```
print(f'input_2d: {input_2d}')
```

2. 创建批量归一化层(BN层)

参1: 输入特征数 = 图片的通道数.

参2: 噪声值(小常数)，默认为 $1e-5$.

参3: 动量值，用于计算移动平均统计量的 动量值.

参4: 表示使用可学习的变换参数(λ , β) 对归一化(标准化)后的数据进行 缩放和平移.

```
bn2d = nn.BatchNorm2d(num_features=2, eps=1e-5, momentum=0.1,  
affine=True)
```

3. 对数据进行 批量归一化处理.

```
output_2d = bn2d(input_2d)
```

```
print(f'output_2d: {output_2d}')
```

2. 定义函数，处理：一维数据.

```
def dm02():
```

1. 创建样本数据.

2行2列，2条样本，每个样本有2个特征

```
input_1d = torch.randn(size=(2, 2))
```

```
print(f'input_1d: {input_1d}')
```

2. 创建线性层.

```
linear1 = nn.Linear(2, 4)
```

3. 对数据进行 线性变换.

```
l1 = linear1(input_1d)
```

```
print(f'l1: {l1}')
```

4. 创建批量归一化层.

```
bn1d = nn.BatchNorm1d(num_features=4)
```

5. 对线性处理结果l1 进行 批量归一化处理.

```
output_1d = bn1d(l1)
```

```
print(f'output_1d: {output_1d}')
```

3. 测试

```
if __name__ == '__main__':  
    # dm01()  
    dm02()
```

手机价格分类

需求分析

小明创办了一家手机公司，他不知道如何估算手机产品的价格。为了解决这个问题，他收集了多家公司的手机销售数据。该数据为二手手机的各个性能的数据，最后根据这些性能得到4个价格区间，作为这些二手手机售出的价格区间。主要包括：

```
battery_power: 电池一次可储存的总能量，单位为毫安时  
blue : 是否有蓝牙  
clock_speed: 微处理器执行指令的速度  
dual_sim: 是否支持双卡  
fc: 前置摄像头百万像素  
four_g: 是否有4G  
int_memory: 内存 (GB)  
m_dep: 移动深度 (cm)  
mobile_wt: 手机重量  
n_cores: 处理器内核数  
pc: 主摄像头百万像素  
px_height: 像素分辨率高度  
px_width: 像素分辨率宽度  
ram: 随机存取存储器 (兆字节)  
sc_h: 手机屏幕高度 (cm)  
sc_w: 手机屏幕宽度 (cm)  
talk_time: 一次电池充电持续时间最长的时间  
three_g: 是否有3G  
touch_screen: 是否有触控屏  
wifi: 是否能连wifi  
price_range: 价格区间 (0, 1, 2, 3)
```

高级数字化人才

我们需要帮助小明找出手机的功能（例如：RAM等）与其售价之间的某种关系。我们可以使用机器学习的方法来解决这个问题，也可以构建一个全连接的网络。

需要注意的是: 在这个问题中，我们**不需要预测实际价格**，而是一个**价格范围**，它的范围使用 **0、1、2、3 来表示**，所以该问题也是一个**分类问题**。接下来我们还是按照四个步骤来完成这个任务：

- 准备训练集数据
- 构建要使用的模型
- 模型训练
- 模型预测评估

我们前面的网络模型在测试集的准确率为: 0.64250, 我们可以通过以下方面进行调优:

1. 优化方法由 SGD 调整为 Adam
2. 学习率由 $1e-3$ 调整为 $1e-4$
3. 对数据进行标准化
4. 增加网络深度, 即: 增加网络参数量
5. 调整训练轮次

代码:

```
import torch
import torch.nn as nn
import pandas as pd
from sklearn.model_selection import train_test_split
from torch.utils.data import TensorDataset
from torch.utils.data import DataLoader
import torch.optim as optim
import numpy as np
import time
from sklearn.preprocessing import StandardScaler

# 构建数据集
def create_dataset():
    # 使用pandas读取数据
    data = pd.read_csv('./data/手机价格预测.csv')
    # 特征值和目标值
    x, y = data.iloc[:, :-1], data.iloc[:, -1]
    # 类型转换: 特征值, 目标值
    x = x.astype(np.float32)
    y = y.astype(np.int64)
    # 数据集划分
    x_train, x_valid, y_train, y_valid = train_test_split(x, y,
        train_size=0.8, random_state=88, stratify=y)
    # 优化①: 数据标准化
```

```

transfer = StandardScaler()
x_train = transfer.fit_transform(x_train)
x_valid = transfer.transform(x_valid)
# 构建数据集,转换为pytorch的形式
train_dataset = TensorDataset(torch.from_numpy(x_train),
torch.tensor(y_train.values))
valid_dataset = TensorDataset(torch.from_numpy(x_valid),
torch.tensor(y_valid.values))
# 返回结果
return train_dataset, valid_dataset, x_train.shape[1],
len(np.unique(y))

# 构建网络模型
class PhonePriceModel(nn.Module):

    def __init__(self, input_dim, output_dim):
        super(PhonePriceModel, self).__init__()
        # 优化②:增加网络深度
        # 1. 第一层: 输入为维度为 20, 输出维度为: 128
        self.linear1 = nn.Linear(input_dim, 128)
        # 2. 第二层: 输入为维度为 128, 输出维度为: 256
        self.linear2 = nn.Linear(128, 256)
        # 3. 第三层: 输入为维度为 256, 输出维度为: 512
        self.linear3 = nn.Linear(256, 512)
        # 4. 第四层: 输入为维度为 512, 输出维度为: 128
        self.linear4 = nn.Linear(512, 128)
        # 5. 输出层: 输入为维度为 128, 输出维度为: 4
        self.linear5 = nn.Linear(128, output_dim)

    def forward(self, x):
        # 前向传播过程
        x = torch.relu(self.linear1(x))
        x = torch.relu(self.linear2(x))
        x = torch.relu(self.linear3(x))
        x = torch.relu(self.linear4(x))
        # 后续CrossEntropyLoss损失函数中包含softmax过程, 所以当前步骤不进行softmax操作
        output = self.linear5(x)
        # 获取数据结果
        return output

# 编写训练函数
def train(train_dataset, input_dim, class_num):

```

```

# 固定随机数种子
torch.manual_seed(0)
# 初始化数据加载器
dataloader = DataLoader(train_dataset, shuffle=True,
batch_size=8)
# 初始化模型
model = PhonePriceModel(input_dim, class_num)
# 损失函数 CrossEntropyLoss = softmax + 损失计算
criterion = nn.CrossEntropyLoss()
# 优化器:使用Adam优化方法, 优化器:学习率变为1e-4
optimizer = optim.Adam(model.parameters(), lr=1e-4)
# 遍历每个轮次的数据
num_epoch = 50
for epoch_idx in range(num_epoch):
    # 训练时间
    start = time.time()
    # 计算损失
    total_loss = 0.0
    total_num = 0
    # 遍历每个batch数据进行处理
    for x, y in dataloader:
        model.train()
        output = model(x)
        # 计算损失
        loss = criterion(output, y)
        # 梯度清零
        optimizer.zero_grad()
        # 反向传播
        loss.backward()
        # 参数更新
        optimizer.step()
        # 损失计算
        total_num += len(y)
        total_loss += loss.item() * len(y)
    # 打印损失变换结果
    print('epoch: %4s loss: %.2f, time: %.2fs' %
          (epoch_idx + 1, total_loss / total_num, time.time()
- start))
    # 模型保存
    torch.save(model.state_dict(), './model/phone-price-
model12.pth')

def evaluate(valid_dataset, input_dim, class_num):
    # 加载模型和训练好的网络参数

```

```

model = PhonePriceModel(input_dim, class_num)
# load_state_dict:将加载的参数字典应用到模型上
# load:加载用来保存模型参数的文件
model.load_state_dict(torch.load('./model/phone-price-
model2.pth'))
# 构建加载器
dataloader = DataLoader(valid_dataset, batch_size=8,
shuffle=False)
# 评估测试集
correct = 0
# 遍历测试集中的数据
for x, y in dataloader:
    # 将其送入网络中
    # model.eval()
    output = model(x)
    # 获取预测类别结果
    y_pred = torch.argmax(output, dim=1)
    # 获取预测正确的个数
    correct += (y_pred == y).sum()
# 求预测精度
print('Acc: %.5f' % (correct / len(valid_dataset)))

if __name__ == '__main__':
    train_dataset, valid_dataset, input_dim, class_num =
create_dataset()
    train(train_dataset, input_dim, class_num)
    evaluate(valid_dataset, input_dim, class_num)

```

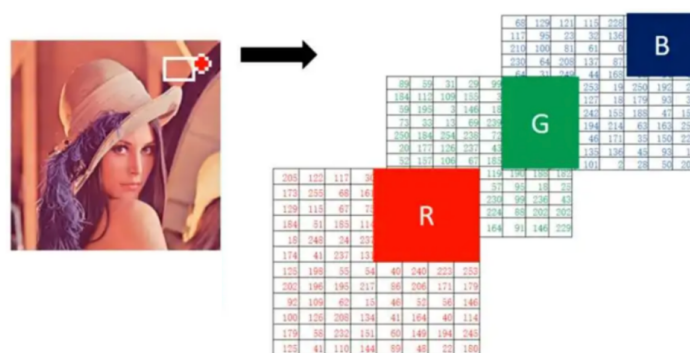
四、卷积神经网络(CNN)

1 简介

图像的基本概念

图像是由像素点组成的，每个像素点的取值范围为: [0, 255]。像素值越接近于0，颜色越暗，接近于黑色；像素值越接近于255，颜色越亮，接近于白色。

在深度学习中，我们使用的图像大多是彩色图，彩色图由RGB3个通道组成，如下图所示



卷积神经网络 (Convolutional Neural Network) 是含有卷积层的神经网络. 卷积层的作用就是用来自动学习、提取图像的特征.

CNN网络主要由三部分构成：卷积层、池化层和全连接层构成：

1. 卷积层负责提取图像中的局部特征；
2. 池化层用来大幅降低参数量级(降维)；
3. 全连接层用来输出想要的结果。

CNN概述

图中CNN要做的事情是：给定一张图片，是车还是马未知，是什么车也未知，现在需要模型判断这张图片里具体是一个什么东西，总之输出一个结果：如果是车 那是什么车

最左边是

数据输入层：对数据做一些处理，比如去均值（各维度都减对应维度的均值，使得输入数据各个维度都中心化0，避免数据过多偏差，影响训练效果）、归一化（把所有的数据都归一到同样的范围）、PCA等等。CNN只对训练集做“去均值”这一步。

中间是

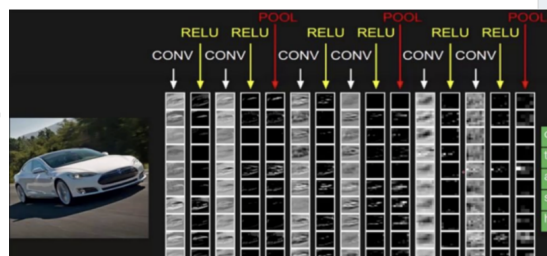
卷积层(CONV)：线性乘积求和，提取图像中的局部特征

激励层(RELU)：ReLU激活函数,输入数据转换成输出数据

池化层(POOL)：取区域平均值或最大值，大幅降低参数量级(降维)

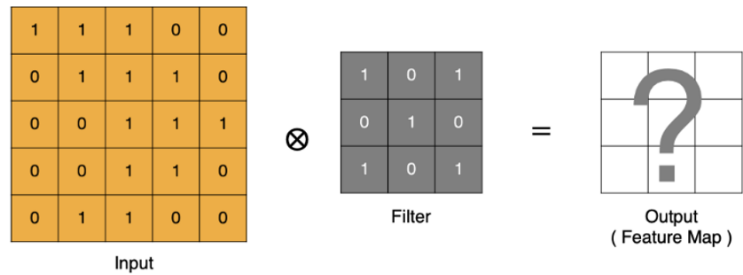
最右边是

全连接层(FC)：输出CNN模型预测结果



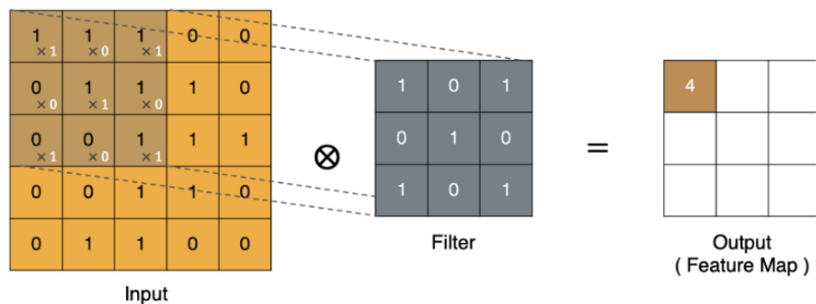
2 卷积层

卷积计算

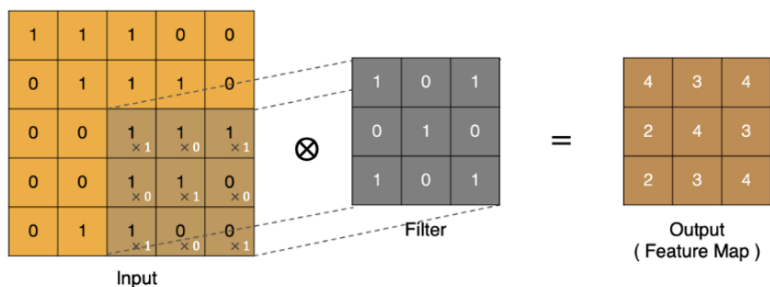


1. input 表示输入的图像
2. filter 表示卷积核, 也叫做卷积核(滤波矩阵)
3. input 经过 filter 得到输出为最右侧的图像, 该图叫做特征图

卷积运算本质上就是在卷积核和输入数据的局部区域间做点积。



最终的特征图结果为:

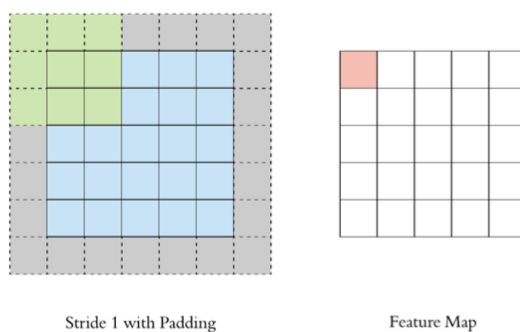


2.1 Padding(填充)

通过上面的卷积计算过程, 最终的特征图比原始图像小很多, 如果想要保持经过卷积后的图像大小不变, 可以在原图周围添加 padding 来实现。

padding (填充) 操作用于处理卷积时图像边缘的像素。

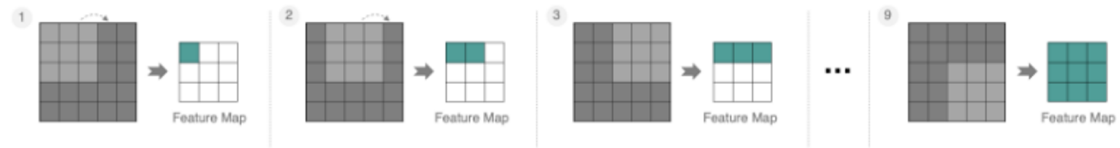
其目的是在输入图像的边界周围添加额外的像素 (通常是零), 从而解决卷积操作时边缘信息丢失的问题。



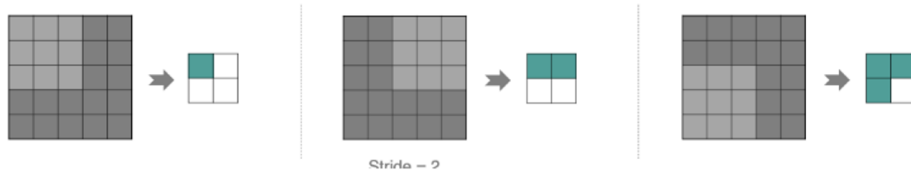
2.2 Stride(步长)

stride (步长) 指的是卷积核在图像上滑动时的步伐大小，即每次卷积时卷积核在图像中向右（或向下）移动的像素数。步长直接影响卷积操作后输出特征图的尺寸，以及计算量和模型的特征提取能力。

按照步长为1来移动卷积核，计算特征图如下所示：



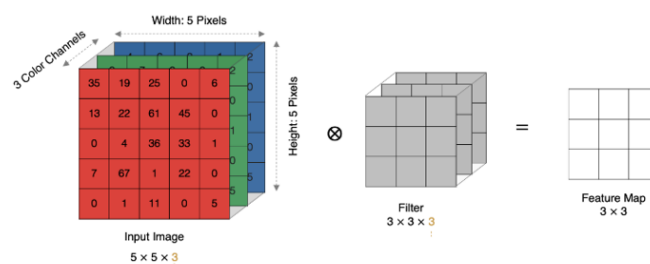
如果把 Stride 增大为2，也是可以提取特征图的，如下图所示：



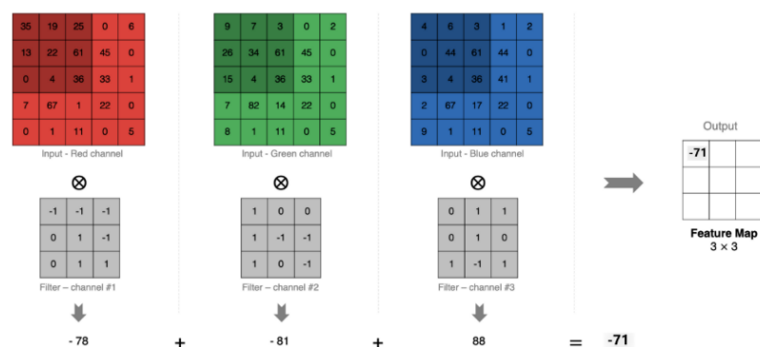
2.3 多通道卷积

多通道卷积计算

实际中的图像都是多个通道组成的，我们怎么计算卷积呢？

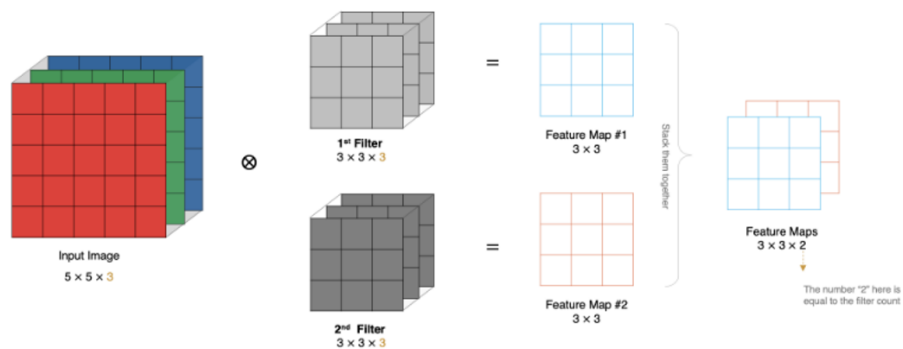


如下图所示：



2.4 多卷积核卷积

当使用多个卷积核时, 应该怎么进行特征提取呢?



Input Volume (+pad 1) (7x7x3)	Filter W0 (3x3x3)	Filter W1 (3x3x3)	Output Volume (3x3x2)																																																																												
$x[:, :, 0]$ <table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>2</td><td>2</td><td>0</td></tr><tr><td>0</td><td>0</td><td>1</td><td>1</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td><td>1</td><td>1</td><td>1</td><td>0</td></tr><tr><td>0</td><td>0</td><td>2</td><td>0</td><td>1</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	0	0	0	0	0	1	1	2	2	0	0	0	1	1	0	0	0	0	1	1	0	1	0	0	0	1	0	1	1	1	0	0	0	2	0	1	0	0	0	0	0	0	0	0	0	$w0[:, :, 0]$ <table><tr><td>1</td><td>1</td><td>-1</td></tr><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>-1</td><td>-1</td><td>0</td></tr></table>	1	1	-1	-1	0	1	-1	-1	0	$w1[:, :, 0]$ <table><tr><td>-1</td><td>-1</td><td>0</td></tr><tr><td>-1</td><td>1</td><td>0</td></tr><tr><td>-1</td><td>1</td><td>0</td></tr></table>	-1	-1	0	-1	1	0	-1	1	0	$o[:, :, 0]$ <table><tr><td>1</td><td>0</td><td>-3</td></tr><tr><td>-6</td><td>1</td><td>1</td></tr><tr><td>4</td><td>-3</td><td>1</td></tr></table>	1	0	-3	-6	1	1	4	-3	1
0	0	0	0	0	0	0																																																																									
0	0	1	1	2	2	0																																																																									
0	0	1	1	0	0	0																																																																									
0	1	1	0	1	0	0																																																																									
0	1	0	1	1	1	0																																																																									
0	0	2	0	1	0	0																																																																									
0	0	0	0	0	0	0																																																																									
1	1	-1																																																																													
-1	0	1																																																																													
-1	-1	0																																																																													
-1	-1	0																																																																													
-1	1	0																																																																													
-1	1	0																																																																													
1	0	-3																																																																													
-6	1	1																																																																													
4	-3	1																																																																													
$x[:, :, 1]$ <table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td><td>1</td><td>2</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>2</td><td>1</td><td>1</td><td>2</td><td>0</td></tr><tr><td>0</td><td>1</td><td>2</td><td>0</td><td>0</td><td>2</td><td>0</td></tr><tr><td>0</td><td>0</td><td>2</td><td>1</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>2</td><td>0</td><td>1</td><td>2</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	0	0	0	0	1	1	1	2	0	0	0	0	2	1	1	2	0	0	1	2	0	0	2	0	0	0	2	1	2	1	0	0	2	0	1	2	0	0	0	0	0	0	0	0	0	$w0[:, :, 1]$ <table><tr><td>-1</td><td>0</td><td>-1</td></tr><tr><td>0</td><td>0</td><td>-1</td></tr><tr><td>1</td><td>-1</td><td>0</td></tr></table>	-1	0	-1	0	0	-1	1	-1	0	$w1[:, :, 1]$ <table><tr><td>1</td><td>-1</td><td>0</td></tr><tr><td>-1</td><td>0</td><td>-1</td></tr><tr><td>-1</td><td>0</td><td>0</td></tr></table>	1	-1	0	-1	0	-1	-1	0	0	$o[:, :, 1]$ <table><tr><td>-1</td><td>-6</td><td>-4</td></tr><tr><td>-2</td><td>-3</td><td>-4</td></tr><tr><td>-1</td><td>-3</td><td>-3</td></tr></table>	-1	-6	-4	-2	-3	-4	-1	-3	-3
0	0	0	0	0	0	0																																																																									
0	1	1	1	2	0	0																																																																									
0	0	2	1	1	2	0																																																																									
0	1	2	0	0	2	0																																																																									
0	0	2	1	2	1	0																																																																									
0	2	0	1	2	0	0																																																																									
0	0	0	0	0	0	0																																																																									
-1	0	-1																																																																													
0	0	-1																																																																													
1	-1	0																																																																													
1	-1	0																																																																													
-1	0	-1																																																																													
-1	0	0																																																																													
-1	-6	-4																																																																													
-2	-3	-4																																																																													
-1	-3	-3																																																																													
$x[:, :, 2]$ <table><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>2</td><td>0</td><td>2</td><td>0</td><td>2</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td><td>2</td><td>2</td><td>1</td><td>0</td></tr><tr><td>0</td><td>2</td><td>0</td><td>2</td><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>1</td><td>1</td><td>2</td><td>0</td></tr><tr><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td><td>0</td></tr></table>	0	0	0	0	0	0	0	0	2	0	2	0	2	0	0	0	0	1	2	1	0	0	1	0	2	2	1	0	0	2	0	2	0	0	0	0	0	0	1	1	2	0	0	0	0	0	0	0	0	$w0[:, :, 2]$ <table><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>-1</td><td>1</td></tr></table>	0	1	0	1	0	1	0	-1	1	$w1[:, :, 2]$ <table><tr><td>-1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>0</td><td>-1</td><td>0</td></tr></table>	-1	0	1	1	0	1	0	-1	0										
0	0	0	0	0	0	0																																																																									
0	2	0	2	0	2	0																																																																									
0	0	0	1	2	1	0																																																																									
0	1	0	2	2	1	0																																																																									
0	2	0	2	0	0	0																																																																									
0	0	0	1	1	2	0																																																																									
0	0	0	0	0	0	0																																																																									
0	1	0																																																																													
1	0	1																																																																													
0	-1	1																																																																													
-1	0	1																																																																													
1	0	1																																																																													
0	-1	0																																																																													
Bias $b0$ (1x1x1) $b0[:, :, 0]$ <table><tr><td>1</td></tr></table>	1	Bias $b1$ (1x1x1) $b1[:, :, 0]$ <table><tr><td>0</td></tr></table>	0																																																																												
1																																																																															
0																																																																															

toggle movement

toggle movement

2.5 特征图大小

特征图大小

输出特征图的大小与以下参数息息相关:

1. size: 卷积核/过滤器大小, 一般会选择为奇数, 比如有 1*1、3*3、5*5
2. Padding: 零填充的方式
3. Stride: 步长

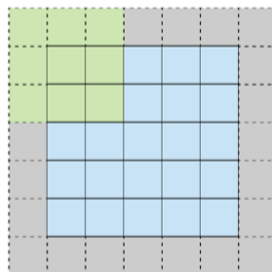
那计算方法如下图所示:

1. 输入图像大小: $W \times W$
2. 卷积核大小: $F \times F$
3. Stride: S
4. Padding: P
5. 输出图像大小: $N \times N$

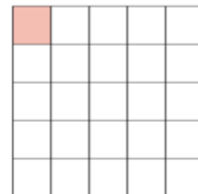
$$N = \frac{W - F + 2P}{S} + 1$$

以下图为例:

1. 图像大小: 5×5
2. 卷积核大小: 3×3
3. Stride: 1
4. Padding: 1
5. $(5 - 3 + 2) / 1 + 1 = 5$, 即得到的特征图大小为: 5×5



Stride 1 with Padding



Feature Map

代码:

"""

案例:

演示卷积层的API, 用于 提取图像的局部特征, 获取: 特征图(Feature Map)

卷积神经网络介绍:

概述:

全称叫: convolutional neural network, 即: 包含卷积层的神经网络.

组成:

卷积层(Convolutional):

用于提取图像的 局部特征，结合 卷积核(每个卷积核 = 1个神经元) 实现，处理后的结果叫：特征图。

池化层(Pooling):

用于 降维，降采样

全连接层(Full Connected, fc, linear):

用于 预测结果，并输出结果的。

特征图计算方式:

$$N = (W - F + 2 * P) / S + 1$$

W: 输入图像的大小

F: 卷积核的大小

P: 填充的大小

S: 步长

N: 输出图像的大小(特征图大小)

"""

导包

```
import torch
```

```
import torch.nn as nn
```

```
import matplotlib.pyplot as plt
```

```
import os
```

```
os.environ["KMP_DUPLICATE_LIB_OK"]="TRUE"
```

1. 定义函数，用于完成图像的加载，卷积，特征图可视化操作。

```
def dm01():
```

1. 加载RGB真彩图。

```
img = plt.imread('./data/a.jpg')
```

2. 打印读取到的图像信息。

```
# print(f'img: {img}, shape: {img.shape}')      # HWC (640, 640, 3)
```

3. 把图像的形状从 HWC -> CHW, 思路: img -> 张量 -> 转换维度。

```
img2 = torch.tensor(img, dtype=torch.float)
```

```
img2 = img2.permute(2, 0, 1)
```

```
# print(f'img2: {img2}, shape: {img2.shape}')    # [3, 640, 640]
```

4. 因为这里只有1张图，所以我们给它增加1个维度，从 CHW -> (1, C, H, W), 1张3通道的 640*640像素的 图

```
img3 = img2.unsqueeze(dim=0)
```

```
# print(f'img3: {img3}, shape: {img3.shape}')    # [1, 3, 640, 640]
```

5. 创建卷积层对象，提取 特征图。

参1: 输入图像的通道数, 参2: 输出图像的通道数(几个特征图), 参3: 卷积核的大小, 参4: 步长, 参5: 填充。

```
conv = nn.Conv2d(3, 4, 3, 2, 0)
```

```

# 6. 具体的卷积计算.
conv_img = conv(img3)

# 7. 打印卷积后的结果.    1张4通道的 319*319像素的 图
# print(f'conv_img: {conv_img}, shape: {conv_img.shape}') #
(1, 4, 319, 319)

# 8. 查看提取到的 4个 特征图.
img4 = conv_img[0]
# print(f'img4: {img4}, shape: {img4.shape}')          # (4, 319,
319) -> CHW

# 9. 把上述的图从 CHW -> HWC
img5 = img4.permute(1, 2, 0)
# print(f'img5: {img5}, shape: {img5.shape}')          # (319,
319, 4) -> HWC

# 10. 可视化第1个通道的特征图.
feature1 = img5[:, :, 3].detach().numpy()              # 第0通道
(即: 第1通道的) (319, 319)像素图
plt.imshow(feature1)
plt.show()

# 2. 测试
if __name__ == '__main__':
    dm01()

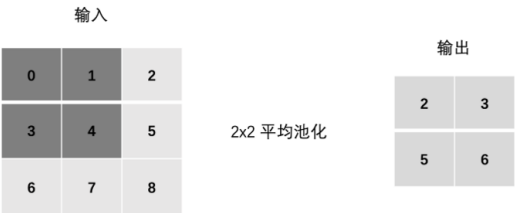
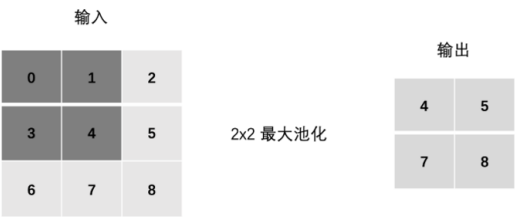
```

3 池化层

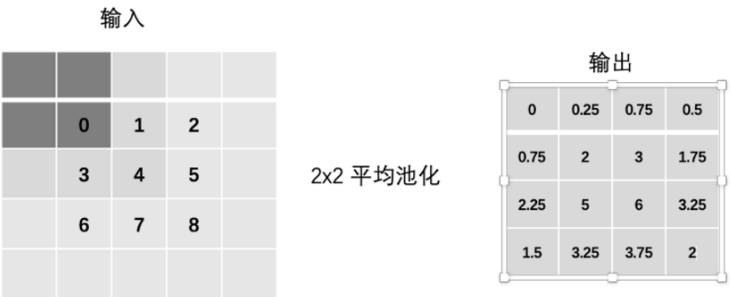
池化层计算

池化层 (Pooling) 降低维度, 从而减少计算量、减少内存消耗，并提高模型的鲁棒性。

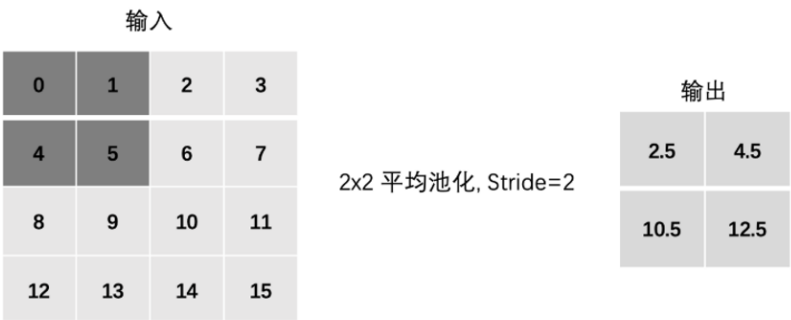
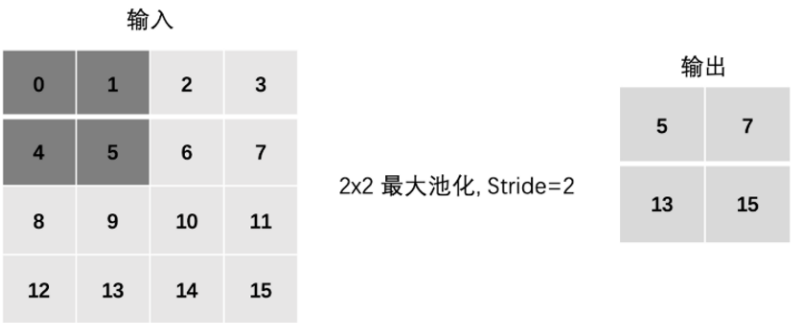
池化层通常位于卷积层之后，它通过对卷积层输出的特征图进行下采样，保留最重要的特征信息，同时丢弃一些不重要的细节。



Padding

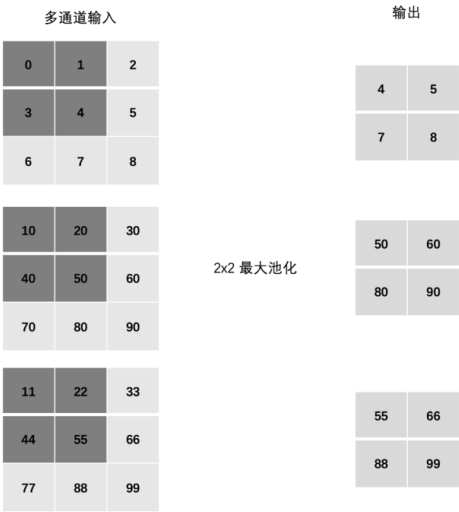


Stride



多通道池化层计算

在处理多通道输入数据时，池化层对每个输入通道分别池化，而不是像卷积层那样将各个通道的输入相加。这意味着池化层的输出和输入的通道数是相等。



代码：

.....

案例：
演示池化层相关操作。

池化层解释(Pooling)：

目的：
降维。

思路：
最大池化。
平均池化。

特点：

池化不会改变数据的 通道数.

"""

导包

import torch

import torch.nn as nn

1. 定义函数，演示单通道池化.

def dm01():

1. 创建1个 1通道 3*3的二维矩阵.

```
inputs = torch.tensor([      # 1 通道C
    [      # 3 高度H
        [0, 1, 2],      # 3 宽度W
        [3, 4, 5],
        [6, 7, 8]
    ]
])
```

print(f'inputs: {inputs}, shape: {inputs.shape}') # (1, 3, 3)

2. 创建最大池化层.

参1: 池化核(池化窗口)大小, 参2: 步长, 参3: 填充.

pool1 = nn.MaxPool2d(2, 1, 0)

output = pool1(inputs)

print(f'output: {output}, shape: {output.shape}') # (1, 2, 2)

3. 创建平均池化层.

pool2 = nn.AvgPool2d(2, 1, 0)

output = pool2(inputs)

print(f'output: {output}, shape: {output.shape}') # (1, 2, 2)

2. 定义函数，演示多通道池化.

def dm02():

1. 创建1个 3通道 3*3的二维矩阵.

```
inputs = torch.tensor([      # 3 通道C
    [      # 通道1, HW 3,3
        [0, 1, 2],
        [3, 4, 5],
        [6, 7, 8]
    ],
```

```

        [
            # 通道2, HW 3,3
            [10, 20, 30],
            [40, 50, 60],
            [70, 80, 90]
        ],

        [
            # 通道3, HW 3,3
            [11, 22, 33],
            [44, 55, 66],
            [77, 88, 99]
        ]
    ])
    # print(f'inputs: {inputs}, shape: {inputs.shape}')    # (3,
3, 3)

    # 2. 创建最大池化层.
    # 参1: 池化核(池化窗口)大小, 参2: 步长, 参3: 填充.
    pool1 = nn.MaxPool2d(2, 1, 0)
    output = pool1(inputs)
    print(f'output: {output}, shape: {output.shape}')    # (3, 2,
2)

    # 3. 创建平均池化层.
    pool2 = nn.AvgPool2d(2, 1, 0)
    output = pool2(inputs)
    print(f'output: {output}, shape: {output.shape}')    # (3, 2,
2)

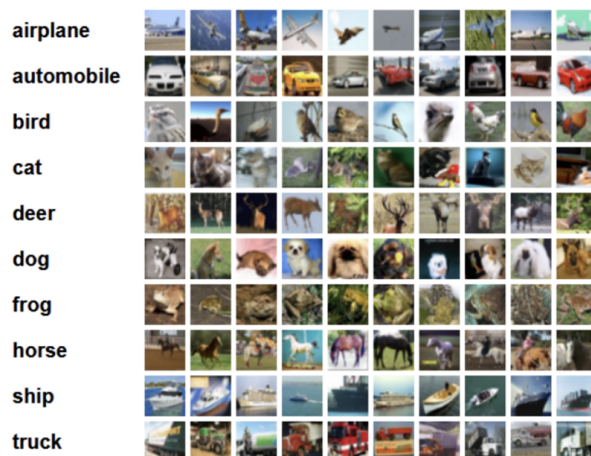
# 3. 测试.
if __name__ == '__main__':
    # dm01()
    dm02()

```

图像分类案例

CIFAR10 数据集

CIFAR-10数据集5万张训练图像、1万张测试图像、10个类别、每个类别有6k个图像，图像大小 $32 \times 32 \times 3$ 。下图列举了10个类，每一类随机展示了10张图片：



京东数智化人才培训

我们要搭建的网络结构如下：



我们要搭建的网络结构如下：

1. 输入形状: 32×32
2. 第一个卷积层输入 3 个 Channel, 输出 6 个 Channel, Kernel Size 为: 3×3
3. 第一个池化层输入 30×30 , 输出 15×15 , Kernel Size 为: 2×2 , Stride 为: 2
4. 第二个卷积层输入 6 个 Channel, 输出 16 个 Channel, Kernel Size 为 3×3
5. 第二个池化层输入 13×13 , 输出 6×6 , Kernel Size 为: 2×2 , Stride 为: 2
6. 第一个全连接层输入 576 维, 输出 120 维
7. 第二个全连接层输入 120 维, 输出 84 维
8. 最后的输出层输入 84 维, 输出 10 维

我们在每个卷积计算之后应用 relu 激活函数来给网络增加非线性因素。

从程序的运行结果来看，网络模型在测试集上的准确率并不高。我们可以从以下几个方面来进行优化：

1. 增加卷积核输出通道数
2. 增加全连接层的参数量
3. 调整学习率
4. 调整优化方法
5. 修改激活函数
6. 等等...

代码：

```
"""
```

案例：

演示CNN的综合案例，图像分类。

回顾：深度学习项目的步骤

1. 准备数据集。

这里我们用的时候 计算机视觉模块 `torchvision`自带的 `CIFAR10`数据集，包含6w张 `(32,32,3)`的图片，5w张训练集，1w张测试集，10个分类，每个分类6K张图片。

你需要单独安装一下 `torchvision`包，即：`pip install torchvision`

2. 搭建(卷积)神经网络
3. 模型训练。
4. 模型测试。

卷积层：

提取图像的局部特征 -> 特征图(Feature Map)，计算方式： $N = (W - F + 2P) // S + 1$

每个卷积核都是1个神经元。

池化层：

降维，有最大池化 和 平均池化。

池化只在HW上做调整，通道上不改变。

案例的优化思路：

1. 增加卷积核的输出通道数(大白话：卷积核的数量)
2. 增加全连接层的参数量.
3. 调整学习率
4. 调整优化方法(optimizer...)
5. 调整激活函数...
6. ...

"""

导包

```
import torch
import torch.nn as nn
from torchvision.datasets import CIFAR10
from torchvision.transforms import ToTensor # pip install
torchvision -i https://mirrors.aliyun.com/pypi/simple/
import torch.optim as optim
from torch.utils.data import DataLoader
import time
import matplotlib.pyplot as plt
from torchsummary import summary
```

每批次样本数

BATCH_SIZE = 8

1. 准备数据集.

def create_dataset():

1. 获取训练集.

参1: 数据集路径. 参2: 是否是训练集. 参3: 数据预处理 -> 张量数据. 参4:
 是否联网下载(直接用我给的, 不用下)

train_dataset = CIFAR10(root='./data', train=True,
transform=ToTensor(), download=True)

2. 获取测试集.

test_dataset = CIFAR10(root='./data', train=False,
transform=ToTensor(), download=True)

3. 返回数据集.

return train_dataset, test_dataset

2. 搭建(卷积)神经网络

class ImageModel(nn.Module):

1. 初始化父类成员, 搭建神经网络.

def __init__(self):

1.1 初始化父类成员.

super().__init__()

1.2 搭建神经网络.

第1个卷积层, 输入 3通道, 输出6通道, 卷积核大小3*3, 步长1, 填充0

`self.conv1 = nn.Conv2d(3, 6, 3, 1, 0)`

第1个池化层, 窗口大小 2*2, 步长2, 填充0

`self.pool1 = nn.MaxPool2d(2, 2, 0)`

第2个卷积层, 输入 6通道, 输出16通道, 卷积核大小3*3, 步长1, 填充0

`self.conv2 = nn.Conv2d(6, 16, 3, 1, 0)`

第2个池化层, 窗口大小 2*2, 步长2, 填充0

`self.pool2 = nn.MaxPool2d(2, 2, 0)`

第1个隐藏层(全连接层), 输入: 576, 输出: 120

`self.linear1 = nn.Linear(576, 120)`

第2个隐藏层 (全连接层), 输入: 120, 输出: 84

`self.linear2 = nn.Linear(120, 84)`

第3个隐藏层 (全连接层) -> 输出层, 输入: 84, 输出: 10

`self.output = nn.Linear(84, 10)`

2. 定义前向传播

`def forward(self, x):`

第1层: 卷积层(加权求和) + 激励层(激活函数) + 池化层(降维)

分解版.

`x = self.conv1(x)` # 卷积层

`x = torch.relu(x)` # 激励层

`x = self.pool1(x)` # 池化层

合并版 池化 + 激活函数 + 卷积

`x = self.pool1(torch.relu(self.conv1(x)))`

第2层: 卷积层(加权求和) + 激励层(激活函数) + 池化层(降维)

`x = self.pool2(torch.relu(self.conv2(x)))`

细节: 全连接层只能处理二维数据, 所以要将数据进行拉平 (8, 16, 6, 6) -> (8, 576)

参1: 样本数(行数), 参2: 列数(特征数), -1表示自动计算.

`x = x.reshape(x.size(0), -1)` # 8行576列

`print(f'x.shape: {x.shape}')`

第3层: 全连接层(加权求和) + 激励层(激活函数)

`x = torch.relu(self.linear1(x))`

第4层: 全连接层(加权求和) + 激励层(激活函数)

`x = torch.relu(self.linear2(x))`

第5层: 全连接层(加权求和) -> 输出层

```

        return self.output(x) # 后续用 多分类交叉熵损失函数
CrossEntropyLoss = softmax()激活函数 + 损失计算.

# 3. 模型训练.
def train(train_dataset):
    # 1. 创建数据加载器.
    dataloader = DataLoader(train_dataset, batch_size=BATCH_SIZE,
                             shuffle=True)
    # 2. 创建模型对象.
    model = ImageModel()
    # 3. 创建损失函数对象.
    criterion = nn.CrossEntropyLoss() # 多分类交叉熵损失函数 =
softmax()激活函数 + 损失计算.
    # 4. 创建优化器对象.
    optimizer = optim.Adam(model.parameters(), lr=1e-3)
    # 5. 循环遍历epoch, 开始 每轮的 训练动作.
    # 5.1 定义变量, 记录训练的总轮数.
    epochs = 20
    # 5.2 遍历, 完成每轮的 所有批次的 训练动作.
    for epoch_idx in range(epochs):
        # 5.2.1 定义变量, 记录: 总损失, 总样本数据量, 预测正确样本个数, 训练(开始)时间
        total_loss, total_samples, total_correct, start = 0.0, 0,
0, time.time()
        # 5.2.2 遍历数据加载器, 获取到 每批次的数据.
        for x, y in dataloader:
            # 5.2.3 切换训练模式.
            model.train()
            # 5.2.4 模型预测.
            y_pred = model(x)
            # 5.2.5 计算损失.
            loss = criterion(y_pred, y)
            # 5.2.6 梯度清零 + 反向传播 + 参数更新
            optimizer.zero_grad()
            loss.backward()
            optimizer.step()

            # 5.2.7 统计预测正确的样本个数.
            # print(y_pred) # 批次中, 每张图 每个分类的 预测概率.

            # argmax() 返回最大值对应的索引, 充当 -> 该图片的 预测分类.
            # tensor([9, 8, 5, 5, 1, 5, 8, 5])
            # print(torch.argmax(y_pred, dim=-1)) # -1这里表示
行. 预测分类

```

```

        # print(y)                                # 真实分类
        # print(torch.argmax(y_pred, dim=-1) == y)    # 是否预
测正确

        # print((torch.argmax(y_pred, dim=-1) == y).sum())
# 预测正确的样本个数.
        total_correct += (torch.argmax(y_pred, dim=-1) ==
y).sum()

        # 5.2.8 统计当前批次的总损失.                第1批平均损失 * 第1批
样本个数

        total_loss += loss.item() * len(y) # [第1批总损失 + 第
2批总损失 + 第3批总损失 + ...]
        # 5.2.9 统计当前批次的总样本个数.
        total_samples += len(y)
        # break 每轮只训练1批，提高训练效率，减少训练时长，只有测试
会这么写，实际开发绝不要这样做.

        # 5.2.10 走这里，说明一轮训练完毕，打印该轮的训练信息.
        print(f'epoch: {epoch_idx + 1}, loss: {total_loss /
total_samples:.5f}, acc:{total_correct / total_samples:.2f},
time:{time.time() - start:.2f}s')
        # break      # 这里写break，意味着只训练一轮.

# 6. 保存模型.
torch.save(model.state_dict(), './model/image_model.pth')

# 4. 模型测试.
def evaluate(test_dataset):
    # 1. 创建测试集 数据加载器.
    dataloader = DataLoader(test_dataset, batch_size=BATCH_SIZE,
shuffle=False)
    # 2. 创建模型对象.
    model = ImageModel()
    # 3. 加载模型参数.
    model.load_state_dict(torch.load('./model/image_model.pth'))
# pickle文件
    # 4. 定义变量统计 预测正确的样本个数，总样本个数.
    total_correct, total_samples = 0, 0
    # 5. 遍历数据加载器，获取到 每批次 的数据.
    for x, y in dataloader:
        # 5.1 切换模型模式.
        model.eval()
        # 5.2 模型预测.
        y_pred = model(x)

```

5.3 因为训练的时候用了CrossEntropyLoss，所以搭建神经网络时没有加softmax()激活函数，这里要用 argmax()来模拟。

argmax()函数功能： 返回最大值对应的索引，充当 -> 该图片的 预测分类。

```
y_pred = torch.argmax(y_pred, dim=-1) # -1 这里表示行。
```

5.4 统计预测正确的样本个数。

```
total_correct += (y_pred == y).sum()
```

5.5 统计总样本个数。

```
total_samples += len(y)
```

6. 打印正确率(预测结果)。

```
print(f'Acc: {total_correct / total_samples:.2f}')
```

5. 测试

```
if __name__ == '__main__':
```

1. 获取数据集。

```
train_dataset, test_dataset = create_dataset()
```

```
# print(f'训练集: {train_dataset.data.shape}') # (50000, 32, 32, 3)
```

```
# print(f'测试集: {test_dataset.data.shape}') # (10000, 32, 32, 3)
```

```
## {'airplane': 0, 'automobile': 1, 'bird': 2, 'cat': 3, 'deer': 4, 'dog': 5, 'frog': 6, 'horse': 7, 'ship': 8, 'truck': 9}
```

```
# print(f'数据集类别: {train_dataset.class_to_idx}')
```

```
#
```

图像展示

```
# plt.figure(figsize=(2, 2))
```

```
# plt.imshow(train_dataset.data[1111]) # 索引为1111的图像
```

```
# plt.title(train_dataset.targets[1111])
```

```
# plt.show()
```

2. 搭建神经网络。

```
# model = ImageModel()
```

查看模型参数，参1：模型，参2：输入维度(CHW，通道，高，宽)，参3：批次大小

```
# summary(model, (3, 32, 32), batch_size=1)
```

3. 模型训练。

```
# train(train_dataset)
```

4. 模型测试。

```
evaluate(test_dataset)
```

五、循环神经网络(RNN)

1 简介

RNN介绍

循环神经网络 (Recurrent Neural Network, RNN) 是一种**专门处理序列数据的神经网络**。与传统的前馈神经网络不同, RNN具有“循环”结构, 能够处理和记住前面时间步的信息, 使其特别适用于时间序列数据或有时序依赖的任务。

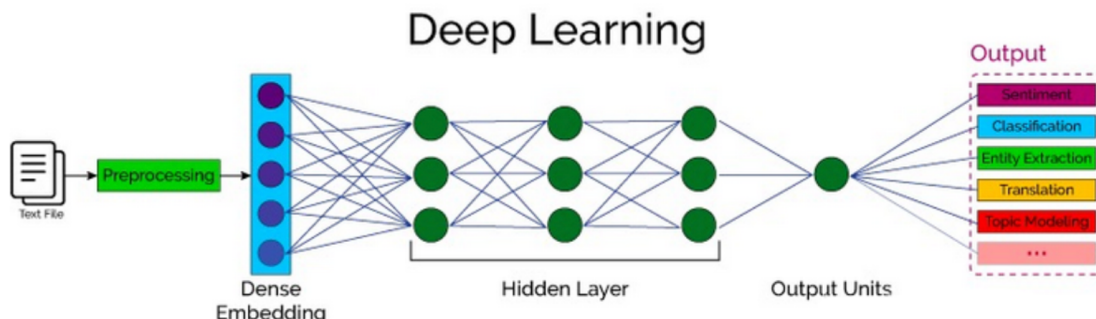
我们要明确什么是**序列数据**, 时间序列数据是指在不同时间点上收集到的数据, 这类数据反映了某一事物、现象等随时间的变化状态或程度。这是时间序列数据的定义, 当然这里也可以不是时间, 比如文字序列, 但总归序列数据有一个特点——**后面的数据跟前面的数据有关系**。

RNN的应用:

- **自然语言处理 (NLP)** : 文本生成、语言建模、机器翻译、情感分析等。
- **时间序列预测**: 股市预测、气象预测、传感器数据分析等。
- **语音识别**: 将语音信号转换为文字。
- **音乐生成**: 通过学习音乐的时序模式来生成新乐曲。

自然语言处理概述

自然语言处理 (Nature language Processing, NLP) 研究的主要是通过计算机算法来理解自然语言。对于自然语言来说, 处理的数据主要就是人类的语言, 例如: 汉语、英语、法语等, 该类型的数据不像我们前面接触过的结构化数据、或者图像数据可以很方便的进行数值化。



2 词嵌入层

词嵌入层作用

词嵌入层的作用就是将文本转换为向量。

词嵌入层在 RNN 中的作用有**输入表示**、**降低维度**和**捕捉语义相似性**。

词嵌入层首先会根据输入的的词的数量构建一个**词向量矩阵**，例如：我们有 100 个词，每个词希望转换成 128 维度的向量，那么构建的矩阵形状即为：100*128，输入的每个词都对应了一个该矩阵中的一个向量。

cat =>	1.2	-0.1	4.3	3.2
mat =>	0.4	2.5	-0.9	0.5
on =>	2.1	0.3	0.1	0.4
...				

词嵌入层工作流程

- **初始化词向量**：词嵌入层的初始词向量通常会使用随机初始化或者通过加载预训练的词向量（如 Word2Vec或GloVe）进行初始化。
- **输入索引**：每个单词在词汇表中都有一个唯一的索引。输入文本（例如一个句子）会先被分词，然后每个单词会被转换为相应的索引。
- **查找词向量**：词嵌入层将这些单词索引映射为对应的词向量。这些词向量是一个低维稠密向量，表示该词的语义。
- **输入到RNN**：这些词向量作为RNN的输入，RNN处理它们并根据上下文生成一个序列的输出。

词嵌入层使用

在 `PyTorch` 中，使用 `nn.Embedding` 词嵌入层来实现输入词的向量化。

```
nn.Embedding(num_embeddings=10, embedding_dim=4)
```

`nn.Embedding` 对象构建时，最主要有两个参数：

1. `num_embeddings` 表示**词的数量**
2. `embedding_dim` 表示**用多少维的向量来表示每个词**

词嵌入层使用

接下来，我们将会学习如何将词转换为词向量，其步骤如下：

1. 先将语料进行分词，构建词与索引的映射，我们可以把这个映射叫做词表，词表中每个词都对应了一个唯一的索引
2. 然后使用 `nn.Embedding` 构建词嵌入矩阵，词索引对应的向量即为该词对应的数值化后的向量表示。

例如，我们的文本数据为："北京冬奥的进度条已经过半，不少外国运动员在完成自己的比赛后踏上归途。"

代码：

```
"""
```

案例：

演示词嵌入层的API应用。

RNN介绍：

全称叫**Recurrent neural network**，循环神经网络，主要处理 序列数据的。

序列数据：后边数据对前边的数据有依赖，例如：天气预测，股市分析，文本生成...

组成：

词嵌入层

循环网络层

输出层

词嵌入层介绍(作用)：

把词（或者 词对应的索引）转成 词向量。

```
"""
```

```
# 导包
```

```
import torch
```

```
import jieba                # jieba分词器，需要安装一下，pip install
```

```
jieba
```

```
import torch.nn as nn
```

```
# 1. 定义函数，用于演示 词嵌入层的API，如何把 词(词的索引) -> 词向量
```

```
def dm01():
```

```
    # 1. 定义一句话
```

```
    text = '北京冬奥的进度条已经过半，不少外国运动员在完成自己的比赛后踏上归途。'
```

```
    # 2. 使用jieba模块进行分词。
```

```
    words = jieba.lcut(text)
```

```
    print(f'分词结果：{words}')          # ['北京', '冬奥', '的', '进度条', ...]
```

```
    # 3. 创建词嵌入层。
```

```
    # 参1：词表大小(词的个数)，参2：词向量的维度
```

```
    embed = nn.Embedding(len(words), 4)
```

```
    # 4. 获取每个词对象的下标索引。
```

```
    # i = 0
```

```
    # for word in words:
```

```
    #     print(i, word)
```

```
    #     i += 1
```

```

# enumerate(): 返回列表中每个值 及其对应的 索引.
for i, word in enumerate(words):    # 效果同上.
    # print(i, word)

# 5. 把词索引(张量形式) 转成 词向量.
word_vector = embed(torch.tensor(i))    # 随机的, 每次都不一样, 无所谓.
print(f'词: {word}, \t\t词向量: {word_vector}')

# 2. 测试
if __name__ == '__main__':
    dm01()

```

结果:

```

文本分词: ['北京', '冬奥', '的', '进度条', '已经', '过半', ' ', ' ', '不少', '外国', '运动员']
去重后词的个数:
18
词嵌入的结果:
Embedding(18, 4)
进度条 tensor([-0.3230,  0.6118,  1.1127, -1.8573], grad_fn=<EmbeddingBackward0>)
北京 tensor([ 0.4924,  1.9053,  0.5551, -0.4056], grad_fn=<EmbeddingBackward0>)
冬奥 tensor([-0.7237, -0.3153, -1.0946,  0.5241], grad_fn=<EmbeddingBackward0>)
后 tensor([ 1.7348, -0.3621, -0.1740,  0.6375], grad_fn=<EmbeddingBackward0>)
运动员 tensor([ 0.9090,  0.2903,  1.3590, -1.0113], grad_fn=<EmbeddingBackward0>)
归途 tensor([-0.8221,  0.4773,  1.0013, -0.1903], grad_fn=<EmbeddingBackward0>)
已经 tensor([ 0.2596,  1.5450,  1.7050, -0.3949], grad_fn=<EmbeddingBackward0>)
比赛 tensor([-0.9220, -1.0200, -0.3822,  0.5478], grad_fn=<EmbeddingBackward0>)
踏上 tensor([ 1.2970, -0.2169,  0.3552,  0.1369], grad_fn=<EmbeddingBackward0>)
, tensor([ 0.3914, -0.4157,  1.1152,  0.0727], grad_fn=<EmbeddingBackward0>)
自己 tensor([-1.5722, -0.1995,  0.1097, -0.4549], grad_fn=<EmbeddingBackward0>)
完成 tensor([ 0.2163, -0.9119,  1.1095,  0.8218], grad_fn=<EmbeddingBackward0>)
过半 tensor([0.9948, 2.6069, 1.5194, 0.4751], grad_fn=<EmbeddingBackward0>)
外国 tensor([-0.0833,  0.6791,  0.0911, -0.8597], grad_fn=<EmbeddingBackward0>)
。 tensor([ 0.2335, -1.3990,  0.8407, -0.5460], grad_fn=<EmbeddingBackward0>)
不少 tensor([ 0.2476, -0.5952, -0.3386,  0.7944], grad_fn=<EmbeddingBackward0>)
的 tensor([ 0.6788,  0.6474, -1.2109,  0.0276], grad_fn=<EmbeddingBackward0>)
在 tensor([-1.9989,  0.4322, -0.5250,  0.2621], grad_fn=<EmbeddingBackward0>)

```

3 循环网络层

RNN网络原理

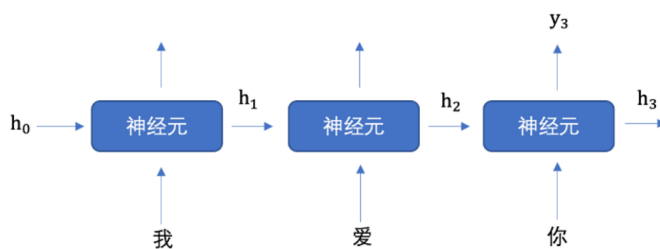
文本数据是具有序列特性的

例如: "我爱你", 这串文本就是具有序列关系的, "爱" 需要在 "我" 之后, "你" 需要在 "爱" 之后, 如果颠倒了顺序, 那么可能就会表达不同的意思。

为了表示出数据的序列关系, 需要使用循环神经网络(Recurrent Neural Networks, RNN) 来对数据进行建模, RNN 是一个作用于处理带有序列特点的样本数据。

RNN网络原理

RNN 计算过程是什么样的呢？



h 表示隐藏状态, 保存了**序列数据中的历史信息**, 并将这些信息传递给下一个时间步, 从而允许RNN处理和预测序列数据中的元素。

每一次的输入包含两个值: 上一个时间步的隐藏状态、当前状态的输入值 x

每一次的输出也会包含两个值: 当前时间步的隐藏状态、当前时间步的预测结果 y

隐藏状态作用:

1. **记忆功能**: 隐藏状态就像RNN的记忆, 它能够在不同的时间步之间传递信息。当一个新的输入进入网络时, 当前的隐藏状态会结合这个新输入来生成新的隐藏状态。
2. **上下文理解**: 由于隐藏状态携带了过去的信息, 它可以用于理解和生成与上下文相关的输出。这对于语言模型、机器翻译等任务尤其重要。
3. **连接不同时间步**: 隐藏状态通过网络内部的循环连接将各个时间步连接起来, 使得网络可以处理变长的序列数据。

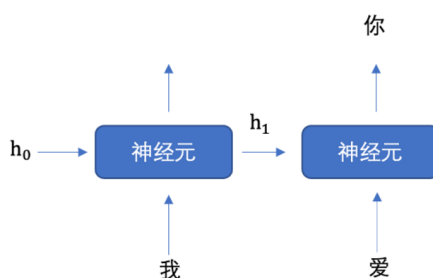
原理:

RNN网络原理

我们举个例子来理解上图的工作过程, 假设我们要实现文本生成, 也就是输入 "我爱" 这两个字, 来预测出 "你", 其如下图所示:



将上图展开成不同时间步的形式, 如下图所示:



首先初始化出第一个隐藏状态 h_0 , 一般都是全0的一个向量, 然后将 "我" 进行词嵌入, 转换为向量的表示形式, 送入到第一个时间步, 然后输出隐藏状态 h_1 , 然后将 h_1 和 "爱" 输入到第二个时间步, 得到隐藏状态 h_2 , 将 h_2 送入到全连接网络, 得到 "你" 的预测概率。

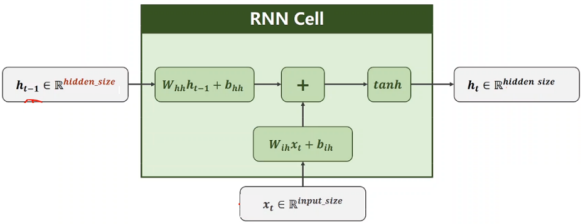
RNN神经元内部是如何计算的？

计算隐藏状态：每个时间步的隐藏状态 h_t 是根据当前输入 x_t 和前一时刻的隐藏状态 h_{t-1} 计算的。

$$h_t = \tanh(W_{ih}x_t + b_{ih} + W_{hh}h_{t-1} + b_{hh})$$

上述公式中：

- 1. W_{ih} 表示输入数据的权重
- 2. b_{ih} 表示输入数据的偏置
- 3. W_{hh} 表示输入隐藏状态的权重
- 4. b_{hh} 表示输入隐藏状态的偏置
- 5. h_{t-1} 表示输入隐藏状态
- 6. h_t 表示输出隐藏状态



最后对输出的结果使用 $\tanh$ 激活函数进行计算，得到该神经元的输出隐藏状态。

RNN神经元内部是如何计算的？

计算当前时刻的输出：网络的输出 y_t 是当前时刻的隐藏状态经过一个线性变换得到的。

$$y_t = W_{hy}h_t + b_y$$

上述公式中：

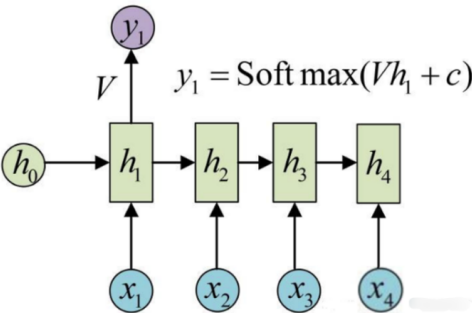
- 1. y_t 是当前时刻的输出（通常是一个向量，表示当前时刻的预测值，RNN层的预测值）
- 2. h_t 是当前时刻的隐藏状态
- 3. W_{hy} 是从隐藏状态到输出的权重矩阵
- 4. b_y 是输出层的偏置项

词汇表映射：

输出 y_t 是一个**向量**，该向量经过**全连接层**后输出得到最终预测结果 Y_{pred} ， Y_{pred} 中每个元素代表当前时刻生成词汇表中某个词的得分（或概率，通过激活函数如**softmax**）。**词汇表有多少个词， Y_{pred} 就有多少个元素值，最大元素值对应的词就是当前时刻预测生成的词。**

神经元工作机制总结：

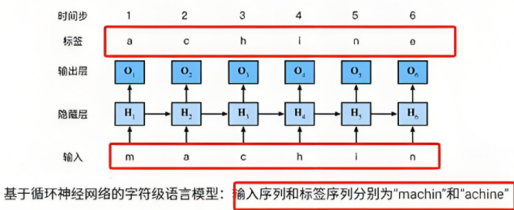
- **接收输入：**每个RNN神经元接收来自输入数据 x_t 和前一时刻的隐藏状态 h_{t-1} 。
- **更新隐藏状态：**神经元通过一个加权和（由权重矩阵和偏置项组成）更新当前时刻的隐藏状态 h_t ，该隐藏状态包含了来自过去的记忆以及当前输入的信息。
- **输出计算：**基于当前隐藏状态 h_t ，神经元生成当前时刻的输出 y_t ，该输出可以用于任务的最终预测。



文本生成示例:

假设我们使用RNN进行文本生成，输入是一个初始词语或一段上下文（例如，“m”）。RNN会通过隐藏状态逐步生成下一个词的概率分布，然后根据概率选择最可能的下一个词。

- 1. 输入：“m” → 词向量输入x1（对应“m”）
- 2. 初始化隐藏状态h0，一般初始值为0
- 3. 隐藏状态更新h1，并计算输出y1
- 4. 经过全连接层输出层计算输出ypred，使用softmax函数将ypred转换为概率分布
- 5. 选择概率最高的词作为输出词（例如“a”）
- 6. 输入新的词“a”，继续处理下一个时间步，直到生成完整的词或句子



PyTorch RNN层的使用

● API介绍

```
RNN = torch.nn.RNN(input_size,hidden_size, num_layers)
```

参数意义是:

- 1. `input_size`: 输入数据的维度，一般设为词向量的维度;
- 2. `hidden_size`: 隐藏层h的维度，也是当前层神经元的输出维度;
- 3. `num_layers`: 隐藏层h的层数，默认为1.

将RNN实例化就可以将数据送入进行处理。

代码:

.....

案例:

演示RNN层(循环网络层)的API.

循环网络层作用:

基于 上一次的隐藏状态 + 本次的输入 -> 本次的隐藏状态，本次的输出.

公式:

本次的隐藏状态 = $\tanh$ (上次的隐藏状态加权求和 + 本次的输入 加权求和)
本次的输出 = 本次的隐藏状态加权求和，有词汇表中所有词的概率，选概率最大的哪个词作为 最终预测结果.

简单总结下RNN:

词嵌入层:

将词(词的索引) 转换为 词向量表示.

RNN层(循环网络层):

逐步处理词向量，生成 每个时间步的 隐藏状态.

全连接层(输出映射):

通过线性变换将隐藏状态映射到输出，通常是1个词汇表中词的概率分布.

.....

```

# 导包
import torch
import torch.nn as nn

# 大白话：RNN就像是你的大脑，在看电影的过程中 记住剧情。

# 1. 创建 循环网络层。
# 参1：词向量的维度，就像电影里每一帧画面有128个细节(比如：颜色，动作，表情等)，RNN(你的大脑)每次看到的就是这128个细节。
# 参2：隐藏状态向量维度，你的大脑能记住的剧情信息量有多大，比如说：记住角色关系，前景提要，值越大，能存储的剧情记忆就越多。
# 参3：隐藏层数量，默认是1，你的大脑只有1层来处理它，如果是1，说明2层大脑接力理解剧情，第1层处理，第2层深加工。
rnn = nn.RNN(input_size=128, hidden_size=256, num_layers=1)

# 2. 定义变量，表示输入的 x
# 参1：每个句子的词的个数(长度)， 假设电影总共有 5帧画面。
# 参2：句子的数量， 同时有32个人在看这 5帧画面。
# 参3：词向量的维度。 每一帧画面的细节是128个。
x = torch.randn(size=(5, 32, 128))

# 3. 定义变量，记录：上一时刻的隐藏状态。
# 参1：隐藏层的层数， RNN的层数，和num_layers一样，1层大脑。
# 参2：句子的数量， 批量大小，和上述的 x一样，32个人...
# 参3：隐藏状态向量维度， 和上述的 hidden_size一样，256维。
h0 = torch.randn(size=(1, 32, 256))

# 4. 调用RNN处理，获取到当前时刻的预测值 和 当前的隐藏状态。
# 参1：x，本次的输入， 参2：h0，上一次的隐藏状态。

# 返回值1 output：每个时间步的输出，包含了所有时间步的隐藏状态。 每看1帧画面，
大脑能记住的剧情 -> 共5帧，32个人，256维。
# 返回值2 h1：最后1个时间步的隐藏状态。 看完最后1帧
画面后，大脑里的最新剧情记忆 -> 只关注最后1帧。
output, h1 = rnn(x, h0)
print(f'output: {output.shape}')    # [5, 32, 256]
print(f'h1: {h1.shape}')           # [1, 32, 256]

```

PyTorch RNN层的使用

- 输入数据和输出结果

将RNN实例化就可以将数据送入其中进行处理，处理的方式如下所示：

```
output, hn = RNN(x, h0)
```

- 输入数据:输入主要包括词嵌入的x、初始的隐藏层h0
 - x的表示形式为[seq_len, batch, input_size]，即[句子的长度，batch的大小，词向量的维度]
 - h0的表示形式为[num_layers, batch, hidden_size]，即[隐藏层的层数，batch的大，隐藏层h的维数]
- 输出结果：主要包括输出结果output,最后一层的hn
 - output的表示形式与输入x类似，为[seq_len, batch, hidden_size]，即[句子的长度，batch的大小，输出向量的维度]
 - hn的表示形式与输入h0一样，为[num_layers, batch, hidden_size]，即[隐藏层的层数，batch的大，隐藏层h的维度]

文本生成案例

项目需求

文本生成任务是一种常见的自然语言处理任务，输入一个开始词能够预测出后面的词序列。本案例将会使用循环神经网络来实现周杰伦歌词生成任务。



数据集

数据集

我们收集了周杰伦从第一张专辑《Jay》到第十张专辑《跨时代》中的歌词，来训练神经网络模型，当模型训练好后，我们就可以用这个模型来创作歌词。数据集如下：

```
想要有直升机  
想要和你飞到宇宙去  
想要和你融化在一起  
融化在宇宙里  
我每天每天每天在想想想著你  
这样的甜蜜  
让我开始相信命运  
感谢地心引力  
让我碰到你  
漂亮的让我面红的可爱女人  
...
```

该数据集共有 5819 行文本。

获取数据集并构建词表

在进行自然语言处理任务之前，首要做的就是构建词表。

所谓的词表就是将数据进行分词，然后给每一个词分配一个唯一的编号，便于我们送入词嵌入层获取每个词的词向量。

1	who
2	when
3	where
4	what
5	why
6	whose
7	which

获取数据集并构建词表

接下来, 我们对周杰伦歌词的数据进行处理构建词表，具体实现如下所示：

整体流程是：

- 获取文本数据
- 分词，并进行去重
- 构建词表

构建网络模型

我们用于实现《歌词生成》的网络模型，主要包含了三个层：

1. 词嵌入层: 用于将语料转换为词向量
2. 循环网络层: 提取句子语义
3. 全连接层: 输出对词典中每个词的预测概率

构建训练函数

前面的准备工作完成之后, 我们就可以编写训练函数。训练函数主要负责编写数据迭代、送入网络、计算损失、反向传播、更新参数，其流程基本较为固定。

由于我们要实现文本生成，文本生成本质上，输入一串文本，预测下一个文本，也属于分类问题，所以，我们使用多分类交叉熵损失函数。优化方法我们学习过 SGB、[AdaGrad](#)、Adam 等，在这里我们选择学习率、梯度自适应的 Adam 算法作为我们的优化方法。

训练完成之后，我们使用 [torch.save](#) 方法将模型持久化存储。

代码：

"""

案例：

演示RNN层(循环网络层)的API。

循环网络层作用：

基于 上一次的隐藏状态 + 本次的输入 -> 本次的隐藏状态，本次的输出。

公式：

本次的隐藏状态 = $\tanh$ (上次的隐藏状态加权求和 + 本次的输入 加权求和)

本次的输出 = 本次的隐藏状态加权求和，有词汇表中所有词的概率，选概率最大的哪个词作为 最终预测结果。

简单总结下RNN：

词嵌入层：

将词(词的索引) 转换为 词向量表示。

RNN层(循环网络层)：

逐步处理词向量，生成 每个时间步的 隐藏状态。

全连接层(输出映射)：

通过线性变换将隐藏状态映射到输出，通常是1个词汇表中词的概率分布。

"""

导包

import torch

import torch.nn as nn

大白话：RNN就像是你的大脑，在看电影的过程中 记住剧情。

1. 创建 循环网络层。

参1：词向量的维度，就像电影里每一帧画面有128个细节(比如：颜色，动作，表情等)，RNN(你的大脑)每次看到的就是这128个细节。

参2：隐藏状态向量维度，你的大脑能记住的剧情信息量有多大，比如说：记住角色关系，前景提要，值越大，能存储的剧情记忆就越多。

参3：隐藏层数量，默认是1，你的大脑只有1层来处理它，如果是1，说明2层大脑接力理解剧情，第1层处理，第2层深加工。

rnn = nn.RNN(input_size=128, hidden_size=256, num_layers=1)

2. 定义变量，表示输入的 x

参1：每个句子的词的个数(长度)， 假设电影总共有 5帧画面。

参2：句子的数量， 同时有32个人在看这 5帧画面。

参3：词向量的维度。 每一帧画面的细节是128个。

x = torch.randn(size=(5, 32, 128))

3. 定义变量，记录：上一时刻的隐藏状态。

参1: 隐藏层的层数, RNN的层数, 和num_layers一样, 1层大脑.

参2: 句子的数量, 批量大小, 和上述的 x一样, 32个人...

参3: 隐藏状态向量维度, 和上述的 hidden_size一样, 256维.

h0 = torch.randn(size=(1, 32, 256))

4. 调用RNN处理, 获取到当前时刻的预测值 和 当前的隐藏状态.

参1: x, 本次的输入, 参2: h0, 上一次的隐藏状态.

返回值1 output: 每个时间步的输出, 包含了所有时间步的隐藏状态. 每看1帧画面, 大脑能记住的剧情 -> 共5帧, 32个人, 256维.

返回值2 h1: 最后1个时间步的隐藏状态. 看完最后1帧画面后, 大脑里的最新剧情记忆 -> 只关注最后1帧.

output, h1 = rnn(x, h0)

print(f'output: {output.shape}') # [5, 32, 256]

print(f'h1: {h1.shape}') # [1, 32, 256]